



Accessibility Design Tools

Second Edition



d

Contents

Welcome	5
Foreword.....	5
Our Mission.....	6
Our Target Groups.....	8
What's New in This Release	9
Introduction	11
Shifting the Paradigm.....	11
How To Start.....	12
Respect the Global Accessibility Market.....	12
Be Aware of Fixing Violation Costs.....	12
Establish a Shift-Left Strategy.....	13
Provide Accessibility Step by Step.....	15
Foundation	35
Principles	36
Designer	36
Developer	38
Annotations.....	41
Component Name.....	42
Floorplan Name	46
Design Note	49
Visual Experience.....	52
Principles	53
Visual Impairment.....	53
General Design Tips.....	55
Annotations.....	56
Color Contrast	57
Theme	62
Responsiveness.....	65
Text Resize.....	72
Text Spacing.....	77
Tooltip	80
Interaction Experience	85
Principles	86
Limited Mobility.....	86
General Design Tips.....	88
Annotations	90
Input Mechanism.....	91
Focus Order.....	101
Initial Focus Position.....	108

Focus Restore	115
Skipping Group	120
Shortcut	125
Trigger	132
Motion Alternative	136
Minimum Target Size	140
Journey	144
Screen Reading Experience.....	148
Principles	149
Blindness.....	149
General Design Tips.....	150
Annotations	153
Reading Order	154
Label	161
Description.....	168
Live Message	175
Heading.....	180
Landmark	184
Page Title	190
Role and Properties	196
Speech Output.....	210
Audio Control.....	214
Audio Description	219
Language	221
Cognitive Experience	225
Principles	226
Neurodiversity	226
General Design Tips.....	228
Annotations	230
Wayfinding and Orientation.....	231
Semantic	236
Error Handling.....	245
Motion Content.....	250
Time Limit.....	254
Multiple Ways.....	263
Redundant Entry.....	267
Auditory Experience	270
Principles	271
Hard of Hearing.....	271
General Design Tips.....	273
Annotations	274
Caption.....	275
Transcript.....	281

Closing Remarks	288
Authors.....	289
Contributors.....	290
Appendix	292
WCAG 2.2 Coverage.....	292
Perceivable	293
Operable.....	294
Understandable.....	296
Robust	297
Conformance.....	297

Welcome



Foreword

Design serves people. As software suppliers, we need to ensure that our design accounts for the wide variety of needs arising from the variances amongst our users.

Accessibility shall be in its core. It provides the technical baseline for access by many means and ensures that software can be used at all. Creating an accessible user experience means to support different ways of perception and interaction; this also means to work below the visual surface.

Designing the non-visual experience is sometimes perceived as abstract and not tangible. But it is doable – it just needs continuous practicing and adjustments in our working routines. With SAP Accessibility for Designers, we provide you with mature guidance.

The guidelines are based on 20 years of accessibility at SAP and have been co-created by our central SAP Accessibility Competence Center and product design experts. No matter if you are new to accessibility or already an accessibility pro, we are sure you will find helpful information.

At SAP, we believe that our software can support a more inclusive future. I am excited to see your work supporting our vision.



Nicole Windmann

Head of the SAP Accessibility Competence Center
Vice President Accessibility and Inclusive Design at SAP SE



Our Mission

Enable correct implementation of accessibility from the beginning.

We designers are committed to providing delightful experiences to all users. This includes easy access to information and clear, usable, and efficient workflows to enable users to accomplish tasks. We also want to deliver effective and engaging communication.

The goal is to attend to accessibility earlier in the software development life cycle; a shift left approach. By reducing the burden of addressing accessibility during the development phase, we recommend taking a shift left approach by attending to accessibility during the design phase. Thus, we ensure that we are intentionally designing accessible experiences.

This handbook offers guidance and support to annotate accessibility during the design phase. The library of visual elements can be used to inform visual design, screen reading, keyboard support, and complementary annotations. These annotation elements can be used on pages, screens, and components.

The annotation elements are followed by descriptions and examples so that a designer understands how it impacts the users' experiences. The goal is to design for all types of users, delivering them an accessible and inclusive experience.

Do we comprehend how users interact with our applications? Do we know what accessibility means during the design phase? Do we think about what distinguishes user types? Do we consider our users' differences in ability or background so that our designs enhance their understanding and ability to work efficiently? These are just a few questions that drive this handbook. Our goal is not to prescribe a set of rules for accessible design. We will not tell you imperatively what to do but propose ways to enhance your design. Our goal is to share a practical guide that enables designers to create inclusive designs that meet the various needs of all user types.



This material offers a vast collection of assets to empower designers to annotate components and screens for correct implementation of inclusive experiences.

Irla Bocianoski Rebelo

User Experience Designer Expert and Accessibility Lead
at SAP SuccessFactors

Stefan Schnabel

User Experience Design Expert at SAP Product & Engineering

Nina Krauss

Accessibility Specialist at SAP Product & Engineering



Our Target Groups

UX Designers

As the primary beneficiaries of this content, UX designers are responsible for annotating designs to inform accessibility.

Visual Designers

.. will specify the look needed to fulfill accessibility requirements, including theming, color contrast, spacing, size, and layout.

System Designers

.. will prepare reusable components with accessibility specifications to enable an efficient work process among designers.

UX Writers

.. will review content for screen reading annotations. UX writers can address all visible or hidden content in the section “Screen Reading.”

Researchers

.. will address the essential accessibility topics that need to be examined and covered in the five groups of annotations.

Developers

.. will get clear instructions about what to use as variable values for designated control properties. They will also follow annotated design for accessibility to address responsive scenarios, follow the right sequence for screen reading and focus order, implement meaningful headings hierarchy, and more.

Quality Assurances

Annotated components being part of specifications will allow them to confirm visuals and behaviors before reporting deviations.

Product Managers

The presence of annotations in component and application designs will help them to plan and prioritize Accessibility for their strategic roadmaps.



What's New in This Release

The first edition of the SAP Accessibility Design Tools was released in July 2023. It was created to help designers focus on and work with accessibility annotations in wireframes and components and to ease the communication among designers and developers

Since then, the annotations have been consumed by designers from 160 SAP teams to create numerous accessibility design specifications with number of annotation inserts reaching up to 10K in a single week.

Annotations like keyboard keys, reading order, focus order, component name, and role are the most popular annotations and together they have been used over 90,000 times in the past year.

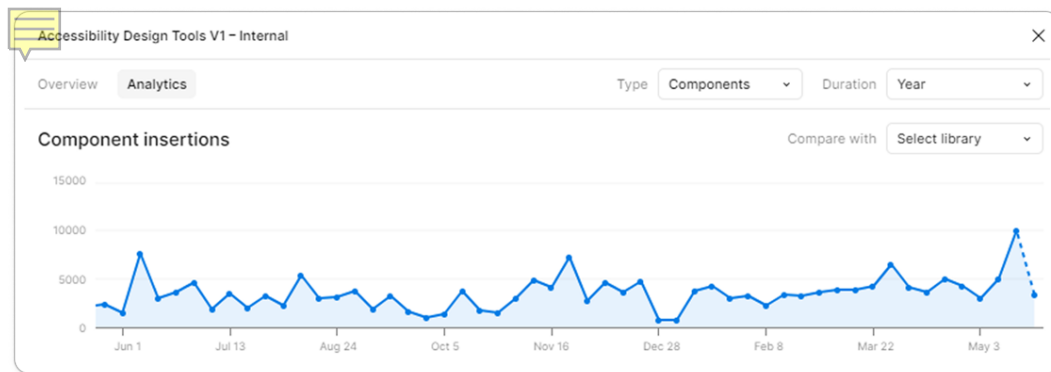


Figure 1: Accessibility Design Tools Usage June 2023 - June 2024

This revised second edition of the SAP Accessibility design tools contains various enhancements for existing annotations and lists numerous new annotations. Information for developers is now also part of the annotation documentation. Quotes from designers using the annotations were added, too.

A refined and extended documentation structure covers a broader scope of accessibility requirements and user groups.

Grouping annotations per persona represents users of similar disabilities, creates empathy and makes the design annotation work more user centric and humane.

Annotation examples were enhanced and extended to cover more use cases. Occasional use of grey boxes or skeletons as placeholders removes



distractions and keeps the focus on the structure, interaction, and semantic meaning of elements.



*Ēāuō AηηōϠηŋŋŋ Œ ōŦ ſi âBBŦŦ Ľ Ľ ŸŦŦŦ Ŧō ŦŦ
ŦŦŦŦ ŦŦŦŦ Ŧ ŦŦ ŦŦŦŦ Ŧ ŦŦŦ ŦŦŦŦ ŦŦŦŦ ŦŦŦŦ ŦŦŦŦ
ŦŦŦŦ ŦŦŦŦ ŦŦŦŦ ŦŦŦŦ ŦŦŦŦ ŦŦŦŦ ŦŦŦŦ ŦŦŦŦ ŦŦŦŦ
ŦŦŦŦ ŦŦŦŦ ŦŦŦŦ ŦŦŦŦ ŦŦŦŦ ŦŦŦŦ ŦŦŦŦ ŦŦŦŦ ŦŦŦŦ
ŦŦŦŦ ŦŦŦŦ ŦŦŦŦ ŦŦŦŦ ŦŦŦŦ ŦŦŦŦ ŦŦŦŦ ŦŦŦŦ ŦŦŦŦ*

Marvin Gille – UX Designer & Accessibility Advocate

Introduction

Shifting the Paradigm

Design beyond the visible

Strong usability and interaction principles aren't just good design — they're the foundation of accessibility on any platform, desktop or mobile.

A designer truly committed to accessibility thinks beyond the visuals — they annotate designs to ensure that every user, including those with low vision, cognitive challenges, or limited mobility, is considered from the start.

But here's the truth: designing for the invisible experience — what screen readers and assistive technologies interpret — is often the trickiest part.

The same is true for keyboard design, where the entire experience depends on a logical, intuitive focus order and visible cues that guide users without relying on sight or a mouse. That's where we come in.

We'll show you how to turn visible UI elements into a clear, well-organized invisible structured announcement that supports inclusive, intuitive screen reading essential for navigation and interaction, whether a user is using a mouse, keyboard, or gestures.

By understanding how users rely on visible cues to navigate and interact, you'll gain a powerful ally in creating inclusive experiences — from your first design draft to final delivery.

This thought-provoking introduction invites you to shift your mindset toward more inclusive and accessible design. Embrace accessibility as a core design principle accepting the challenge to rethink your design approach.



How To Start

Respect the Global Accessibility Market

Teams don't integrate accessibility into development and Design due to a lack of awareness, expertise, prioritization, or maybe because the global accessibility testing market size is huge. It is estimated to grow over 50% in the next 10 years - From USD 564.73 M in 2023 to USD 825.43 M in 2032.

This reality is supported by where accessibility services available in the market are focused on: audits, automated testing, and manual evaluations. This reactive behavior is expensive.

Be Aware of Fixing Violation Costs

Design sits at the earliest stage of the software development lifecycle, making it the most effective point to embed accessibility and prevent future defects.

Addressing accessibility during design is not only good practice—it's also strategic. Waiting until later stages can lead to costly remediation efforts, poor user experience, and legal risk.

According to industry estimates, defects found during design may cost 1x to fix, while issues discovered post-release can cost up to 100x more. For accessibility, this principle is particularly relevant—early planning can prevent expensive rework and ensure compliance with laws like the ADA. In fact, digital accessibility lawsuits in the U.S. jumped by over 180% between 2017 and 2018, emphasizing the growing legal and user demand for inclusive digital products.



Establish a Shift-Left Strategy

Shifting accessibility efforts left ensures that your team creates inclusive experiences from the start—reducing risk, improving quality, and empowering all users.



Figure 2: Shifting from just detecting defects helps avoiding them proactively

A Shift-Left strategy brings accessibility and inclusion to the forefront of design and research, embedding them early in the process rather than waiting until development or post-release phases. By planning inclusive experiences from the beginning, teams can prevent accessibility issues before they arise, avoid costly retroactive fixes, and ultimately create better experiences for all users.

When usability is carefully considered from the start, it helps prevent long debates later about how to "fix" accessibility. And if questions do come up, return to your usability foundations and ask: What more can I do to improve accessibility for everyone?

This approach empowers designers and researchers to define inclusive interactions upfront, based on user needs, behavior, and expectations. With clear design and accessibility specifications, developers can implement with greater confidence, efficiency, and accuracy—resulting in a more robust and accessible product from the start. The shift Left strategy also relieves developers from having to resolve issues late in the cycle.

Testing and audits hold a strong position in the market. Accessibility specialists have a deep and structured understanding of accessibility requirements, international standards, and assistive technologies. Shifting from just detecting defects to proactively avoiding them is a true advancement for accessibility. A Shift left approach for designers is an opportunity to integrate accessibility from the start - during the design phase.

To achieve that we need to follow the steps of those who audit systems for accessibility. Testers had undergone rigorous training and have extensive hands-on practice. They translate abstract guidelines – like WCAG – to

identify defects and suggest improvements. They have the clarity, and the precision needed to identify, document, and guide teams toward meaningful accessibility enhancements —expertise that many designers and developers lack.

Designers perceive accessibility as a fuzzy requirement – it's vague and secondary. To do the shift left let's address the basic concepts of accessibility, understand the accessibility life cycle, and get prepared to work on accessibility documentation.



Provide Accessibility Step by Step

1. Improve Your Skills

Know the History of Accessibility

The trajectory of digital accessibility and its history starts in 1960 with a USA congress Act in the Office of Education.

The Rehabilitation Act of 1973 marked a pivotal moment by prohibiting discrimination based on disability in federally funded programs. Building on this foundation, the Americans with Disabilities Act (ADA) of 1990 extended accessibility requirements to public spaces and employment.

With time the concept evolved with regulations for physical spaces and expanding to digital products in 1998.

The evolution continued with the Web Content Accessibility Guidelines (WCAG), introduced to address digital barriers and ensure inclusivity in web content.

Table 1: History of Accessibility

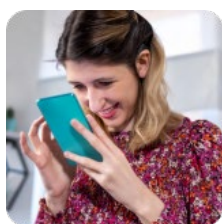
1960 Special Services in the Office of Education created by an Act of Congress	1973 Rehabilitation Act	1990 Americans with Disabilities Act (ADA) ensures equal opportunity for persons with disabilities	1996 Department of Justice (DOJ) & digital accessibility
1998 US Congress amended the Rehabilitation Act with Section 508 enacted to address digital content	1999 Web Content Accessibility Guidelines WCAG 1.0	2000 Maguire vs Sydney Organizing Committee for the Olympic Games (1999/2000)	2005 VoiceOver released for Apple products
2008 WCAG 2.0 released	2014 EN 301 549 introduced	2018 WCAG 2.1 released (51 Success Criteria)	2022 European Accessibility Act
2023 WCAG 2.2 Release	2025 European Accessibility Act		



Understand User Groups and Disabilities

When designing for accessibility, it's essential to recognize the diverse range of users who benefit. This includes people with disabilities—grouped by shared characteristics such as vision, mobility, hearing, or cognitive differences.

But accessibility also supports many others: people without permanent disabilities who face temporary challenges, like a broken arm, bright sunlight, or a noisy environment. By considering these overlapping needs, we create experiences that are more usable for everyone.



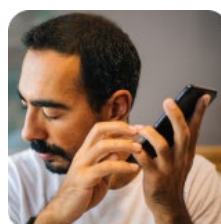
Limited Vision

The **visual experience** explores crucial factors to keep in mind when designing for people with limited vision like being color blind, having short vision, tunnel vision, etc.



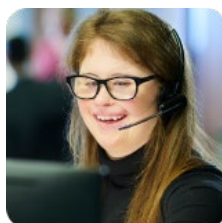
Limited Mobility

The **interactive experience** addresses what to look for when designing for users with limited mobility. They may find it hard to control different input mechanisms like a mouse or touch devices.



Blind or Very Limited Vision

The **screen reading experience** dives into what should be considered for screen reader users that may be blind or with very limited vision.



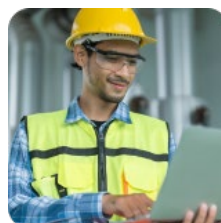
Limited Vision

The **cognitive experience** highlights important aspects to focus on when designing for people with cognitive issues like concentrating, remembering or learning.



Deaf & Hard of Hearing

The **hearing experience** addresses aspects to look for regarding users that are deaf or hard of hearing, like supporting captions, transcripts or sound alternatives.



Situational and Temporary Disabilities

These users benefit from accessible features when they find themselves in **situational or temporary** contexts.

The SAP Accessibility Design Tools propose a simplified approach to covering the 50+ WCAG requirements by introducing personas that represent groups of people with similar disabilities.

This strategy aims to help designers and other stakeholders build empathy with end users who have specific accessibility needs. By aligning WCAG

requirements with key personas, we enable a more efficient design documentation process—supported by checklists and practical guidance associated to these personas.

In this strategy, each persona acts as a bridge to understanding real user needs, helping ensure that accessibility is integrated with intention and consistency.

Situational & Temporary Disabilities

We must expand what we understand about usability to address invisible experiences. This necessitates providing conditions for all types of users to enjoy the experience yet perform tasks safely, effectively, and efficiently.

Temporary and situational disabilities are different from permanent disabilities in terms of their duration and impact on the individual's ability to interact with the environment, including digital products. Including "situational" disabilities in each of the five main disability groups, helps designers recognize that situational or temporary limitations often overlap with permanent conditions, therefore, requiring special attention during the design phase.

You will find a note in each chapter explaining how that disability group can be expanded to include situational disabilities.

- Permanent Disabilities are conditions that last for a long time or indefinitely and typically do not change over time. Individuals with permanent disabilities may require long-term adjustments, accommodations, and assistive technology to navigate the world effectively. They are often present from birth, because of an accident, or due to an illness and remains largely constant during lifetime. This requires chronic need for support and continuous specialized devices, tools, or assistive technologies (e.g., screen readers for blind users, hearing aids for deaf users, prosthetics for individuals with limited mobility).
- Temporary Disabilities occur because of specific, short-term events that impair someone's ability to interact with their environment for a limited period. These can happen due to injury, illness, or external factors. After a while, the individual typically returns to full function or a baseline level of ability.
- Situational disabilities are context-specific impairments that arise based on an individual's current environment, circumstances, or actions. They are not tied to a permanent condition but occur due to factors like the physical context (e.g., lighting, noise) or temporary emotional/mental states (e.g., stress, fatigue).



By understanding the differences between permanent, temporary, and situational disabilities, you can design more flexible and inclusive experiences that accommodate a wide range of users, even if their needs change over time or depending on external conditions.

The table below shows key contrasts between permanent, temporary, and situational disabilities.

Table 2: Permanent, temporary and situational disabilities

	Permanent disabilities	Temporary disabilities	Situational disabilities
Duration	Lifelong or long-term	Short-term (recoverable)	Variable, based on context (can be momentary or hours-long)
Impact on Daily Life	Consistent and ongoing adjustments are needed	Short-term adjustments for a limited period	Context-specific, often requiring temporary adjustments for specific tasks
Recoverability	Not recoverable or only partially recoverable	Full recovery possible with time or medical treatment	No long-term recovery needed; situation resolves when context changes
Examples	Blindness, permanent mobility impairments, chronic hearing loss	Broken limb, temporary vision loss, concussion	Sweating hands, fatigue, stress, temporary loud background noise
Design Focus	Assistive technologies and long-term accommodations	Short-term adaptability and accommodations for a fixed period	Context-aware design and flexibility based on situational needs
Remarks	Often require persistent accommodations and assistive technologies (e.g., screen readers, sign language interpretation, alternative input methods).	Require short-term solutions or assistive devices (e.g., voice-to-text for a person with a broken hand, large-button keyboards for people recovering from surgery).	Can be mitigated by adaptive and responsive design, allowing users to adjust settings or interfaces depending on the environment or mental state (e.g., offering high-contrast mode in a bright environment or one-handed mode when holding something).



Think Beyond Assistive Tools Support

Popular misunderstandings include:

- Supporting Accessibility is just supporting assistive technology
- All users with disabilities will use tools to access content
- “We do it on top for assistive tech users”

In reality,

- Accessibility is not just about assistive technology compatibility. It is fundamentally **built in** and not **bolt on**.
- Not everyone with a disability uses a tool. All users benefit from good design.
- "Doing it only for assistive tech users" misses the broader inclusion impact.

Accessibility is design quality. It's usability under diverse conditions. It's adaptability for different contexts, bodies, and minds.

When accessibility is discussed in projects, some stakeholders quickly equate it with screen reader compatibility or keyboard navigation — tools commonly associated with assistive technologies.

While these are critical, it's a misconception to think accessibility is only about supporting assistive tech users.

Let's break that down by looking at common assistive technologies and beyond — because not all disabilities require assistive tools, and not all accessibility needs are solved by supporting devices alone.



Table 3: Disabilities, needs and assistive technologies

Blind Users	Low and Limited Vision	Limited Mobility	Cognitive Impairments	Hearing Impairments
Assistive Technologies: Screen readers (e.g., NVDA, JAWS, VoiceOver) Braille displays Voice assistants	Assistive Technologies: Screen magnifiers High contrast settings Zoom and browser tools	Assistive Technologies: Switch devices Alternative keyboards Voice control (e.g., Dragon NaturallySpeaking) On-screen keyboards	Some will use: Text-to-speech Distraction blockers Simple interface overlays	Many will use: Hearing aids Cochlear implants Live captioning or transcripts
But it's not enough to say: "We support screen readers." If the content structure, headings, labels, and interaction flows are confusing or inconsistent, even the best screen reader can't create a good experience. And sighted users relying on keyboard navigation (e.g., due to temporary impairments or preferences) are impacted too.	Yet, design choices like tiny text, low contrast, or hidden focus indicators hurt far more than assistive tech users. Many older users or people using mobile in bright sunlight benefit from accessible visual design — without needing specialized tools.	But someone with a repetitive strain injury might just use a standard keyboard but avoid using a mouse. If buttons can't be reached via tab, or if timeouts interrupt the task, it's not about technology — it's about usability.	But many users with ADHD, dyslexia, or memory challenges don't use tools at all. They rely on clear layouts, plain language, consistency, and reduced cognitive load — none of which depend on any assistive tech.	Still, captions help everyone — in noisy environments, while multitasking, or learning a second language. And if alerts rely only on sound without visual cues, you're excluding a lot more people than you think.



Study Differences between Web, Tablets and Mobile Devices

Users interact with digital products across many devices — desktops, tablets, and mobile phones — using a range of input methods like touch, keyboard, mouse, or voice, and receiving information through various outputs such as visuals, audio, or haptics. They rely on different senses to understand and interact with what you design.

We begin this section by explaining input and output devices and why they matter for accessibility. Before adding annotations, designers should understand how users interact with different devices to make thoughtful, inclusive decisions.



WEB Desktops devices are mostly use with mouse and keyboard - but some may work with touch and gesture.



Tablets and Mobile Devices common interaction is based on touch and gestures - but some users plug keyboards to work with them.

Figure 3: Different challenges on different devices

Most accessibility annotations apply across web and mobile platforms. While the core principles remain the same - making content perceivable, operable, understandable, and robust - each platform has its unique challenges.

Native mobile devices, for example, introduce specifics such as:

- Touch interactions using fingers of all sizes
- Small screens needing to fit complex content
- Support for both portrait and landscape orientations
- Built-in assistive technologies like VoiceOver (iOS) and TalkBack (Android)

Some annotations differ for mobile - especially those related to screen reader behavior. For instance, mobile apps don't use web standards like ARIA landmarks, and heading structures are simpler (usually a single level).

Mobile operating systems rely on gestures and touch for navigation, while screen readers provide audio feedback. Though annotations like Keyboard Support are more common for web, they also apply to mobile in cases where users connect external keyboards.

Other annotations - such as those for time limits, labels, and roles - apply universally across digital products, regardless of the device.

Perform Automatic and Manual Testing

Accessibility isn't complete until it's tested. During the testing phase, accessibility testers validate that digital products are usable by people with disabilities.

Automated testing can catch about 20–30% of accessibility issues. Tools like Axe, Lighthouse, or WAVE can detect such as missing alt text, low color contrast, missing form labels, ARIA misuse, keyboard traps, and missing document landmarks.

But automated tools can't evaluate whether alt text is meaningful, or if the focus order is logical, or if text in links or buttons is clear and descriptive. Manual testing is essential for the remaining 70–80%.

During manual testing issues pertaining to usability with screen readers, ensuring ARIA and semantic markup is correct, confirming captions, labels, and alt text are present, check dynamic content updates, and overall user experience for different disabilities are reported.

It includes also keyboard-only navigation, screen reader checks, visual inspection for layout and zoom and use by people with disabilities (when possible).

Combining automated and manual testing for full coverage is key to check all accessibility requirements. Automation gives speed; humans give insight.

Some tools used to conduct manual testing are available to many as built-in assistive tool. Screen readers can be found on web desktops and mobile phones.

Design Context

A designer is not expected to conduct all these testing tasks. Accessibility annotation will drive implementation and prevent many accessibility defects. A designer who wants to check the outcome of a designed experience after implementation could use some testing tools.

Some resources for designers to test an implemented application are available online as browser extensions. But many checks won't require any tool, as they can be conducted manually. The annotation design checklists can be used as a reference to plan a design test.

All operational systems (Windows, iOS, Android, etc.) offer a set of configurations to adjust visual elements, including color contrast, theming,



and text spacing. Explore these settings and conduct visual testing of your application adapted with presentation of UI elements and behaviors with unique characteristics.

Built-in assistive technologies for testing on desktop computers and laptops include Narrator screen reader on Windows and Voice Over on Mac and iOS.

For mobile devices, there is the screen reader Talk Back on Android and Voice Over for iOS.

Chrome supports various automatic accessibility checks, including Lighthouse which is built into its DevTools. Open it on Apple using OPTION+CMD+I and on Windows using F12.

More features are offered by separately installable tools on Windows, notably Jaws and NVDA screen readers.

Finally, there are several browser extensions for testing accessibility available such as for [Color Contrast](#) or [Landmark](#) and [Heading](#) structure analysis.

Developers typically use these tools. Still, designers can and should use many features because a bug caught with an automated tool might indicate a bad interactive or navigational experience that can be examined manually.

Figma Support

The sets of annotations for accessibility offer a confident strategy for designers to create accessible designs. But annotating every UI element can be time-consuming. Using automated tools supported by plugins reduces the time burden and increases efficiency.

The Figma community offers many plugins, including free and paid plugins. Here are some of them:

- [Contrast](#): Check color contrast between two elements
- [Able](#): Check color contrast
- [Color Blind](#): Simulates different types of colorblindness
- [Text Resizer Accessibility Checker](#): Simulate readability by increasing or decreasing font sizes
- [Stark](#): A paid plugin that offers color contrast check, typography, alt-text, focus order, Landmarks and more.
- [Evinced](#): A paid solution that provides full support including annotations, accessibility design report, and design analysis for single elements and full design.



Figma Accessible Prototype

There are Figma resources to create accessible prototypes supporting NVDA, Jaws, and VoiceOver - no narrator for now. Here is a short list of how to prepare your prototype to be accessible:

- Well-organized layers enable screen reading navigation by region (landmarks).
- Alternative text, same as invisible description annotation suggested in this guideline, allows images to be announced during screen reading exploration. Text is based on the shape layer name.
- Prepare interactive elements to be activated by ENTER or SPACE using the "onclick" event when creating a prototype. These elements will be rendered as buttons and indicate interactivity.
- Open overlays simulating modals like dialogs and pop-ups. Simulate a state using the "Change to" feature for component variants. The state will be announced on screen readers when interacting or exploring a button or a checkbox.
- Scrolling content using arrow keys while using VPC mode (Screen Reading).
- Lists are announced with the number of elements when the user enters the list.

Unfortunately, focus for keyboard support and TAB navigation is not well supported yet. Nonetheless, this is an opportunity for designers to test the accessibility of their designs before implementation.

We recognize that some might perceive designing for accessibility as an increased workload. However, we believe this is an opportunity to consider the benefits of addressing accessibility during the design phase. With increased knowledge and available tools, designers can better prepare to design for all user types.



2. Develop a Strategy

List components used on the application, floorplans, and pages. This will set the basis for consistent re-use of structural elements (components and floorplans), allowing you to annotate them only once.

An inventory of components reduces the amount of work needed to annotate designs and results in less accessibility defects reported during the testing phase.

Create Blueprints as Foundation

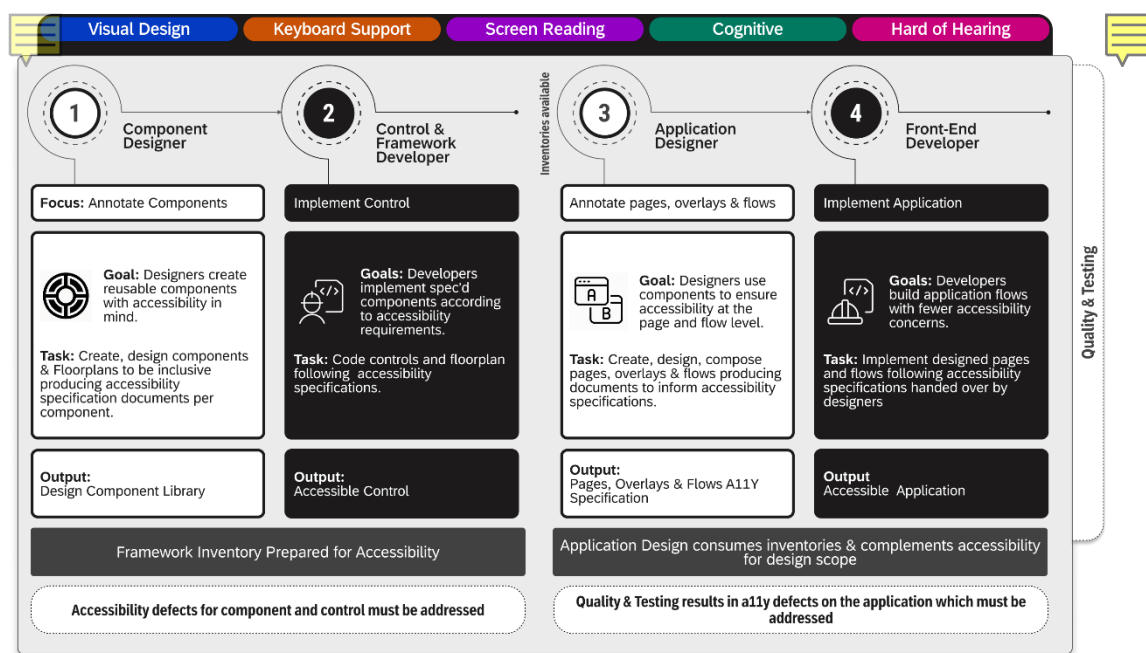


Figure 4: Infographic relationship between design and development

Blueprint is not just a static diagram — it's a strategic foundation for collaboration, consistency, and accountability.

A blueprint is a detailed plan or guide that outlines the structure, components, and relationships of a system before it's built. In product development, specially design and accessibility, a blueprint serves as a shared source of truth that informs how something should function and how it should be constructed across teams.

To think Accessibility as a Building Blueprint, think of accessibility like the electrical wiring plan in a building blueprint. You don't see it at first glance, but it's critical to functionality, safety, and the experience of anyone who

enters. If it's not designed correctly from the start, retrofitting it later becomes complex, costly, and risky.

Just like an architect collaborates with engineers and electricians early on, designers collaborate with developers and technical writers. Accessibility must be integrated at the design phase — ensuring all components, layouts, and interactions support inclusive use from the ground up.

The accessibility blueprint of your product is the result of a thoughtful process that includes building detailed inventories and aligning closely with engineering. These artifacts are shared across teams — from QA and front-end developers to technical writers and product managers — ensuring everyone is working from the same foundation.

Easy access to well-documented components and controls significantly improves efficiency. When Component Designers define reusable components with accessibility in mind, developers can implement them consistently and confidently. This empowers Application Designers to use those components knowing accessibility has been addressed at the component level, allowing them to focus on ensuring accessibility at the page and flow level.

Front-end developers can then build application flows with fewer accessibility concerns and less need for rework or troubleshooting.



Leverage Libraries and Inventories

Plan, Prepare and Share

Extensive libraries of design components and implementation techniques are powerful tools for designers, front-end developers, and engineers. They help create consistent user experiences and significantly improve efficiency in meeting accessibility requirements.

A design component library should include accessibility annotations directly in the documentation, making it easier for designers to incorporate inclusive practices from the start. In parallel, an accessible control library used by developers should offer configurable properties to adapt the controls to specific contexts without compromising accessibility.

The biggest advantage of these libraries is efficiency — the ability to produce high-quality work with less effort. Efficiency in design and development thrives on reusability, and accessible libraries make it possible to work faster while maintaining consistency and quality.

To maximize performance:

- Ensure easy access to component libraries with documented accessibility specifications.
- Make control libraries readily available and confirm they are implemented with accessibility in mind.
- Use consistent visuals, behavior, and terminology. UX Writers can support this by reinforcing shared language across teams.

When component libraries (used by designers) and control libraries (used by developers) are aligned — with matched naming conventions and consistent interaction patterns — everyone involved can work more confidently and effectively.

To support this alignment and drive efficiency, maintain three key inventories:

- Floorplans – Reusable layout templates for pages
- Components – Reusable UI elements, including core and custom components
- Pages – A list of all implemented pages in the solution, forming the ultimate blueprint of your application

As noted in the WCAG 2.1 documentation: “WCAG 2.1 will be supported by an extensive library of implementation techniques and educational materials.” This reinforces the value of well-maintained, accessible libraries as an essential part of any inclusive design and development workflow.



Design Library



Specification

To speed up the work while assembling pages, designers use components ready to use and directly available from a central library. Sometimes, more than one library is available (like core and customer components). The Material Design by Google is an example of a core library. The component contains visual design specifications. In Figma, reusable components can also contain predefined behaviors, making the creation of functional prototypes more efficient. Designers are owners of these libraries.

Control Library



Fundamental

Implementation

This library contains controls (implemented components) carrying all visual aspects needed to fulfill accessibility requirements, like color contrast. It may include behaviors for interaction and navigation, screen reading, and keyboard support. Engineering is the owner of this library, and designers consult controls to check look and behavior.

Design Guidelines



Fiori Design Guidelines

Direction

Design guidelines are the base of a design system. They provide designers and developers with valuable references to support their design decisions as they build new controls. An example are the SAP Fiori Design Guidelines.



*Ե՞ս Միտ ԲԼՈՂ նորՄԱՆՈՒՄ օժՄԻ ֆի ՅԵՅՈժիւի ֆիճՅ
նձ ԼՔԻ Ի ֆիօ Ի նդԻՈՂ իՄ ղՏ ԽՍԻՏ Ի նԲֆիՄԹֆիՅԶԱՎՄ
մԽօ Մի ԼՔօ ձ ֆիՏ ԽՕՏ օ մսօֆի նԲձ ղԲԼ ՄիՏ մԽ
ՅԻ ֆԼՂԷ*

Sup Suh – Senior Product Designer

3. Establish an Accessibility Design Process

Use Annotations

Annotating accessibility in the design phase requires diligence and preparation. Yet, remembering and applying every accessibility aspect relevant to the design phase is a heavy cognitive task.

Start planning and aligning with product management. Then, plan the accessibility blueprint of your product as described in the accessibility for designer's infographic.

It explains why it is essential to understand the difference between the component design library used to produce designs and the control libraries used by front-end developers to code the application.

To help you accelerate accessible designs, we propose an annotation set for components to address accessibility. A designer supported by a library with accessible components contributes to an consistent inclusive product. Use checklists to track accessibility guidelines and ensure all aspects are considered during the design phase.

This guideline aligns with and covers many WCAG standards and international requirements to annotate designs. It covers the many aspects needed to ensure that you deliver an accessible product, including the four groups of requirements from the Web Content Accessibility Guidelines (WCAG 2.2): Perceivable, Operable, Understandable, and Robust.

There are a variety of elements to annotate screens and components to inform accessibility considerations during the design phase.

To annotate designs and create accessibility specifications, follow the checklists and use the annotation assets from the library for a precise design documentation. Use the annotations pointing them to an element or highlighting a specific area. This can be done manually or using available plugins (screen reading, focus order, landmarks, and headings).

Work with Checklists

In this book, there are six main groups of annotations to support an accessible design, and they are available as checklists.

A checklist is a good tool to guide designers and track the progress of fulfilling accessibility through annotations. Eventually, annotations will become a natural task during the design process and help you comfortably talk about accessibility and usability.



The Accessibility Checklists by SAP were created to guide designers and help them observe and consider every accessible requirement during the design phase and when creating documentation. The items on the list can be updated to different statuses: Work in Progress if you are currently working on it; Done if the annotation is complete; Blocker if you wait a decision; Not Applicable if annotation does not apply to that design; and Review if the criteria need attention.

Plan Design Reviews

One last thing! Plan for accessibility design reviews. An A11Y expert provides a critical eye on the design specifications, ensuring all accessibility standards are met and usability is optimized for everyone. Incorporating accessibility into design reviews not only fosters inclusivity but also enhances usability, ensuring a more seamless experience for all users.

Annotating accessibility in the design phase requires diligence and preparation. Yet, remembering and applying every accessibility aspect relevant to the design phase is a heavy cognitive task. Here are a few steps design teams can use to ensure that the design is prepared to avoid violation like blockers and barriers.

1. Alignment

Start aligning with product management, development, and testing teams to ensure a shared understanding of accessibility goals, constraints, and timelines. This early collaboration helps clarify expectations, surface potential blockers and barriers, and ensure accessibility is embedded into user stories, technical planning, and testing coverage from the outset.

It's also the moment to establish a common accessibility vocabulary and agree on how issues will be categorized—such as “potential barriers” (which may impact usability for some) and “potential blockers” (which can prevent access entirely)—to streamline communication and prioritization throughout the process.

The alignment is also essential to understand the difference between a component design library used to produce designs and the control libraries used by front-end developers to code the application.



2. Accessibility Blueprint

Prepare the design specification using the accessibility checklists - this will be the product blueprint. Checklists help the designer to track every accessibility requirement needed during the design phase. This collection of accessibility annotations and the guidelines aligns with numerous WCAG standards and international accessibility requirements for design annotation.

To help you accelerate the creation of accessibility specification consider a central documentation where components also address accessibility and are annotated to inform possible configurations and usage to meet visual, interactions and, cognitive and screen reader requirements. A designer supported by a library with accessible components contributes to an assertive inclusive product.

Design review is a second pair of eyes in the design specification to ensure accessibility has been fully addressed. The A11Y design expert can use a card to collect a note from the review to inform design decisions that could become an accessibility defect reported by Quality team or in future by customers. These Developer Notes can be used as a plan for future fixes informing dependencies, upcoming requirements or deprecation.

3. Informal Check with Experts

Consultations act as informal yet valuable checkpoints with accessibility design experts throughout the design process. These quick, collaborative discussions provide timely feedback, help clarify uncertainties and ensure that accessibility considerations are thoughtfully integrated before formal reviews. By making consultation part of the workflow, teams can prevent issues early, build confidence in their decisions, and reinforce a shared understanding of inclusive design best practices.

But most importantly, consultations help guide designers in shaping a clear and complete accessibility design specification—an essential handoff document for developers and testers.

These expert touchpoints ensure that annotations are meaningful, precise, and aligned with technical expectations, reducing ambiguity and interpretation errors during implementation. This shared clarity between design and development fosters consistency across teams, supports more efficient testing, and ultimately leads to a more robust and accessible end-to-end user experience.

4. Formal Review



Final check of an accessibility specification before implementation.

The final check, or Accessibility Design Review, is a second pair of eyes in the design specification to ensure accessibility has been fully addressed. It's an assessment of the design annotations to identify potential blockers or barriers that may be reported by the testing team at the end of the software development lifecycle -or worse, by customer after release. This process aims to ensure that the design is usable and navigable by all individuals, adhering to accessibility standards and best practices using the A11Y checklists.

Checklists are also helpful to support the expert during the review as they track accessibility requirements and ensure all aspects were considered in the design specification. By following the checklists, designers can confidently address the critical elements needed to ensure an accessible product—covering all four core principles of WCAG 2.1: Perceivable, Operable, Understandable, and Robust.

Initiate a review by signaling that a design is ready for accessibility review. This step typically follows the completion of annotations and core user flow definitions. The operational model you choose—whether through design system labels, workflow tools, or direct communication—should clearly indicate that an A11Y Design Expert is needed to assess the design. Formalizing this step helps teams track how accessibility is being addressed throughout the design phase and builds accountability. It also ensures that potential accessibility issues are identified and resolved early, reducing delays and rework during development.

6. Review Outcome

Once the A11Y Design Expert completes the review, the design is marked as Reviewed. This means that the design has undergone accessibility review, and all known annotations have been provided.

However, the reviewed status does not imply that the design is free of issues. Instead, it comes with Developer Notes and documented observations, which inform the team about potential accessibility violations identified during the review. These Developer Notes are essential for tracking, to support transparency cross-team coordination, and prioritizing accessibility concerns.

These typically fall into two categories:

- **Potential Blockers** – Issues that would prevent people with disabilities from completing tasks or accessing key content. For example, if keyboard-only users cannot activate or reach a feature, the task becomes fundamentally inaccessible. Such issues must be



resolved before development. These must be resolved before development to avoid creating inaccessible features.

- **Potential Barriers** – These refer to accessibility violations but could be confused with usability concerns that may make it difficult or inefficient for users with disabilities to understand or complete tasks. For instance, if screen reader users cannot locate key information due to missing landmarks or inconsistent headings, they may become disoriented or give up. Beyond usability or clarity issues, these may hinder, confuse, or slow down users with disabilities. These are important to address but may be prioritized differently or planned for future releases.

The goal of this reviewed status is to inform all stakeholders—including product managers, developers, and testers—about where accessibility risks exist so that mitigation strategies can be planned. It also supports shared accountability by aligning expectations and enabling informed decisions about what to fix now and what to log for future releases.



*Ėāuō nā Ľā ŪΦΛβλŭōЯuōzΦЯŭōfiЭт fflī fir
лѠă ЭВfiЯ ĽЭŭflФĭ ηЭŪzŷō ĭ ôBfflŭiB ôεōηЭ
Bfiō ĭ fiôŭit ηĭ fi ΦΛβλōfiЭ ηuĭ ŭi Bĭ ĭ ηηōЯŭŭŭ
ŪЯŌЯĭ fiô Я лō Яŭ fiŭ ηĭ fiЭfflō ĭ fiô ôεBLĐĖ*

Sanket Kundu - UX Design Associate

Foundation

Imagine a developer who doesn't just implement designs, but enables access. Each spec isn't just layout—it's a set of cues for people navigating by touch, voice, keyboard, or screen reader. Build it like someone depends on it—because someone does.

This chapter helps designers to connect design and development.

The Inclusive Foundation Experience

Meeting the needs of developers to code a fully accessible experience

Do you want to create a culture of accessibility for designers? Nurture a mindset of accessibility by providing a source of truth for designers to reference components used on the designs. The main benefits associated with this strategy is that Designers will trust accessibility is considered in the foundational controls used by developers.

Another benefit is the developer confidence during implementation as they will use known controls to create the accessible experiences. And lastly, users will explore and interact more confidently, knowing they can trust patterns and consistent behaviors across the product.

If you have started an annotation effort in your group, you may have discovered the importance of one source of truth for designers and developers to follow. Strategic design starts from the base where annotated components are part of a reusable library of components.

The best practices presented in this section focus on basic information to build consistent interaction and navigation styles. The annotation proposed provide basic understanding of the layout and flow.



Principles

Designer



Persona: Dan

“I create the design specification using SAP Design Tools for Accessibility to give a solid basis for an accessibility-ready implementation”

General Design Tips

1. **Make Clear Component Definitions:** Ambiguity in component naming or mismatches between design libraries and code libraries can lead to rework or inaccessible implementations.

Design Tip: Use consistent named annotations for all components (e.g., Button, Dialog, Table Row). Reference design system tokens and document expected behaviors clearly.

2. **Use Reusable Layout Structures (Floorplans):** Developers waste time reinterpreting layouts when reusable page structures (e.g., navigation, headers, footers) are not identified.

Design Tip: Annotate floorplan names in design specs and include expected ARIA landmarks (e.g., main, navigation, banner) to simplify implementation and improve accessibility.

3. **Annotate Accessibility Behaviors:** The challenge is associated to missing details about keyboard focus, ARIA roles, or label associations lead to inaccessible code or extra guesswork.

Design Tip: Provide accessibility annotations for screen reader expectations, keyboard flows, focus return, live regions, and error states directly on the design.

4. **Focus on Minimal Change Sets:** When design updates are broad or lack clear scoping, it becomes difficult for developers to identify what specifically needs to be implemented, leading to inefficiencies and missed details.

Design Tip: Use Delta Design Annotations to highlight what changed. Link to previous specs when only a subset of the UI is updated.

5. **Provide Testable and Predictable Interactions:** Inconsistent or non-standard interaction patterns are difficult to implement accessibly and test with assistive tech.

Design Tip: Stick to established interaction models from your design system, and annotate any deviations with intended keyboard behavior, expected focus order, and visual feedback.

Developer



Persona: Devi

“I consult the accessibility specifications in a design to implement the end user experience free of accessibility defects.”

The developer role goes far beyond typing lines of code; They bridge between creative vision and real-world use, ensuring every user, regardless of ability, can enjoy a seamless experience. You are the architect of accessibility, translating the designer’s vision into a living, breathing application that welcomes everyone.

Design & Development Collaboration

Design and development collaboration begins with a shared language that includes component names, floorplan names, and external content annotations. They are the basic elements defining the design blueprint for consistency and clarity. These foundational elements aren’t mere labels; they’re the guiding framework that ensures every part of the app fits perfectly within the larger structure, bridging the creative ideas with technical implementation.

It’s also their task to deeply understand the shared accessibility language. The accessibility annotations serve as coordinates of the app’s architecture, pointing precisely to where every piece fits and should be configured, ensuring usable and inclusive flows, consistency and clarity across the entire digital landscape.

When designers specify components with exact names and tie them to floorplans—those reusable page structures, developers gain a powerful map. This map guides the implementation, allowing the developer to connect the dots between design intent and code reality without guesswork.

External content annotations point to third-party elements—such as embedded widgets, external libraries, or API-driven content—that fall outside your direct control but still affect the user experience. While the

developer may not manage how these elements are built, they're still responsible for understanding their behavior, ensuring they integrate smoothly, and flagging any accessibility risks they introduce.

Prepare the code for testing

But your work doesn't stop at translating designs into code. Preparing the app for testing is where your craftsmanship truly shines. By embedding semantic HTML, following ARIA standards, and ensuring proper landmarks and roles, you lay the groundwork for assistive technologies to interpret and interact with your app flawlessly. You're crafting invisible threads that empower screen readers, keyboard navigation, and other assistive tools to deliver a coherent story to every user.

Collaborating closely with designers to clarify annotations and accessibility Developer Notes means fewer surprises during testing and smoother certification processes.

Ultimately, your role is that of an accessibility steward, ensuring that the app's architecture is both robust and flexible—ready to welcome diverse users without compromise.

The foundation you help build today, with meticulous attention to design alignment and accessibility compliance, becomes the launching pad for inclusive digital experiences that don't just meet standards, but genuinely open doors. In this dance of design and development, your expertise turns vision into vibrant reality, one accessible component at a time.



General Development Tips

When you encounter accessibility annotations in a design:

Identify the Annotations

- Find the annotations in respective subchapters of this documentation

Understand the Use Cases

- Read all info there and in associated links
- Scrutinize documentation examples

Implement According to Best Practices

- Clarify missing edge cases in design.
- Follow the extra Developer Notes for developers in annotation subchapters
- Consult developer documentation for respective technology about implementation methods

Iterate on your Work

- Showcase proof of concept samples to designers
- Align and harmonize expectations
- Contact subject matter experts for open questions

Annotations

Foundation Annotations

The checklist helps designers remember key accessibility aspects that need to be addressed during the design phase. Foundation annotations help designers make thoughtful choices about components, page templates, layouts, flow, and navigation. They also clearly communicate these decisions to developers, ensuring the designed experience meets the needs of all users.

Foundation

- ☐ Component Name
- ☐ Floorplan Name
- ☐ Design Note

Component Name

Identify the connection between the design component and the control used for implementation aligned with developers to ensure an efficient development process.

Variants

Component Name



Figure 5: Informs the name of a component used on the layout

Component Name (Scope)



Figure 6: Informs the name of a component used on the layout with scope

Component Name (External)



Figure 7: Informs an external component or app is used in the layout

About Component Name

Components that have the same functionality within a set of web pages must be identified consistently.

WCAG 3.2.4 Consistent Identification requires that components with the same functionality are identified consistently across a product or set of pages. This means using the same visual symbols, labels, or roles for elements that perform the same action—like a “Search” icon or a “Submit” button. Consistency supports recognition, reduces cognitive load, and helps all users—especially those with cognitive or visual disabilities—navigate interfaces more easily and predictably. It’s a key principle in building intuitive, inclusive user experiences.

Naming components in the design as per shared component and control libraries is the first step to provide precision and remove doubts from design intention. Mapping design component and floorplans to their

technical UI library used for implementation is the best way to communicate the design construct.

Component Name and Component Name (Scope)

The Component Name annotation identifies the UI element in the design and map it to a corresponding component in the technical UI library used for implementation. It can be used scoped to frame the section where the component resides in the UI.

This ensures developers understand which component to reference, even if the visual design combines multiple UI patterns. This also prevents ambiguity during handoff and improves alignment between design intent and implementation.

- Note: The component name may differ from the internal name used in the design tool—it should reflect the name used in development or the design system.

Component Name (External)

Use the Component (External) variant when the design includes elements outside the immediate control of the product team. These may come from a third-party source, external design system, or API-based widget.

- When to use:
 - Embedded 3rd-party components (e.g., a chatbot, calendar picker, video player).
 - Shared components not owned by your team but consumed via the UI framework.

Accessibility Notes: **Any accessibility issues must be addressed at the source**, since your product relies on how the external content is structured and coded.

Table 4: Component annotation types

Annotation Type	Purpose	Accessibility Relevance
Component Name and Component Name (Scope)	Identify UI elements and their matching components in the implementation library	Helps developers apply correct semantics and interactive roles
Component Name (External)	Annotate 3rd-party or non-owned elements embedded in the design	Clarifies ownership; any accessibility fixes must be made at the source



Examples

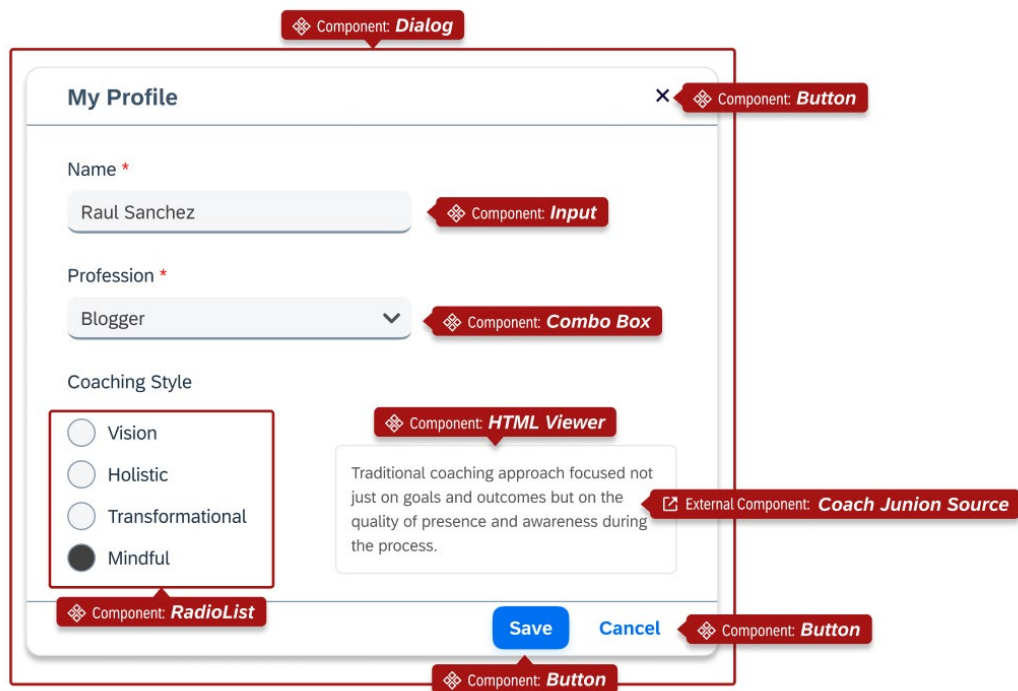


Figure 8: Component, Component (Source) and Component (External) annotations used in a dialog design

Developer Notes

Developers should check during the review process if the names used in Component Name reflect existing component names in the development framework inventories used for implementing the design.

They should also verify if the intended use cases map to the provided features of the respective framework components.

References

WCAG 2.2 Reference

[1.3.1 Info and Relationships](#) (A)

[3.2.4 Consistent Identification](#) (AA)

[5.4 Statement of Partial Conformance - Third Party Content](#)

SAP Product Standard Accessibility 4.3

[ACC 263 - Information, Structures and Relationships](#)

Floorplan Name

Inform the name of a page floorplan template

Variants

Floorplan Name

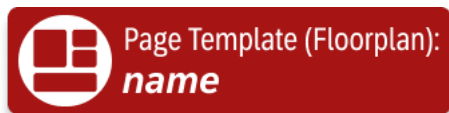


Figure 9: Informs the name of a page template used to create a new page layout

About Floorplan Name

The Floorplan Name annotation identifies a reusable page-level layout or structure, such as templates that include headers, footers, menus, breadcrumbs, or other navigational elements. It serves as a reference to standard page templates defined in the design system or framework and helps ensure consistency across screens.

Why It Matters

- **Design Consistency:** Using named floorplans encourages the reuse of familiar patterns, streamlining both design and development.
- **Clarity in Handoff:** Clear naming reduces ambiguity for developers and content creators, enabling accurate implementation of page structures.
- **Efficiency:** Annotating with Floorplan Names saves time by eliminating the need to redefine structure or behavior for every page.

Accessibility Relevance

Floorplan annotations are especially critical for accessibility. They help ensure:

- **Consistent Landmarks:** Reusable templates can define key landmark regions such as main, banner, navigation, and content info (footer), making navigation easier for screen reader users.
- **Structured Headings:** Templates often dictate where `<h1>` and other headings should appear, supporting a logical and predictable heading hierarchy.
- **Labeling of Repeating Elements:** Floorplans can include labels for repeating regions, improving clarity and orientation within complex pages.

Inclusive Design Benefits

Standardizing floorplan names across the design process helps ensure that accessibility best practices are not applied ad hoc. Without this structure:

- Heading levels may be inconsistent.
- Landmark roles could be missed or misused.
- Navigational regions may appear unpredictably across screens.

By naming and referencing floorplans clearly, teams reinforce semantic consistency and help assistive technologies interpret content more effectively creating a more inclusive user experience

Examples

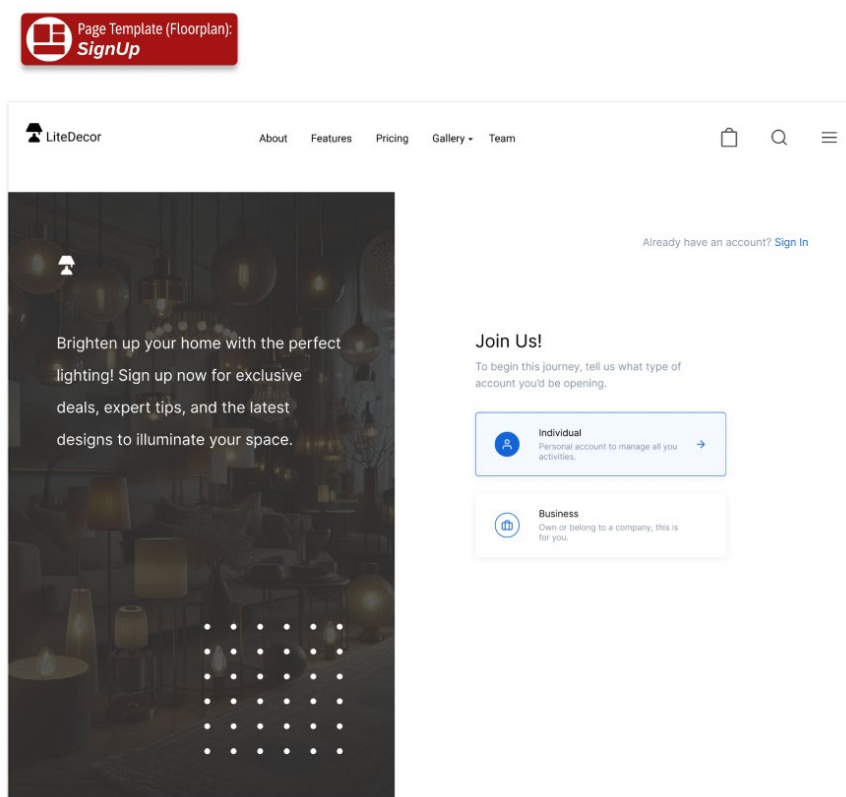


Figure 10: Floorplan name annotation used to identify a Sign-Up page floorplan from design and framework inventories

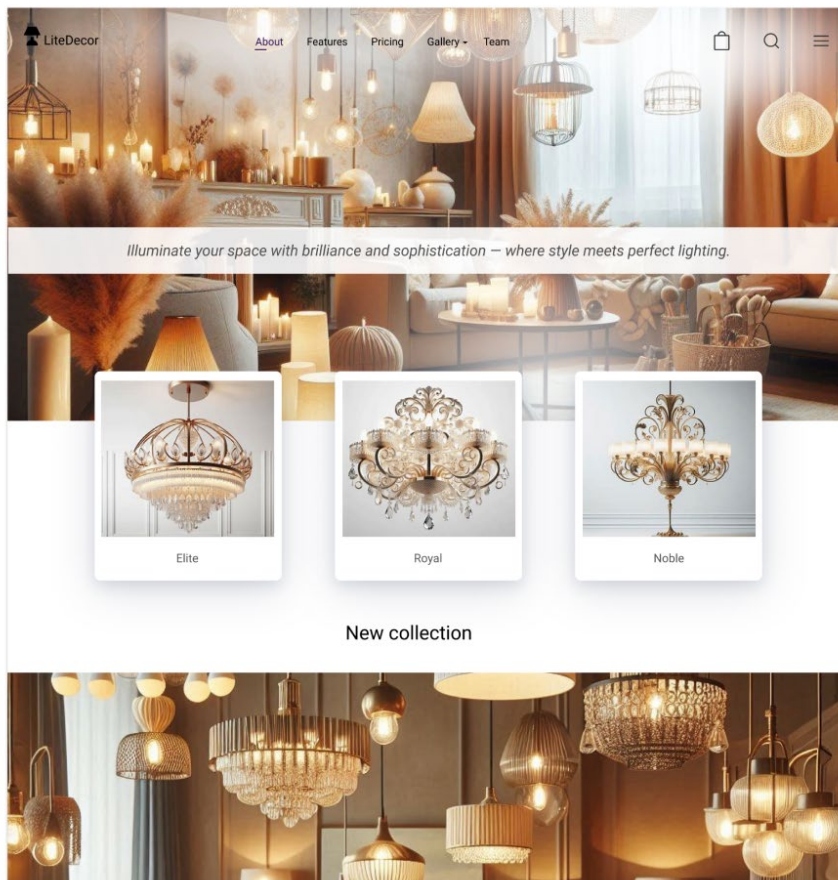


Figure 11: Floorplan name annotation used to identify a home page floorplan from design and framework inventories

Developer Notes

Developers should check during the review process if the names used in Floorplan Name reflect existing floorplan template names in the development framework inventories.

References

WCAG 2.2 Reference

[3.2.3 Consistent Navigation](#) (AA)

SAP Product Standard Accessibility 4.3

[ACC 267 - Consistent Navigation and Help](#)

Design Note

Add info in Developer Notes to complete your design

Variants

Design Note

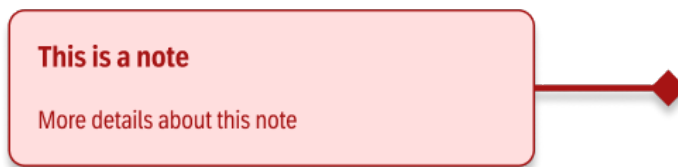


Figure 12: Alignments with other stakeholders may result on added details to the design specification. Use a Design Note to inform such details.

About Design Note

The set of annotations proposed in the document will help designers clarify how the accessible experience has been addressed during the design phase. But as presented in the introduction chapter, not all WCAG success criteria are covered.

There are also edge cases not addressed in this guide which may become critical to delivering a fully accessible experience. Sometimes these scenarios can be solved with user research; other times, it requires alignment with engineers to discuss technical challenges. The results of such strategies may require a more elaborated explanation of a design decision that affects accessibility.

This is where you may find the note asset helpful. We chose to make it pink to differentiate it from all other assets, as it could be used in any scenario: Foundation, Visual Design, Screen Reading, Keyboard Support, or Complementary annotation.

The pink note is intended to inform developers about something that should be considered during development that cannot be informed using existing annotation. Use the note when discussing design details during design reviews that are worth mentioning to developers. It is also helpful to address implementation scope and user research results to be addressed during development.

The pink annotation is also useful for supporting informal design reviews. It became helpful to summarize long comment threads using the Figma Comment functionality because engineers want to consume a document,

not comments. The note is used to address highlights of a discussion that happened in the comments.

Examples

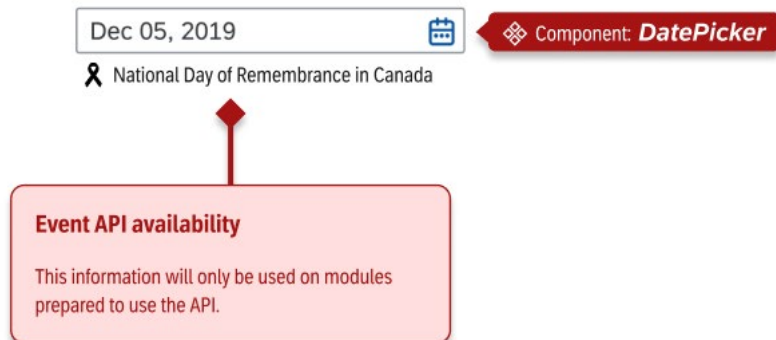


Figure 13: Pink note used to inform a dynamic information under an input field that will be handled by and API event.

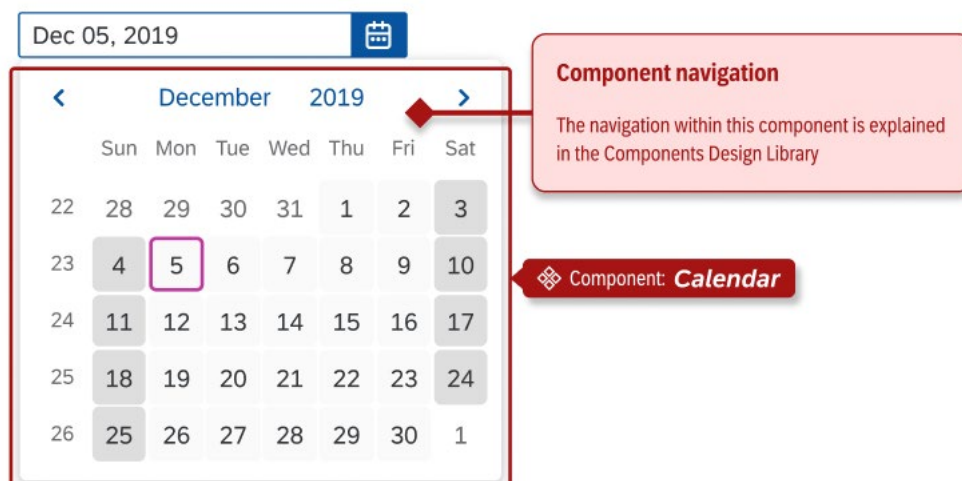


Figure 14: Pink note used to inform that component navigation and exploration is detailed in the library of components.

Visual Experience

Imagine you can see, but sometimes interpreting visual content, forms, or colors is difficult. Maybe the contrast is too low, the layout is confusing, or different colors look the same. What would help you quickly find what matters, understand what to do, and avoid mistakes—without needing perfect vision or relying on color alone?

This chapter helps designers to address accessibility for visual content.

The Inclusive Visual Experience

Meeting the needs of people with low vision, glare sensitivity and color blindness

Accessible Visual Designs are vital for users with visual impairments caused by eye diseases, but they also benefit people who are part of the neurodiverse group. This chapter recommends annotations to create a design specification that covers all accessibility standards for single and combined UI element to make them visibly clear to all users.

The design checklist proposed for this chapter helps designers create the Visual Experience inclusive to everyone. Some considerations for the native mobile experience are listed and explained.



Principles

Visual Impairment



Persona: Victoria

"I'm color blind, have myopia and I'm sensitive to light. I use a screen magnifier, and screen reading at times. It's a challenge to identify object contours, distinguish certain colors and interpret information effectively. Theming provides appropriate screen contrast."

Visual Impairment is any loss of sight other than total blindness is usually considered a visual impairment — not a disability. There are degrees of vision impairment that people may experience. This can range from total blindness through various levels of increased blindness to complete blindness or the inability to perceive light or movement.

These people represent individuals with color blindness, myopia, low vision, light sensitivity, and other condition that limit or difficult the visibility of digital products presented in digital screens (tunnel vision or peripheral field loss, central vision Loss, floaters and obstructions, double vision or diplopia, visual field distortions).

Globally, 2.2 billion people have a visual impairment. Therefore, visibility, a fundamental principle in design, must be addressed with care to enable this population to perceive information and conduct tasks.

Some statistics:

- About 2.2 billion people worldwide have some form of vision impairment or blindness (WHO).
- Approximately 217 million have moderate to severe vision impairment, meaning they can see but often need aids like magnifiers, enhanced contrast, or larger text.
- Around 300 million people worldwide experience some form of color blindness or color vision deficiency.



- Vision impairments increase with age; nearly one in three people over 65 have vision problems affecting daily tasks.

Situational and Temporary Disabilities

Permanent: Conditions such as color blindness, astigmatism, long-sightedness. Low vision can happen due to diabetes, retinopathy, macular degeneration, retinitis pigmentosa, glaucoma, and cataracts.

By integrating situational and temporary disabilities into this disability group, we help design teams address challenges that might not be permanent but still impact user experience under certain conditions.

- Glare sensitivity in bright light: This could make it hard for users to see screens or navigate in certain lighting conditions (e.g., outdoors in daylight, or in fluorescent lighting).
- Short-sightedness (not wearing corrective lenses): A person may struggle with vision if they forget or cannot access their glasses/contacts.
- Fatigue-induced vision challenges: Extended screen time or other conditions may worsen vision temporarily.



- General Design Tips

1. **Adjustable Text and Element Size:**

Fixed or small text and UI elements can be hard to see and interact with.

Design Tip: Support scalable text and UI components without breaking layouts. Ensure clear resizing without content overlap or truncation.

2. **High Contrast and Clear Visual Separation:**

Low contrast reduces the ability to distinguish interface elements and text.

Design Tip: Use high-contrast color schemes and ensure sufficient contrast ratios for text and key UI elements. Avoid relying on color alone to convey information.

3. **Simplified and Responsive Layouts:**

Challenge: Wide or cluttered layouts can overwhelm reduced visual fields or magnified views.

Design Tip: Design clean, well-structured layouts that adapt smoothly to smaller or magnified viewports, minimizing horizontal scrolling.

4. **Support for Zoom and Magnification:**

Users rely on zooming or magnification tools that may distort poorly designed interfaces.

Design Tip: Test designs at various zoom levels to ensure readability and functionality remain intact.

5. **Avoid Visual Noise and Distracting Effects:**

Flickering, flashing, or overly busy visuals strain users with low vision.

Design Tip: Limit animations and avoid flashing content. Use subtle transitions and maintain visual clarity.

6. **Sufficient Time to Read and Interact:**

Users with limited vision may need extra time to process content or complete tasks.

Design Tip: Avoid time-limited interactions without options to extend time or pause content updates.



Annotations

Visual Experience Annotations

The checklist helps designers remember key accessibility aspects that need to be addressed during the design phase. The visual experience scope helps designers check color contrast, ensure theming works, validate semantics, and address truncation or wrapping through simulations for reflow, responsiveness, text resizing, and text spacing.

Visual Experience

- ☐ Color Contrast
- ☐ Theme
- ☐ Responsiveness
- ☐ Text Resizing
- ☐ Text Spacing
- ☐ Tooltip



Color Contrast

Inform about color contrast values you have used in your design

Variants

Color contrast on Standard Theme

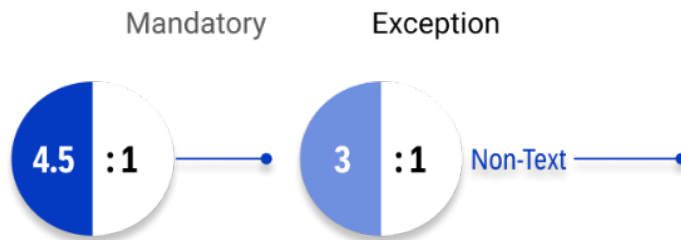


Figure 15: The color contrast annotation is used to inform that a chosen color meets minimum color contrast with its background.

Color contrast on High Contrast Theme

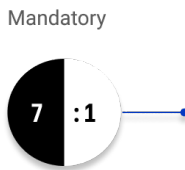


Figure 16: The color contrast annotation is used to inform that a chosen color meets minimum color contrast with its background.

About Color Contrast

Color contrast is the contrast ratio between two values: foreground and background color. It is mandatory for any UI element conveying relevant information to meet the minimum contrast ratio. It applies to text and graphical elements (controls and icons).

4.5:1 or higher is mandatory for:



Figure 17: Text and its states including placeholders

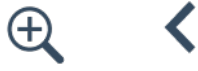


Figure 18: Icons



Figure 19: Images of text

Exempt:

☒ Checkbox label



Figure 20: Disabled controls



Figure 21: Pattern, Border & Decoration



Figure 22: Illustration, photograph & logo

High Contrast Theming aims to meet 7:1 color contrast by default and 4,5:1 for exceptions. It is used by users with low vision, who work in specific light conditions like bright environments, to compensate for moderate visual shortcomings (floaters, clouds, or beginning macular degeneration) and to work more efficiently and without exhaustion.

Use mandatory 4.5:1 (7:1 for high contrast) or higher whenever possible. If not possible, check if this is an exception or exemption. Use at least 3:1 for object borders of UI elements.

Examples

Font size and Icons in reference to Color Contrast

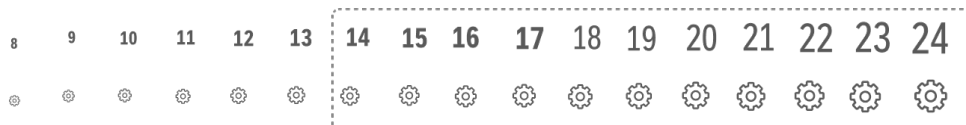


Figure 23: Large text definition starts at 14 pt bold text or regular 18 pt text and larger.

The same rule applies to icons. Aim for 4.5:1 on the default theme. The exception on the default theme is 3:1 is applicable for large text.



Figure 24: Meaningful non-text elements like an input field, require a contrast ratio of at least 3:1 against the background.

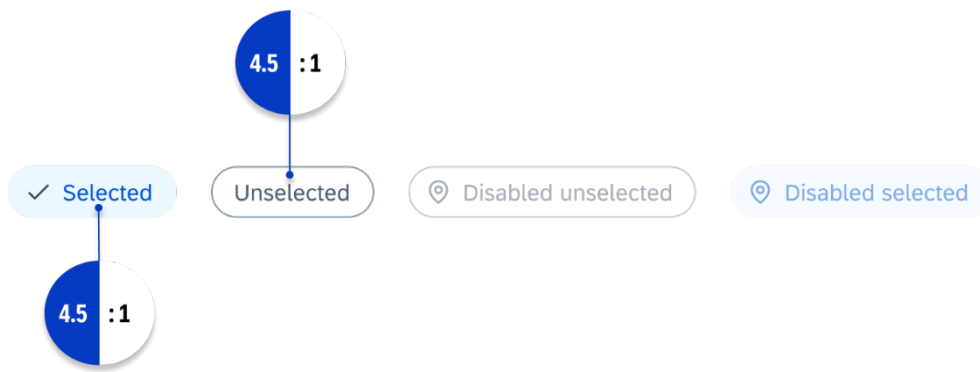


Figure 25: The visual presentation of text has a contrast ratio of at least 4.5:1. Text that is part of an inactive user interface has no contrast requirement.

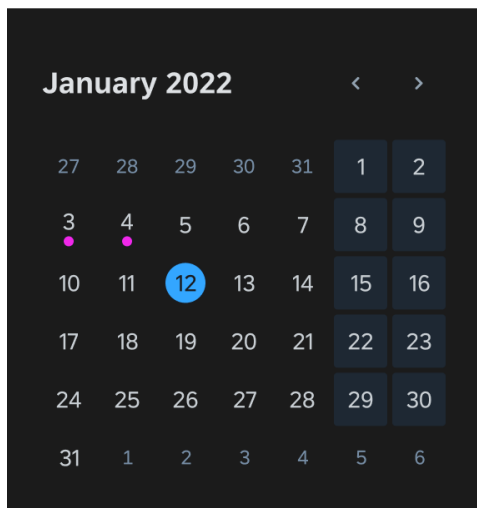


Figure 26: In high-contrast themes, text has a contrast ratio of at least 7:1.

Shared Benefits



Temporary Disabilities

Everyone Benefits

A sufficient color contrast is not only useful for users with low vision. Users have very individual color or contrast vision abilities. Environmental circumstances, such as intense sunlight shining on a display, can make it difficult to read content.



Limited Cognition

Cognitive

For some users with cognitive disabilities, it is crucial to have a sufficient color contrast available. Especially conditions that affect the ability to process information visually, benefit from an improved readability and clarity. Choosing colors that can be perceived easily can also minimize stress for users.

Developer Notes

Developers should check during the implementation if foreground and background color values for all user interface component parts in their frameworks meet the required contrast ratios for the current theme.

References

WCAG 2.2 Reference

[1.4.1 Use of Color](#) (A)

[1.4.3 Contrast \(Minimum\)](#) (AA)

[1.4.11 Non-text Contrast](#) (AA)

SAP Product Standard Accessibility 4.3

[ACC 261 - Minimum Contrast](#)

[ACC 262 - Color and Contrast Setting](#)

Theme

Address Light and Dark contrast themes and provide High Contrast themes in your designs.

Variants

Theme

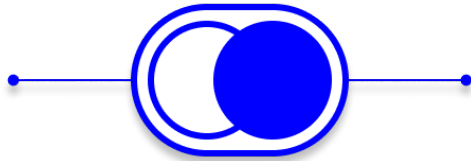


Figure 27: This annotation is used to make a parallel with a screen designed for the standard theme and its related screen designed for high contrast

About Theme

This annotation is used to make a parallel with a screen designed for the standard theme and its related screen designed for high contrast. A high contrast theme is associated with an optional setting.

Both Standard and High Contrast Themes must be addressed in the Design.

Reduce color saturation, avoiding or limiting pure white and black colors. Using pure black (#000) as a background for dark mode, you can dim the text and content slightly to reduce eye bleed due to cutting-edge contrast.

If possible, design light mode first and keep all visual elements and cues in place when creating the dark mode version. Don't ignore or remove colors because the dark mode can't handle the visual design.

There are some plugins to help adjust colors:

- [Appearance](#) (Figma plugin).
- [Dark mode magic](#) (Figma plugin).
- [Camilo](#) (Sketch plugin).

Please rely on the SAP Fiori Design Guidelines for color.

An application should have a color contrast setting control for users who wish to alternate themes. The setting offers the option to switch from a default light theming (4,5:1) to a High Contrast Theming, respecting the 7:1 contrast ratio.

Examples

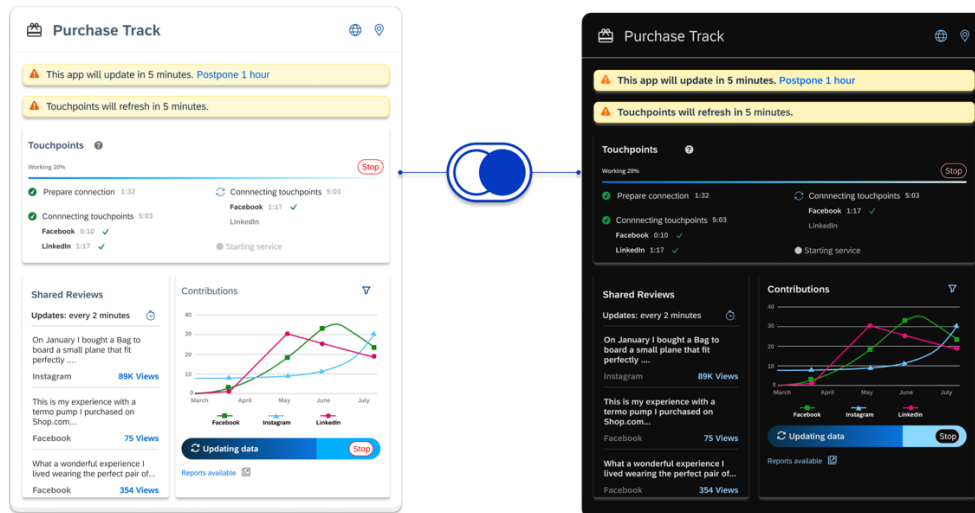
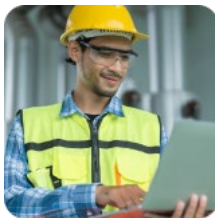


Figure 28: Theme annotation used to make a parallel with a screen designed for the standard theme and its related screen designed for high contrast

Shared Benefits



Temporary Disabilities

Everyone Benefits

The use of High Contrast themes is not limited to specific user groups. People without disabilities might also prefer more visually distinct interfaces and find them easier to read or navigate.



Limited Cognition

Cognitive

Themes like High Contrast do primarily support users with visual deficiencies but can also be beneficial for users with cognitive disabilities. This affects for example people who struggle with focusing on tasks due to sensory sensitivities. However, it is crucial to understand that preferences will differ and some users with cognitive disabilities might prefer a lighter color scheme over high contrast.

Developer Notes

Developers should check during the implementation if they

- provide for end users multiple themes as part of the UI framework, one of them being a theme with 4.5:1 support and another with 7:1 high-contrast support.
- provide an option for end users to switch between these themes

If no own themes are provided as part of the UI framework, a fallback must exist for users to use different OS themes on the platform instead who comply to the given requirements.

References

WCAG 2.2 Reference

[1.4.1 Use of Color](#) (A)

[1.4.11 Non-text Contrast](#) (AA)

SAP Product Standard Accessibility 4.3

[ACC 261 - Minimum Contrast](#)

[ACC 262 - Color and Contrast Setting](#)



Responsiveness

Design the content choreography where UI elements are reorganized in a layout that adjusts and responds to different orientations and screen sizes.

Variants

Screen Resize

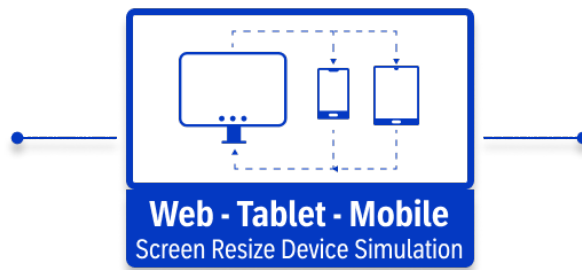


Figure 29: Simulate a WEB screen seen from a mobile device or a tablet to ensure content visibility is preserved.

Orientation



Figure 30: Simulate the mobile layout orientation to explore how content adjusts to Portrait and Landscape.

Content Visibility

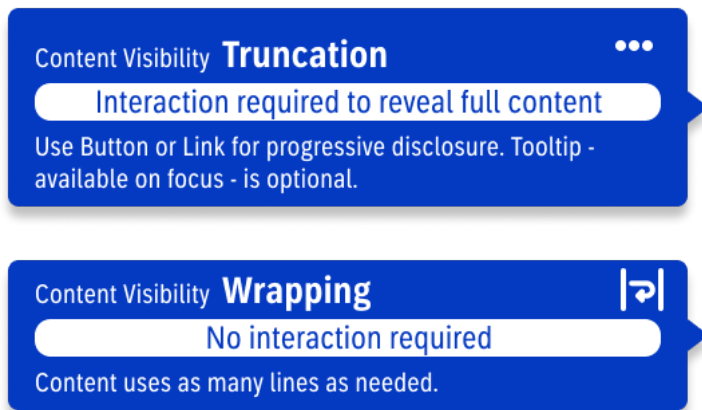


Figure 31: Provide instruction on how to address content visibility issues by proposing truncation or wrapping.

About Responsiveness

Prepare your designs to simulate how layouts and content behave across different screen sizes and orientations, including web, tablet, and mobile. The Screen Size Reflow and Orientation annotations help designers ensure the user interface adapts correctly and remains usable across devices.

This design considerations is supported by two WCAG requirements. The 1.3.4 Orientation (AA) requires that content remains fully operable and understandable in both portrait and landscape modes (or any orientation), unless a specific orientation is essential for the functionality. This ensures that users can maintain consistent spatial cues and mental maps as they switch between device orientations, preventing disorientation and supporting a stable navigation experience. The 1.4.10 Reflow (AA) requires that content can be presented without loss of information or functionality, and without the need for two-dimensional scrolling (both horizontal and vertical), at a width equivalent to 320 CSS pixels. This ensures that users, especially those who zoom in or use small screens, can still follow the layout and understand the page structure without excessive scrolling. This supports continuity, helps users keep their place, and reduces cognitive load during navigation.

Screen Resize

Use the Screen Size Reflow annotation to describe how the layout and content respond when the viewport changes—such as resizing a web browser or adapting a design to tablet and mobile screen sizes.

Purpose: This annotation ensures that the UI is responsive and that it maintains usability and content clarity across a wide range of screen sizes.

It focuses on layout reflow, element wrapping, and content scaling during viewport resizing.

What to Specify:

- **Reflow Behavior:** Describe how content and layout reorganize when transitioning from large to smaller screens (e.g., multi-column to single column).
- **Element Wrapping or Truncation:** Indicate which elements are allowed to wrap and which should truncate. Follow design system guidance.
Minimum Supported Size: Ensure the design works at 320 CSS pixels height × 256 CSS pixels width (portrait mode minimum), with corresponding support for 200% zoom on larger screens.
No 2D Scrolling: Confirm that content doesn't require horizontal scrolling in addition to vertical scrolling, unless essential (e.g., data tables, maps).
- **Zoom and Resize Compatibility:** Ensure designs remain operable at 200% zoom, especially on desktop browsers.

Why It Matters:

Supports accessibility and flexible usage across devices.

Improves design handoff clarity for developers implementing responsive behavior.

Helps users who rely on zooming, screen magnifiers, or custom device settings.

Orientation (Mobile/Tablet)

Use the Orientation annotation to define how the design behaves in portrait vs. landscape orientations, particularly on mobile and tablet devices.

Purpose: This annotation ensures that your design is not locked into a single orientation unless justified, and that it behaves reliably across both modes.

What to Specify:

- **Support for Both Orientations:** Indicate whether the interface adapts properly in portrait and landscape modes without loss of content or functionality.
- **Layout Adjustments:** Note any key layout shifts (e.g., navigation bar repositioning or text scaling) that occur between orientations.
Essential Orientation: If one orientation is required for a functional



reason, clearly document it as a design note (e.g., landscape-only video editing tool).

- **Minimum Size Support:** As with reflow, verify that orientation changes do not drop below accessibility thresholds—320×256 CSS pixels, depending on mode.
- **No Content Lockout:** Ensure users aren't blocked from accessing key functions based on how they hold the device.

Why It Matters:

- Orientation flexibility supports users with different device preferences or physical needs.
- It prevents critical functionality or content from being inaccessible in a non-preferred mode.
- It aligns with WCAG requirements and inclusive design standards, especially for mobile-first experiences.

The takeaway of this chapter is: Prepare your screen to allow the content and layout adjustments to smaller screens. This will forcefully require layout adjustment, content reorganization, and wrapping or truncation observations.

Table 5: Comparison of Responsiveness annotation variants

Annotations	Focus	Consideration
Screen Size Reflow	From large to small screens	Reflow logic, wrapping/truncation, zoom, 2D scroll prevention, min size support
Orientation	Portrait vs. Landscape modes	Mode adaptability, layout shifts, restrictions, functional requirements



Content visibility

Loss of visibility is a violation of WCAG 1.4.4. Text Resize. When content needs to adjust to the screen the design should plan for potential content visibility issues. These can be addressed with truncation or wrapping.

Wrapping

This is the simplest solution to guarantee full content remains visible when layout adjusts to text resize, but also text spacing and responsiveness. This solution is ideal for content that is not related to an interactive UI element, like page headers.

Truncation

Avoid truncation if possible. But if line wrapping is not possible, truncation is acceptable as long as the full content is available on focus or drill in (after user activation of trigger where truncated content is presented). An indication that this information can be accessed, should be provided to the user in some way, like a button or a link. This trigger will enable user to trigger a flow to visualize the full content. A button can open a dialog, popup, expand text or expand a section. A link will can transport the user to a new page where the full content can be retrieved.

Examples



Figure 32: Simulation of a mobile layout orientation to explore how content adjusts to Portrait and Landscape

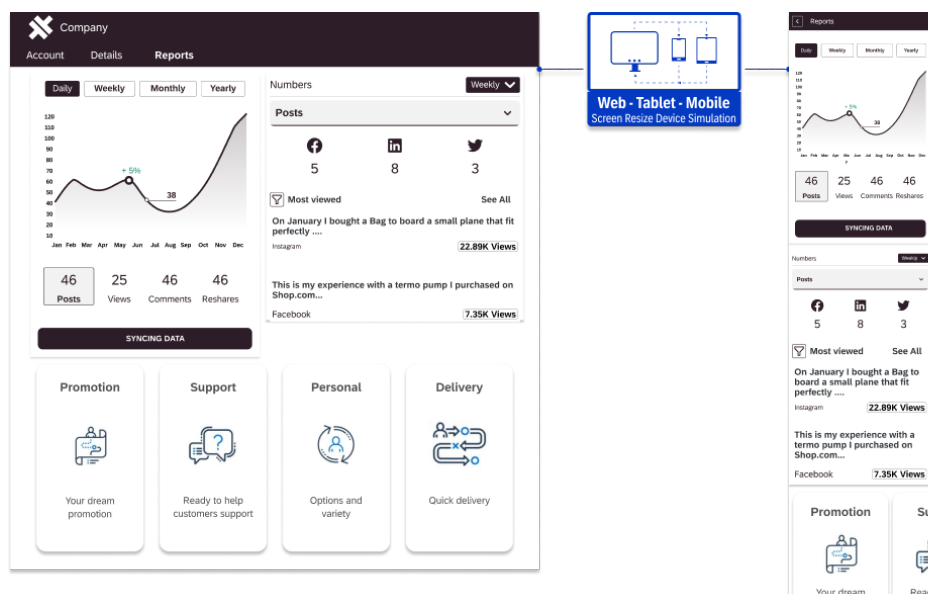


Figure 33: Simulation of a web/tablet/mobile layout orientation to explore how content adjusts to Portrait and Landscape

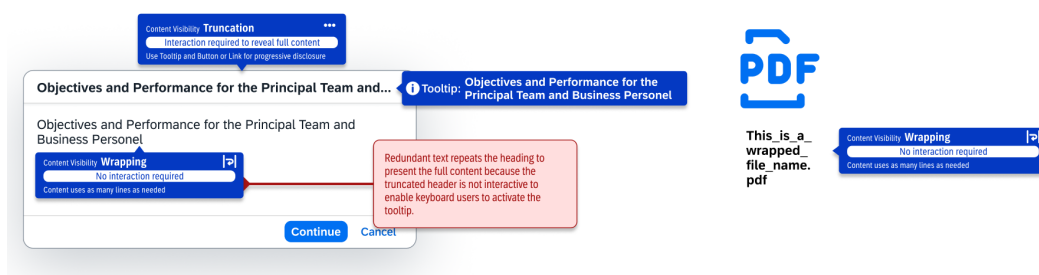


Figure 34: Indication of different truncation and wrapping cases using the Content Visibility annotation variant

Shared Benefits



Temporary Disabilities

Everyone Benefits

Responsive design and adaptable orientation make interfaces more usable for all users—on any device, in any context. Whether resizing a browser window while multitasking or rotating a phone, users benefit from layouts that adjust smoothly without losing content or function.



Limited Cognition

Cognitive

Clear, consistent layout across screen sizes and orientations reduces mental effort, supports predictable navigation, and prevents confusion caused by shifting or hidden content—making the experience more intuitive and less overwhelming.



Limited Mobility

Interactions

For some people with motor control difficulties, scrolling horizontally or vertically, or changing the display orientation can be difficult.

Developer Notes

Developers should check during the implementation if they

- use relative units
- use media queries
- use responsive layout grids
- don't suppress intentionally
responsivity (by setting special flags)
- don't limit device orientation capabilities

References

WCAG 2.2 Reference

[1.3.4 Orientation](#) (AA)

[1.4.10 Reflow](#) (AA)

SAP Product Standard Accessibility 4.3

[ACC 282 - Responsive Design](#)

Text Resize

Simulate text spacing to verify how the text will remain visible on the screen and recommend strategies to wrap or extend elements when necessary.

Variants

Text Resize Annotation



Figure 35: Simulate the web screen layout shifts by a 200% font size increase, ensuring content visibility is maintained.

Text Resize Annotation with Recipe for Success

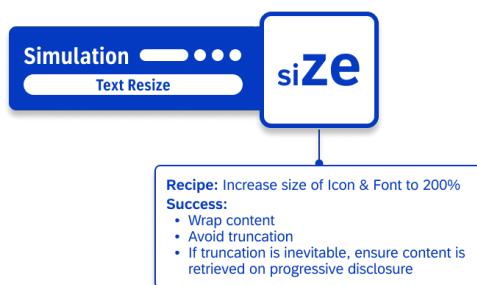


Figure 36: Address content visibility indicating where wrapping occurs and where truncation is acceptable.

About Text Resize

Accommodate the text content reorganizing the layout within the same screen size for when the user magnifies text up to 200%. This accessibility support benefits low-vision users who don't need to use magnification tools but rather built-in capability; it does not rely on assistive technologies.

Prepare your screen to allow the text to resize for all UI elements. This will forcefully require layout adjustment and reorganization.

Users will magnify the text up to 200% using a mouse and keyboard to enlarge the text and content. CTRL key + Scroll wheel up / down enables content to resize on Chrome, Firefox and IE. There is also the possibility of a browser configuration to control the resize.

The screen should provide visual feedback that results in the layout scaling while preserving the usability of all functions on the page, including the text alternatives for non-text content when magnified similarly. Avoid horizontal scrollbars. There will be some issues during content rearrangement. Here are some ways to solve them:

- Give a variable amount of space (that adapts to the text length)
- Wrap the text
- Add a data tip with the full text that becomes active when the text is truncated
- Display the full text on focus (overlap with nearby text is fine here)
- Make the text rollable on focus

Content visibility

Loss of visibility is a violation of WCAG 1.4.4. Text Resize. When content needs to adjust to the screen the design should plan for potential content visibility issues. These can be addressed with truncation or wrapping.

Wrapping

This is the simplest solution to guarantee full content remains visible when layout adjusts to text resize, but also text spacing and responsiveness. This solution is ideal for content that is not related to an interactive UI element, like page headers.

Truncation

Avoid truncation if possible. But if line wrapping is not possible, truncation is acceptable if the full content is available on focus or drill in (after user activation of trigger where truncated content is presented). An indication that this information can be accessed, should be provided to the user in some way, like a button or a link. This trigger will enable user to trigger a flow to visualize the full content. A button can open a dialog, popup, expand text or expand a section. A link will transport the user to a new page where the full content can be retrieved.



Examples



Figure 37: Example of a component showing how content adjusts to sizing inside a component.

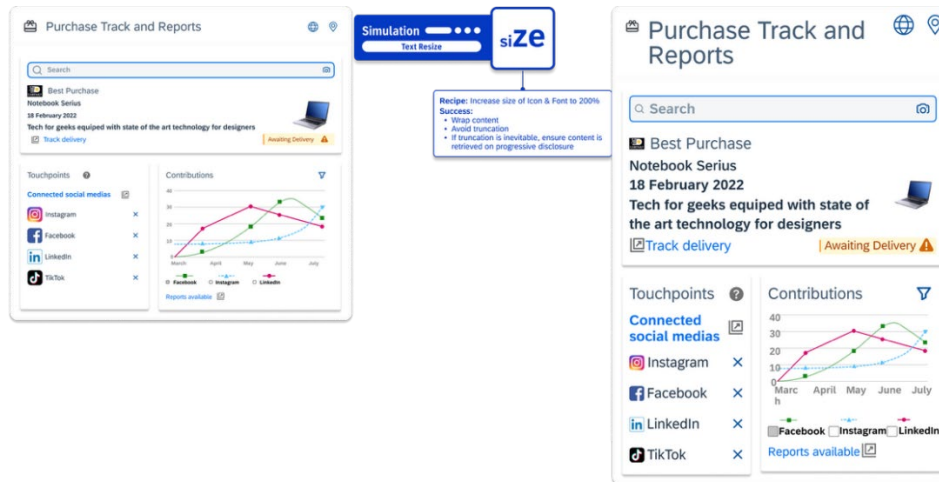


Figure 38: Example of a screen showing how the layout is affected when the content is resized.

Shared Benefits



Temporary Disabilities

Everyone Benefits

Adjusting the text size can not only be a desire of users with visual impairments. Enlarged text can also simply be a user preference or more comfortable for reading. External conditions can also lead users to want to adjust the text size. Therefore, it should be ensured that all content is still usable even at 200% font size increase.



Limited Cognition

Cognitive

An increased text size can potentially help people with cognitive issues to easier process text.

Developer Notes

Web Development: Developers should check during the implementation that they

- use a responsive layout approach (see last chapter)
- Set the preferred browser font size
- Decide on a minimum font size
- Set font-size: 100% on the html tag
- Use em/rem units for font sizes
- Watch it in action (check truncations, wrapping etc.)

Don't suppress any zooming functionality, i.e. NEVER do this:

```
<meta name="viewport" content="width=device-width,  
initial-scale=1, maximum-scale=1, user-scalable=no
```

In native mobile development, they should check if all app text and object borders adapt to font resizing settings of the platform without truncations and wrapping.

References

WCAG 2.2 Reference

[1.4.4 Resize text](#) (AA)

[1.4.8 Visual Presentation](#) (AAA)

SAP Product Standard Accessibility 4.3

[ACC 260 - Text Resizing up to 200 percent](#)

Text Spacing

Simulate how the layout will look like after text resize affects the organization of UI elements in the screen.

Variants

Text Spacing Annotation



Figure 39: Simulate the web screen layout shifts by text spacing, ensuring content visibility is maintained.

Text Spacing Annotation with Recipe for Success

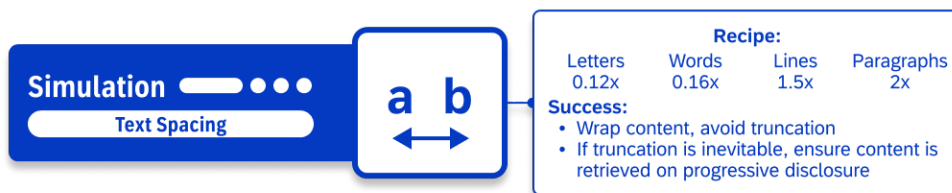


Figure 40: Simulate the web screen layout shifts by text spacing, ensuring content visibility is maintained including recipe for success

About Text Spacing

Many people with cognitive disabilities have trouble tracking lines of text when a block of text is single-spaced. These users resolve this problem by overriding the text spacing resulting in more space used in the layout to present the content and an improved reading experience.

It is essential to observe that when text space increases, the area size should require more space to present the information, preferably without truncation. All information should remain visible.

A designer can simulate the extra space taken in the layout to ensure that the content is still readable and operable when text spacing is activated. As a result UI elements will be enlarged to fit the content or wrap the content to the next line. Here are the rules applied to text spacing:

- Line height (line spacing) to at least 1.5 times the font size
- Spacing paragraphs to at least 2 times the font size

- Letter spacing (tracking) to at least 0.12 times the font size
- Word spacing to at least 0.16 times the font size.

These rules apply to small or large pieces of text, including labels, read-only fields, header and sub header, small strings of text, and text used on tables.

Avoid truncation (...) if possible. Explore wrapping the text and adding an extra line, such as on a table or cards. Content cut-off with no access to the visible information demonstrates poor design. Truncation requires either an extra click to read the whole content or a tooltip.

Examples

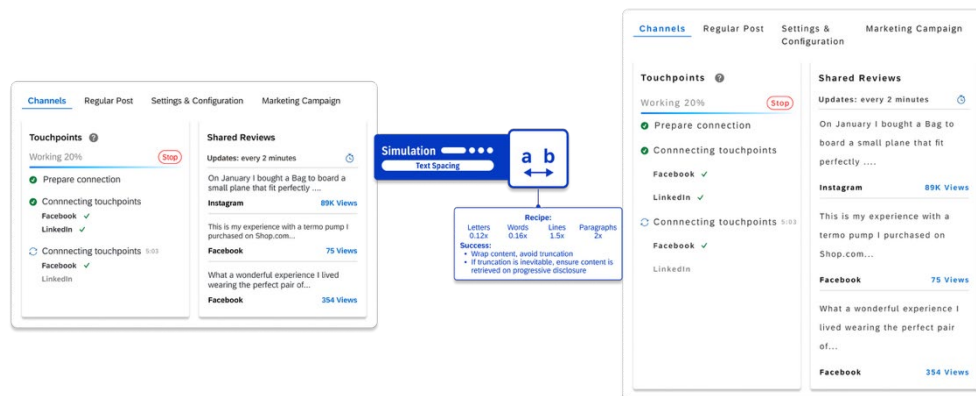


Figure 41: Example for simulating web screen layout shifts by text spacing, ensuring content visibility is maintained including recipe for success

Shared Benefits

All users will benefit when applying this annotation.

Developer Notes

Web Development: Developers should check during the implementation that they

- use a responsive layout approach (see last chapter)
- Set the preferred browser font size
- Decide on a minimum font size
- Set font-size: 100% on the html tag
- Use em/rem units for font sizes
- Additionally, you should offer an option of overriding default text spacing that defines respective CSS like so:

```
* { line-height: 1.5em !important;
    letter-spacing: 0.12em !important;
    word-spacing: 0.16em !important; }
```

```
p { margin-bottom: 2em !important; }
```

This can be set i.e. in application settings.

In native mobile development, they should check if all app text and object borders adapt to extra text spacing settings of the OS without truncations and wrapping (alternatively: check possibility for selecting a font with larger spacing which will have the same effect).

References

WCAG 2.2 Reference

[1.4.12 Text Spacing](#) (AA)

SAP Product Standard Accessibility 4.3

[ACC 283 - Text Spacing](#)



Tooltip

Use tooltips for icon buttons, maps, data points in graphs.

Variants

Tooltip Annotation



Figure 42: Tooltip reminder pointing to the element that will trigger tooltip display on hover.

About Tooltip

A tooltip is a visual label replacement that should also be exposed to screen readers as additional descriptive info for an element. Tooltips are helpful on icon buttons in addition to hidden labels. They are brief informative messages triggered on mouse hover or on focus on UI elements like icons, links, graphics, charts, maps, etc. They are helpful for people with vision - some blind users tend to disable the tooltip narration on assistive tools. Some users may require screen magnifiers to read the content.

Because tooltips appear and disappear with keyboard focus or pointer hover, design the interaction so that users can perceive the additional content and dismiss it without disrupting their experience. Inform developers that the tooltip should be:

- Dismissible by pressing Escape,
- Persistent providing enough time for users to perceive and read the tooltip, and
- Be positioned to ensure it does not block other pertinent information.

Usually not associated with a visual design, tooltips are rendered by the system and are generally available to sighted users because it appears on mouse hover. Tooltips are dismissed when users move the mouse out of the triggered element. Screen reader users can switch off the announcement of tooltips, so we recommend against relying solely on tooltips to communicate important information to the user. You can read about this topic in the Screen Reader chapter under

Tooltips.

Recommended Usages

Because tooltips alone can be missed, they cannot be used as vital information to complete a task. Use a tooltip for secondary or tertiary information. Do not repeat obvious interactions like repeating a visible label of a button. Some use cases:

- Guide to onboard new users considering a flow of max 3 or 4 tooltips.
- Featured adoption to teach users to recognize new capabilities.
- Taxonomy to explain terminologies and remind users about rarely used features.

Tooltip can be used to inform available shortcuts like hotkeys in native tooltip on the screen. But be careful to make the tooltip available only on interactive UI element - if they are placed on a non-interactive UI element it will prevent users with limited mobility (Keyboard users) from accessing the information. It's Principles that if tooltip is equal to invisible label, the screen reader will announce only once, minimizing the verbose for blind users. Informing the same information as an invisible is important when screen reader users disable the tooltip in their assistive tool.

Users can trigger a tooltip using gestures, keyboard focus, and short presses on mobile devices. They can be used on mobile applications and typically requires a "long press." But there is no common standard across industries. Tooltips can be used to present invisible labels, descriptions, messages, spell out abbreviated text, and display.

Beyond Tooltip

The requirement 1.4.13 Content on Hover or Focus covers any content that appears when a user hovers with a mouse or focuses with a keyboard—such as tooltips, popovers, help bubbles, or custom dropdowns. It ensures that this content is usable for everyone by requiring that it can be dismissed without moving the pointer, stays visible while being hovered over, and doesn't disappear unexpectedly. This helps users with limited mobility, low vision, or those using keyboard navigation to fully access and interact with additional content.

Tooltips and popovers are useful to prevent users to move away from their tasks and consult documentation - in this case consider using an overlay popup or popover or show them in context menus.

The Tooltip annotation was moved to the "Visual Experience" category because it enhances interface clarity for sighted users by displaying additional information without cluttering the main layout. It serves as a label alternative required for sighted people and is not used by blind people who tend to disable tooltip on screen readers.



Examples



Figure 43: Example Tooltip for an Edit-Button

Keep in mind for Content for Sighted vs Blind users

Do not confuse content to support screen reader users and content to support sighted people.

Tooltips are used as a secondary information for sighted users who can establish a context by visual proximity with other elements. Tooltip only builds confidence about the action that will be triggered. The content that appears on hover (or keyboard focus) can also be repeated by the screen reader if the user has not turned off tooltips on the settings. But research has shown that users who did not turn them off, will skip the tooltip text if they perceive the repetition.

Besides tooltips, invisible labels should also be provided to support blind users.

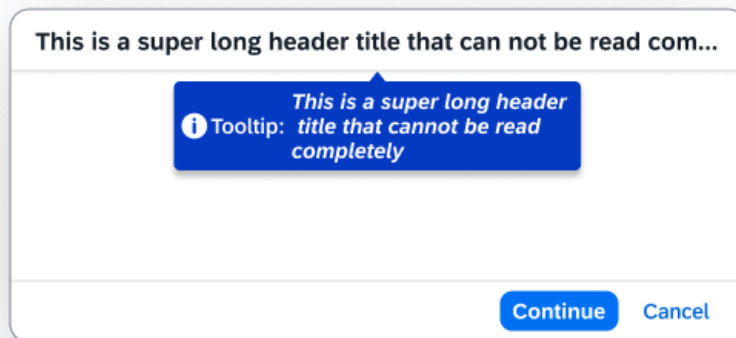


Figure 44: It is okay to add a tooltip on truncated text, but only if text wrapping is explicitly not possible!

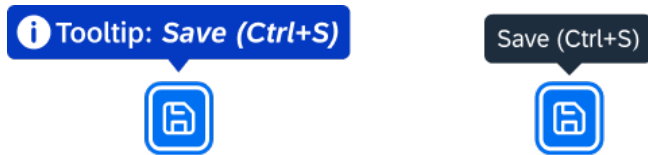


Figure 45: Annotation in a design specification with the proposed content and the final rendering of the tooltip.

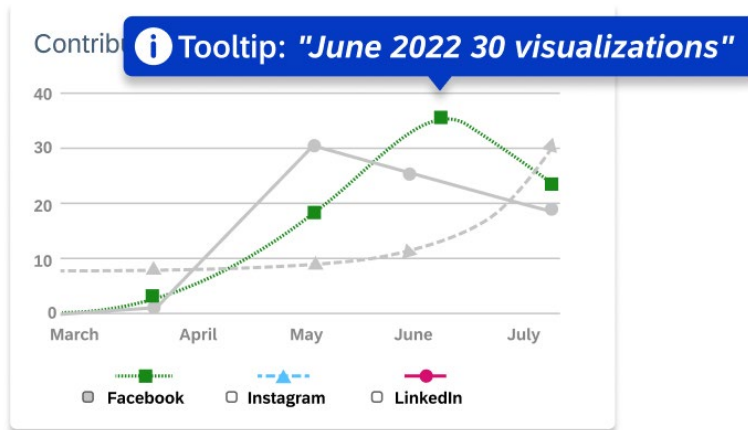


Figure 46: Design specification with proposed content to implement a tooltip as a data tip

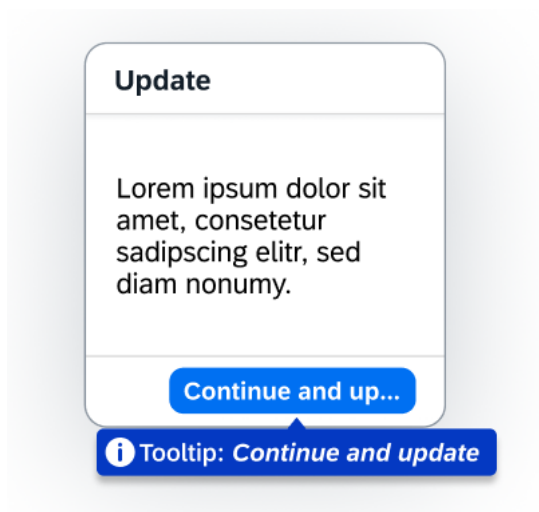


Figure 47: Tooltips are also helpful to show the full label of buttons for screen magnifier users

Shared Benefits

All users will benefit.

Developer Notes

Web Development: Developers should check during the implementation that they either use the native `title` attribute of the respective HTML element or (better) develop their own tooltip widget following best practices, such as [Tooltip Pattern | APG | WAI | W3C](#).

For native mobile development, use the native tooltip support in the SDKs of the given platforms:

iOS: [toolTip | Apple Developer Documentation](#)

Android: [Tooltip | Jetpack Compose | Android Developers](#)

References

WCAG 2.2 Reference

[1.1.1 Non-Text Content](#) (A)

[1.4.5 Images of Text](#) (AA)

[1.4.13 Content on Hover or Focus](#) (AA)

SAP Product Standard Accessibility 4.3

[ACC 255 - Text Alternative for Non-Text Content](#)



IA ЯfflФæ' mōzz'ЯБfQЭQфô BBz3и' Эffli' yōЯ
ôōЯfi fiūit mūi i' ηηōЯfflūā' ūi fflūiô ôi ЯōLj' fiô
fi ЯōLj'

Adelina Todorova – UX Designer

Interaction Experience

Imagine you can only use a keyboard or need to tap with a single finger. How many steps or key presses are too many to complete a task? Is it clear where your focus is on the screen, and can you easily reach what you need without getting stuck or lost?

This chapter helps designers to address interactive patterns.

The Inclusive Interactive Experience

Meeting the needs of people with limited mobility

Prepare your application and UI components to allow any user to interact and navigate with a keyboard as an alternative to the mouse.

Keyboard support benefits users with motor disabilities as some people have tremors that limit their ability to control a mouse. It also benefits expert users for efficient navigation and interactions and fast data input. Blind users benefit from switching between Screen Reading and Keyboard navigation strategies.

The most common navigation strategies are Focus Order (TAB) to explore interactive elements on a screen (links, buttons, form fields) and Skipping Groups (F6)* to skip entire groups on the screen.

Navigation is used on screens and within components. Initial focus placement marks the beginning of a navigation journey.

When a control is focused, the user can manipulate it using keys like Enter or Space. Repeating patterns of interaction and navigation simplify memory load and ensure consistency across the application.

* SAP Proprietary shortcut



Principles

Limited Mobility



Persona: Monique

"I have motor limitation. I'm agile. I work quickly in repetitive tasks, but with age I am experiencing some hand tremor and arthritis which impacts my work when using the mouse. My preferred interactions are keyboard & touch."

Limited mobility often stems from motor impairments such as paralysis, muscle weakness, or conditions like cerebral palsy. These impairments can affect a person's ability to use traditional input devices like a mouse, keyboard or touch.

This user group represents individuals with age-related hand tremors who prefer keyboard and touch interactions over mouse usage for more efficient and accurate task completion.

Types of Motor Impairment:

- Cerebral Palsy, Quadriplegia, Spinal cord injury
- Cerebral palsy, Muscular dystrophy, Parkinson's disease
- Multiple sclerosis, Spina bifida, Lou Gehrig's disease
- Arthritis, Essential tremor
- Lost or damaged limb

Some statistics:

- Mobility limitations are impairments that affect between 1/3 and 1/2 of adults age 65 or older.
- 40% of people with a motor impairment have difficulty using their hands.

Situational & Temporary Disabilities



Permanent: Complete or near-total loss of vision.

By integrating situational and temporary disabilities into this disability group, we help design teams address challenges that might not be permanent but still impact user experience under certain conditions.

Situational and temporary

- Sweating hands (as mentioned earlier): Temporary difficulty with touchscreens or physical devices due to sweating.
- Temporary injuries (e.g., sprained wrist): Limited range of motion in the hands or arms, making it hard to use touch devices or keyboards.
- Carrying objects: When holding heavy objects or bags, a person may find it hard to interact with devices (e.g., texting one-handed while holding groceries).



General Design Tips

1. **Keyboard over mouse navigation:**

Shaky hand movements and limited range of motion make mouse use unreliable, especially when targeting small buttons or moving quickly across the screen, leading to strain and frequent errors.

Design tip: Ensure all functionality is accessible via keyboard (Tab to move, Enter/Space to interact). Maintain a logical tab order and avoid focus traps.

2. **Large and easy-to-target controls:**

Small clickable areas are hard to activate accurately with limited dexterity, often leading to missed or accidental clicks, especially when using touch or a shaky pointer.

Design tip: Design buttons and interactive elements with a minimum touch target of 44×44 px. Increase spacing between adjacent clickable items to reduce accidental activation.

3. **Simple interactions over complex gestures or shortcuts:**

Multi-step gestures (such as pinch-to-zoom, swipe, or long-press) and complex keyboard shortcuts (like Ctrl + Alt + Shift) can be physically challenging or impossible for some users to perform.

Design tip: Simplify touch interactions by offering accessible button alternatives. Use single-key commands or customizable shortcuts where possible.

4. **Interfaces that reduce fatigue:**

Poorly designed flows with long navigation paths or excessive interactions increase physical effort and cause fatigue, especially during extended or repetitive use.

Design tip: Prioritize task efficiency by minimizing steps. Provide shortcuts and customizable UI settings (e.g., skip links, collapsible sections) to reduce effort and preserve user stamina.

5. **Visible focus cues for orientation:**

Lack of visible focus indicators makes it difficult for keyboard users to track their position, increasing the risk of activating the wrong element or losing their place in the interface.

Design tip: Always show a clear, high contrast focus outline for all interactive elements. Ensure the focus order follows a logical, predictable pattern.



6. Customizable input and interaction settings:

Standard input speeds (like key repeat rate or pointer speed) may not suit all users, and lack of control over scaling or input behavior can increase effort and reduce usability.

Design tip: Provide adjustable settings such as pointer speed, key repeat rates, and scaling. Allow users to personalize interaction to match their physical abilities.

There are other factors that impact users with physical restrictions, such as fixed screen orientations forcing uncomfortable positions for those with restricted neck or limb movement, limited input controls including the lack of integrated voice support, and poorly designed flows with long navigation paths or excessive interactions increase physical effort and fatigue, especially during extended or repetitive use.

To address these issues, designs should support flexible screen orientations - Responsiveness supporting portrait and landscape, ensure orientation, compatibility with speech or voice recognition and assistive technologies, and prioritize task efficiency by minimizing steps and offering shortcuts or customizable UI features that reduce effort.



Annotations

Interaction Annotations

The checklist helps designers to remember the important aspects of accessibility that should be addressed in the design phase.

Interaction Experience

- ☐ Input Mechanism
- ☐ Focus Order
- ☐ Initial Focus Position
- ☐ Focus Restore
- ☐ Skipping Group
- ☐ Shortcut
- ☐ Trigger
- ☐ Motion Alternative
- ☐ Minimum Target Size
- ☐ Journey



Input Mechanism

Understand how users trigger actions and explore pages with a mouse, the keyboard, and touch devices. Rely on established patterns to define keyboard interaction and navigation within components.

Variants

Input Mechanisms - Touch Actions

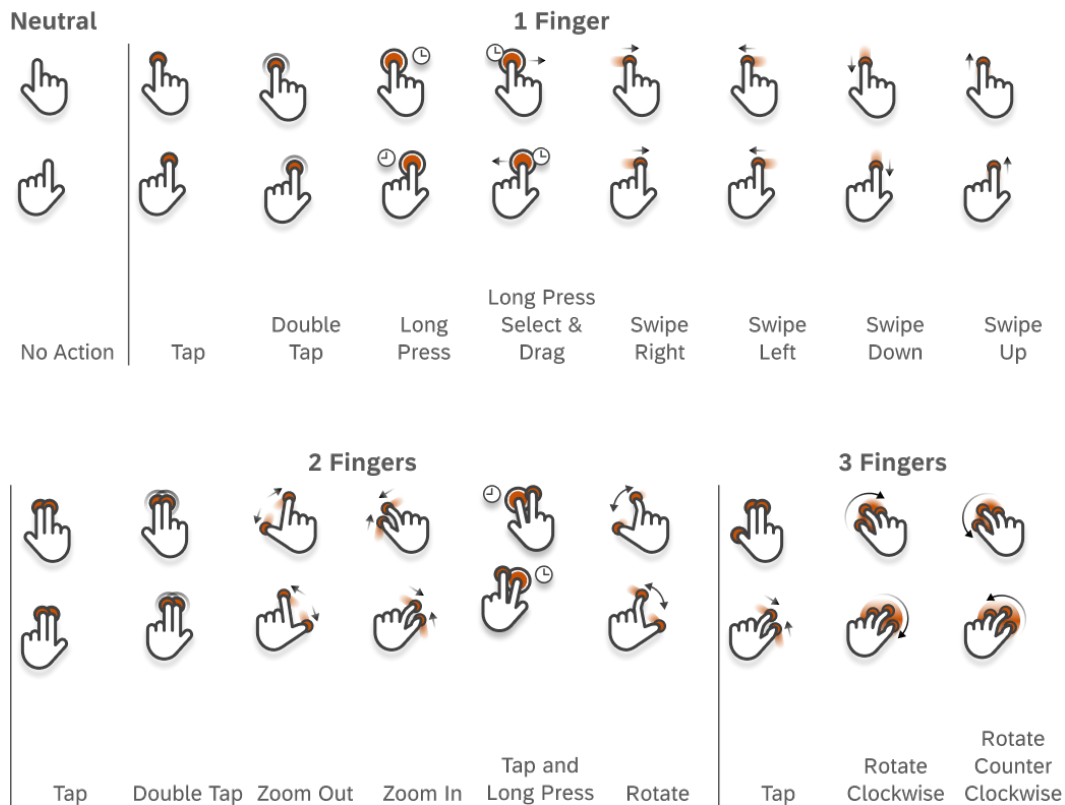


Figure 48: Touch Actions (right and left hands)

Input Mechanisms -Keyboard Keys

Interaction Keys

Alt	Ctrl	Esc	Enter	Space	A	Shift	Del	Fx
Alternate Key	Control Key	Escape	Enter	Space	Letter	Shift	Delete	Function
Combined becomes an Access Key	Combined becomes a Hot Key	Cancel, Close	Trigger action	Select		Modifier	Remove	

Navigation Keys

Tab	F6	Arrow Right	Arrow Left	Arrow Up	Arrow Down	Home	End	Page down	Page up	Others
Tab	F6	Arrow Right	Arrow Left	Arrow Up	Arrow Down	Home	End	Page Down	Page Up	Signs
Focus Order	Skipping Groups									

Figure 49: Keyboard Keys (Conventional Keys)

Input Mechanisms -Mouse Actions

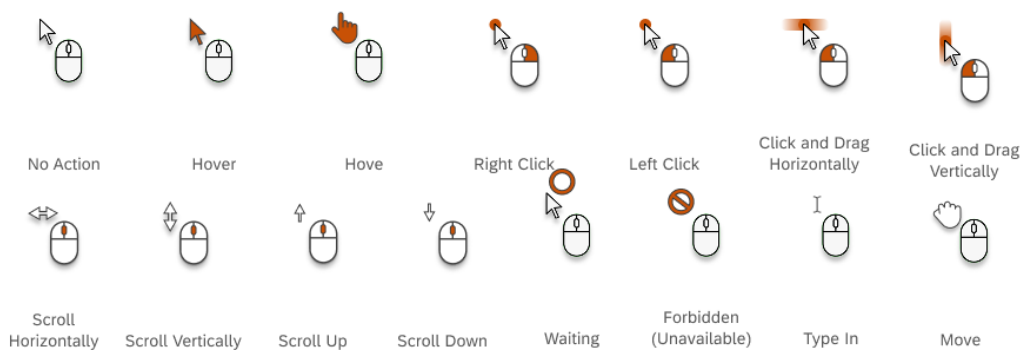


Figure 50: Mouse Actions

About Input Mechanism

Input mechanisms refer to the tools and methods users use to interact with digital interfaces. These can include a wide range of technologies—such as voice input, eye tracking, or assistive switches—but this chapter focuses on the most common: mouse, touch, and keyboard. Each mechanism presents unique interaction patterns and accessibility considerations.

Designing with all three in mind ensures users can navigate, operate, and complete tasks regardless of their preferred or required input method. This approach supports not only users with disabilities but also situational or device-based constraints, such as using a touchscreen with gloves or navigating solely by keyboard due to motor limitations.

A mouse is a typical desktop device for interaction and navigation; it is also used to extend mobile capabilities. This is the most straightforward device to interact with visual elements because it has few controls and combinations.

The touch is a standard mobile and tablet input mechanism. It can become complex due to the number of key combinations needed to create shortcuts.

Keyboard is essential to define interaction and navigation and also extend mobile capabilities. But other input mechanisms should be explored as well. For example, keyboard shortcuts are mapped from what a mouse can perform, and the same happens for touch devices.

Designing for keyboard requires selecting appropriate keys to inform how a user would operate the keyboard to interact with controls. Some key combinations allow for easy command triggering, while other keys enable commands like expanding or closing/leaving control.

Choose common patterns to allow users to reuse learned patterns. Enter or Space replaces the mouse click to access links, save, cancel, edit, open dropdowns, new pages, popovers, etc. Enter or Space is also used to trigger actions like checking and unchecking checkboxes or activating a focused button. F4 opens select lists. ESC closes dialog boxes, dropdown menus, and popovers and returns focus to the element that spawned it.



Keyboard Keys



Figure 51: Alt Key keyboard Annotation. Use to inform expected keyboard support patterns on the designed interaction

Mouse Patterns



Figure 52: Mouse annotations symbolizing no action, vertical scrolling and click and drag vertically. Use to inform expected mouse patterns on the designed interaction

Touch Patterns



Figure 53: Touch annotations symbolizing long press and zoom out. Use to inform expected touch patterns on the designed interaction

Pointer Cancellation

“Any movement of a pointer could be difficult or impossible to use for someone who cannot perform precise movements, therefore alternative forms of interaction are always recommended.” WCAG 2.1 Success Criterion 2.5.1 Pointer Gestures

WCAG 2.5.1 Pointer Gestures (A) requires that all functionality that relies on complex pointer gestures, like swiping, pinching, or multi-finger input, must also be operable through simple, single-pointer alternatives. This ensures that users who cannot perform precise or coordinated gestures due to mobility or motor impairments can still fully interact with the interface.

Complex gestures may be difficult or impossible for users with limited dexterity, tremors, or those using assistive devices like a mouth stick or

head pointer. By providing simple alternatives (like buttons or keyboard commands) designers can make interfaces accessible and inclusive for a wider range of users.

On the other hand, 2.5.2 Pointer Cancellation (A) ensures that actions triggered by pointer input, such as mouse clicks, screen taps, or stylus presses, are intentional and not accidental. To meet this requirement, the interaction must follow one of several safe patterns:

- the action occurs only on release (not on press)
- the action can be canceled by moving the pointer away before releasing
- the system provides an option to undo or confirm the action
- the interaction is essential to the functionality (e.g., in certain game mechanics).

Pointer cancellation protects users, especially those with motor impairments or using touchscreens, from unintentionally triggering critical functions. It improves usability by ensuring that users can interact with interfaces safely and confidently.

Table 6: Different interaction actions and their input variants

Action	Input Variants
Hover or Focus	 
Select or Activate	  
Scroll	  

Dragging Movement

As part of creating accessible input mechanisms, it's important to recognize that not all users interact with digital interfaces through precise gestures or fluid motion. WCAG 2.5.7 Dragging Movements (AA), introduces this requirement recommending that any functionality relying on dragging (such as reordering, drawing, or sliding) must also offer an alternative method that does not require a continuous drag gesture. This benefits users with motor impairments or those relying on assistive technologies that don't support dragging easily. For example, instead of requiring users to drag list items to

reorder them, designers should also offer simple tap or keyboard accessible “Move up/down” controls. By accounting for diverse input needs—including touch, mouse, keyboard, and assistive devices—teams can ensure that all users can perform essential actions without unnecessary physical strain or exclusion.

Design Example:

- Not enough: Only allowing users to rearrange a list by drag-and-drop.
- Accessible: Also providing “Move up/down” buttons to reorder items without dragging.

Table 7: Different dragging actions and their input variants

Action	Mouse	Input
Drag to Move	Drag to Move	 
Select or activate	Drag to Move	
Scroll	  	



Examples

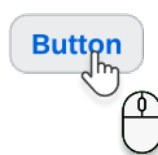


Figure 54: Long press, select and drag

Hover over link



Hover over button



Action loading

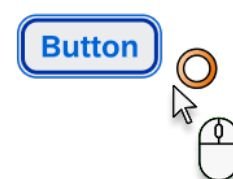
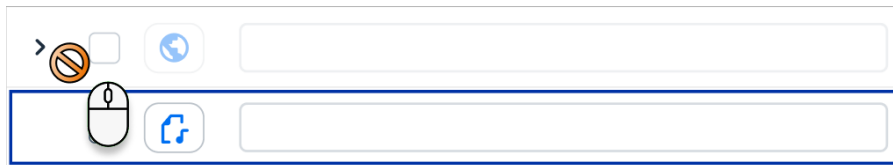


Figure 55: Mouse interactions when hovering over a link, over a button, or if an action is loading



Input

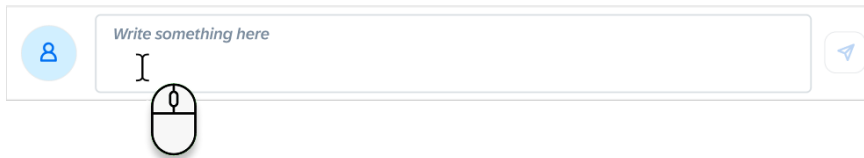


Figure 54: Drag and Drop attempt

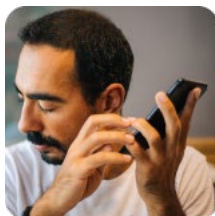
Shared Benefits



Temporary Disabilities

Everyone Benefits

Users with temporary disabilities, such as a broken arm, eye strain, or post-surgery limitations, also gain flexibility from alternative input methods. For example, someone with a wrist injury may rely on voice input or switch controls, while someone experiencing blurred vision may benefit from screen magnification or keyboard shortcuts. Providing multiple input options ensures that users can continue to interact effectively, regardless of their current physical or sensory state, supporting broader accessibility and inclusive design.



Without Vision

Screen Reading

Blind users benefit greatly from a range of input mechanisms, especially keyboard navigation, screen readers, and voice commands. The keyboard provides precise, structured navigation without relying on visual cues, while screen readers convert text and semantic structure into spoken output.

Voice input offers hands-free interaction, useful in mobile or multi-tasking contexts. These mechanisms allow blind users to interact with digital interfaces independently and efficiently when interfaces are properly designed with accessibility in mind.

Developer Notes

Web Development: Developers should check respective documentation on the web that gives basic introduction such as [Developing a Keyboard Interface | APG | WAI | W3C](#)

For native mobile development, consult the documentation for the SDKs of the given platforms:

iOS: [Support Full Keyboard Access in your iOS app - WWDC21 - Videos - Apple Developer](#)

Android: [Handle keyboard input | Views | Android Developers](#)

In all technologies, **make sure that the visible keyboard focus is fully visible** and not obscured or truncated by other controls or control containers.

References

WCAG 2.2 Reference

[2.1.1 Keyboard](#) (A)

[2.1.3 Keyboard \(No Exception\)](#) (AAA)

[2.4.7 Focus Visible](#)

[2.4.11 Focus Not Obscured](#)

[2.5.1 Pointer Gestures](#) (A)

[2.5.2 Pointer Cancellation](#) (A)

[2.5.6 Concurrent Input Mechanisms](#) (AAA)

SAP Product Standard Accessibility 4.3

[ACC 270 - Full Input Channel Support \(Keyboard, Mouse, Touch\)](#)

[ACC 271 Visible Focus](#)



Focus Order

Define the logical sequence of tab order and focus flow for focused controls in the layout. Identifies also focus branching elements.

Variants

Single Tab Stop



Figure 55: Focus Order annotation interactive elements that receives focus during TAB navigation. May indicate with a plus (+) sign additional control-internal tab stops that are not included in actual order numbering.

Multiple Tab Stops Inside



Figure 56: Variant indicating with a plus (+) sign additional control-internal tab stops that are not included in actual order numbering

Focus Scope Horizontal, Vertical and Spatial

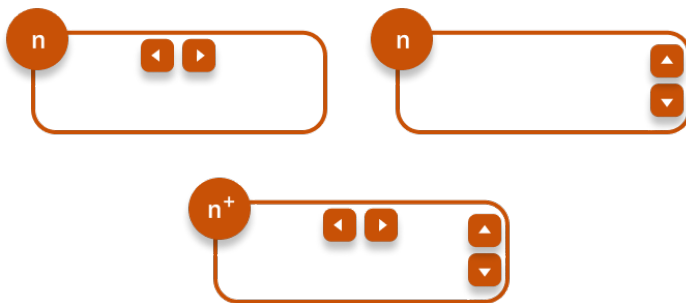


Figure 57: Represents a single Tab Stop in a branched control where navigation within the UI element require further navigation using arrow keys to explore the control vertically or horizontally, or spatial navigation. May indicate with a plus (+) sign additional control-internal tab stops that are not included in actual order numbering (Variants indicated with “+ Tab Stops Inside”).

Focus Flow



Figure 58: To make annotation efficient inform the expected focus order flow left to right or Top to Bottom.

About Focus Order

Focus order indicates the sequence of interactive controls like links, buttons, dropdowns, tabs, checkboxes, etc. Focus Order, also called Tab stops, is the preferred way of a keyboard user to explore interactive elements in the screen.

While the keyboard user perceives the focus visually with a consistent style for every interactive element when on focus, blind users listen to the announcements sent from the screen reader explaining what the control is and the available states and actions. When on focus the user can be triggered using Space or Enter.

An efficient keyboard navigation experience relies on predictable tab stops and focus order—keyboard users can move to the next element with Tab, but when landing on a complex component like a list, table, or region, they often need to use additional keys (like arrows or specific shortcuts) to explore within it before continuing.

Designers should inform the logical sequence of interactive controls leveraging the visual layout and structure used to display UI controls and groups of elements. To document focus order in the design documentation use the element that better represents the interactive UI elements in the screen.

Simple Tab stops are depicted using the outlined orange circle and a number that represents its place in the Tab order. May indicate with a plus (+) sign additional control-internal tab stops that are not included in actual order numbering.

A branched item is a new starting point — a switch strategy to continue navigation and interaction within a complex component visible on the screen (e.g., a tab, a breadcrumb, tables, a combo box, a list inside a popover). After accessing the branched control using TAB, the user explores and navigates the branched control using Arrow keys and Home/End keys, and eventually TAB, therefore it may indicate with a plus (+) sign additional control-internal tab stops that are not included in actual order numbering.

SAP Figma Plugin

The [SAP Figma Plugin “Focus Order”](#) helps designers annotate their screens identifying relevant focus order sequence.



Examples

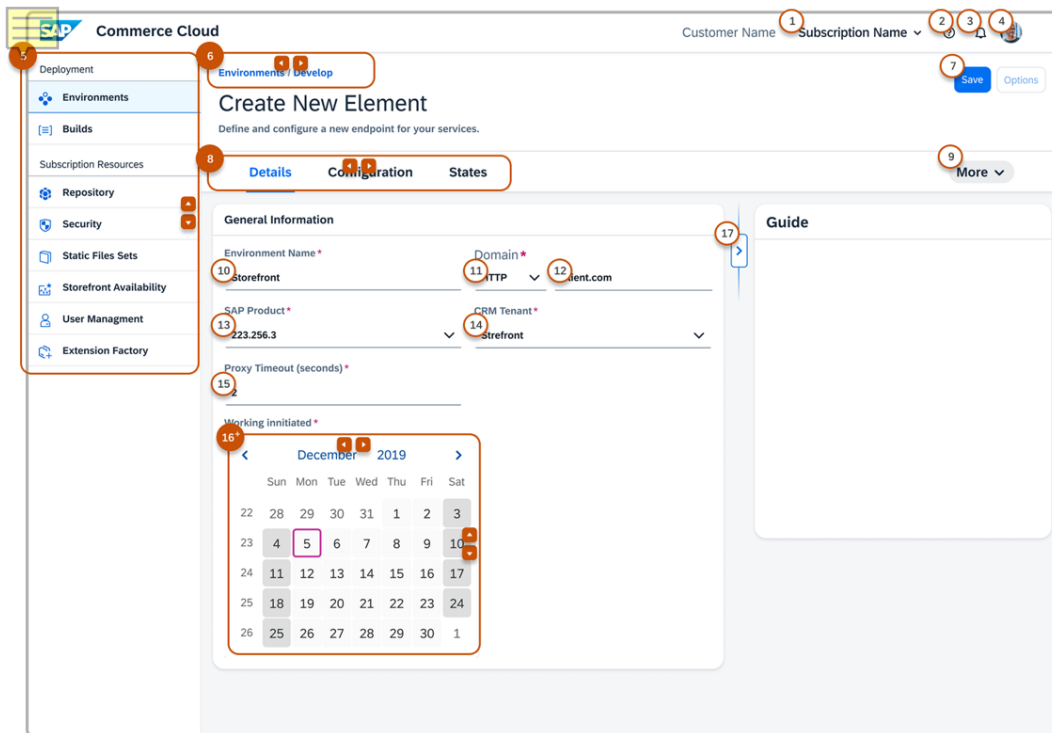
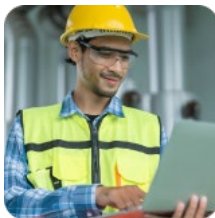


Figure 59: Focus Order sequence including branching and components with arrow navigation and inner additional tabs (indicated by “+” sign)

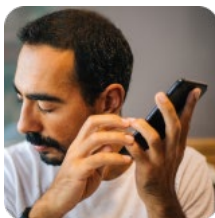
Shared Benefits



Temporary Disabilities

Everyone Benefits

Tab stops or focus order aren't just for accessibility—they're also used by many people who rely on the keyboard to move quickly through forms and interfaces, making navigation faster and more efficient without needing a mouse.



Without Vision

Screen Reading

For blind users using screen readers, tab stops and focus order are essential for navigating and understanding content in a logical, predictable

sequence—ensuring they don't miss important fields, actions, or instructions.

Developer Notes

For Web Development, Tab Order can be programmed using two different concepts:

By DOM sequence plus nesting in layout groups

```
<div>
  <div id="group1"
class="vertical">
    <button>A</button>
    <button>B</button>
    <button>C</button>
  </div>
  <div id="group2"
class="vertical">
    <button>D</button>
    <button>E</button>
  </div>
</div>
```

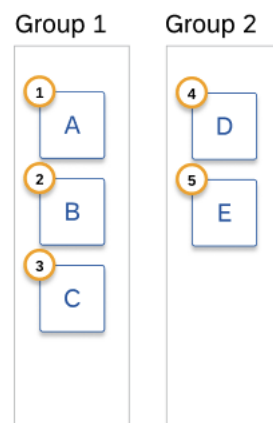


Figure 60: DOM sequence nesting in layout groups

By tabindex property

```
<button tabindex="1">Save</button>
<button
tabindex="3">Delete</button>
<button
tabindex="2">Cancel</button>
```

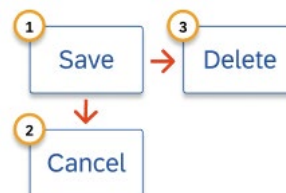


Figure 61: Tab sequence different from default by applying individual tab indices

Notes:

- With `tabindex="0"` any element can be made to receive keyboard focus. Use this **ONLY** for interactive elements (with action handlers associated).
- With `tabindex="-1"` any element can receive focus **ONLY** by code (by JS focus setter)

- `tabindex > 0` to be avoided: is redundant or may cause confusion

To understand the basics, developers should check also documentation on the web that gives an introduction (such as [Developing a Keyboard Interface | APG | WAI | W3C](#)) and then study the best practices for applying correct focus order in WCAG 2.2 [2.4.3 Focus Order](#).

In native mobile development, platform dependent procedures must be applied:

iOS: Focus order depends on the structure of the view.

```
var body: some View {
    VStack {
        TextField("First name", text: $firstName)
            .modifier(InputModifier())
            .focusable()
        TextField("Last name", text: $lastName)
            .modifier(InputModifier())
            .focusable()

        HStack(alignment: .top) {
            Button(action: {}) { Text("Save") }
                .accessibility(label: Text("Save user data and continue
order form"))
            Spacer(minLength: 10)
            Button(action: {}) { Text("Cancel") }
                .accessibility(label: Text("Cancel order form"))
        }
    }
}
```

Android:

Handle tab navigation by layout

```
<RelativeLayout ...>
<Button
    android:id="@+id/button1"
    android:layout_alignParentTop="true"
    android:layout_alignParentRight="true"
    android:nextFocusForward="@+id/editText1"
... />
<Button
    android:id="@+id/button2"
```



```
android:layout_below="@id/button1"
android:nextFocusForward="@+id/button1"
... />
<EditText
android:id="@id/editText1"
android:layout_alignBottom="@+id/button2"
android:layout_toLeftOf="@id/button2"
android:nextFocusForward="@+id/button2"
... />
...
</RelativeLayout>
```

Handle directional navigation by property (D-Pad or arrow keys)

```
android:nextFocusUp
android:nextFocusDown
android:nextFocusLeft
android:nextFocusRight
```

To understand the basics, also consult the documentation for the SDKs of the given platforms:

iOS: [Support Full Keyboard Access in your iOS app - WWDC21 - Videos - Apple Developer](#)

Android: [Handle keyboard input | Views | Android Developers](#)



References

WCAG 2.2 Reference

[1.3.2 Meaningful Sequence](#) (A)

[2.4.3 Focus Order](#) (A)

[2.4.7 Focus Visible](#) (AA)

SAP Product Standard Accessibility 4.3

[ACC 272 - Tab/Reading Order](#)



Initial Focus Position

Mark the UI element that gets the Initial Focus when the page loads or when entering a skip block or a control.

Variants

Page or Dialog



Figure 62: Informs which interactive UI element on the screen will receive focus when page loads or a dialog is displayed.

Skipping Group



Figure 63: Informs which interactive UI element inside a skipping group will receive focus when exploring groups on the page [F6].

Control



Figure 64: Informs which part of a focusable control (simple or complex) will receive focus initially.

Focus



Figure 65: Plain Focus annotation, useful if you just want to display a focus ring somewhere in your design

About Initial Focus

Initial Focus is the position highlight of one interactive UI element - a visual outline is usually used to inform Focus. The annotation initial focus informs the developer to make a certain interactive element focused in preparation for the keyboard experience.

When the system is prepared to inform current focus, a keyboard user feels confident about exploring the page and making interactions. Focus is a cue that an interactive element is active and ready to respond to input, whether from a keyboard or other assistive devices. This state ensures that users can seamlessly navigate and interact with the interface, enhancing usability and accessibility. In essence, focus represents the state where an interactive UI element is primed to receive user input, facilitating efficient and inclusive digital experiences.

The initial focus is given to the most essential element of a workflow in two different contexts:

- **On Page Load:** When a user is transported to a new page, the initial focus will land on the first most important interactive UI element to enable the user to start his task enabling the user to proceed with keyboard navigation [TAB].
- **During Skipping Group:** navigation which works with the keyboard shortcut [F6]. The most common navigation strategies are Focus Order (TAB) to explore interactive elements on a screen (links, buttons, form fields) and Skipping Groups (F6)* to skip entire groups on the screen.
- **Controls and Complex components:** It is also useful to inform where focus should be located on a group components like table, toolbar, lists, carousels and calendar. The focus is also presented on the selected item for single selectable item from a list, menu, or tabs. For multi-selectable items, the focus goes to the first selected item.

The most essential element of a workflow marked as initial focus is also the first to be announced when a blind user lands on a page or if they choose to perform Skipping Group navigation using the keyboard shortcut [F6].

The concept of initial focus serves as a visual cue to indicate "You are here" on a page, providing essential orientation for users, particularly those who rely on keyboards to navigate digital products.

Keyboard focus is a critical accessibility feature that applies exclusively to interactive UI elements, such as buttons, links, or checkboxes. Making this focus visible is vital, as it allows users to identify the element they can interact with, enabling them to trigger actions or explore further to access nested functionalities.

The initial/default focus position on the screen and when entering groups is typically set on the framework level. In many UI technologies, applications can override the default focus position and put it to a logical starting point on the screen instead.



Focus Appearance

The appearance of the focus is typically shown as an outline border or highlight around the focused element. It is automatically provided by web browsers and varies based on the browser. the WCAG 2.4.13 Focus Appearance (Minimum) (AA) ensures that the keyboard focus indicator is clearly visible and meets minimum design requirements enabling users to reliably see which element is currently focused. The appearance of the visual focus on controls is defined by a design system and implemented to look like part of a brand - it's a requirement to meet visual style and contrast.

Focus Visibility

While the WCAG 2.4.7 requires a visible focus indicator appearing whenever a user navigates to an element using the keyboard, the WCAG 2.4.11 requires that when a user tabs to an element, at least part of it must be visible on the screen (i.e., not hidden behind sticky headers, modals, or overlays). The key concerns are:

1. Ensure users can see where keyboard focus is. Is focus style visible at all (like a border, outline, or highlight)?
2. Ensure users can access and interact with focused elements without confusion or being “lost.” Is the focused element obstructed or off-screen?

If you choose to make the entire element fully visible on focus without needing the user to scroll or adjust the view, you are then fulfilling the 2.4.12 (Enhanced – AAA).



Examples

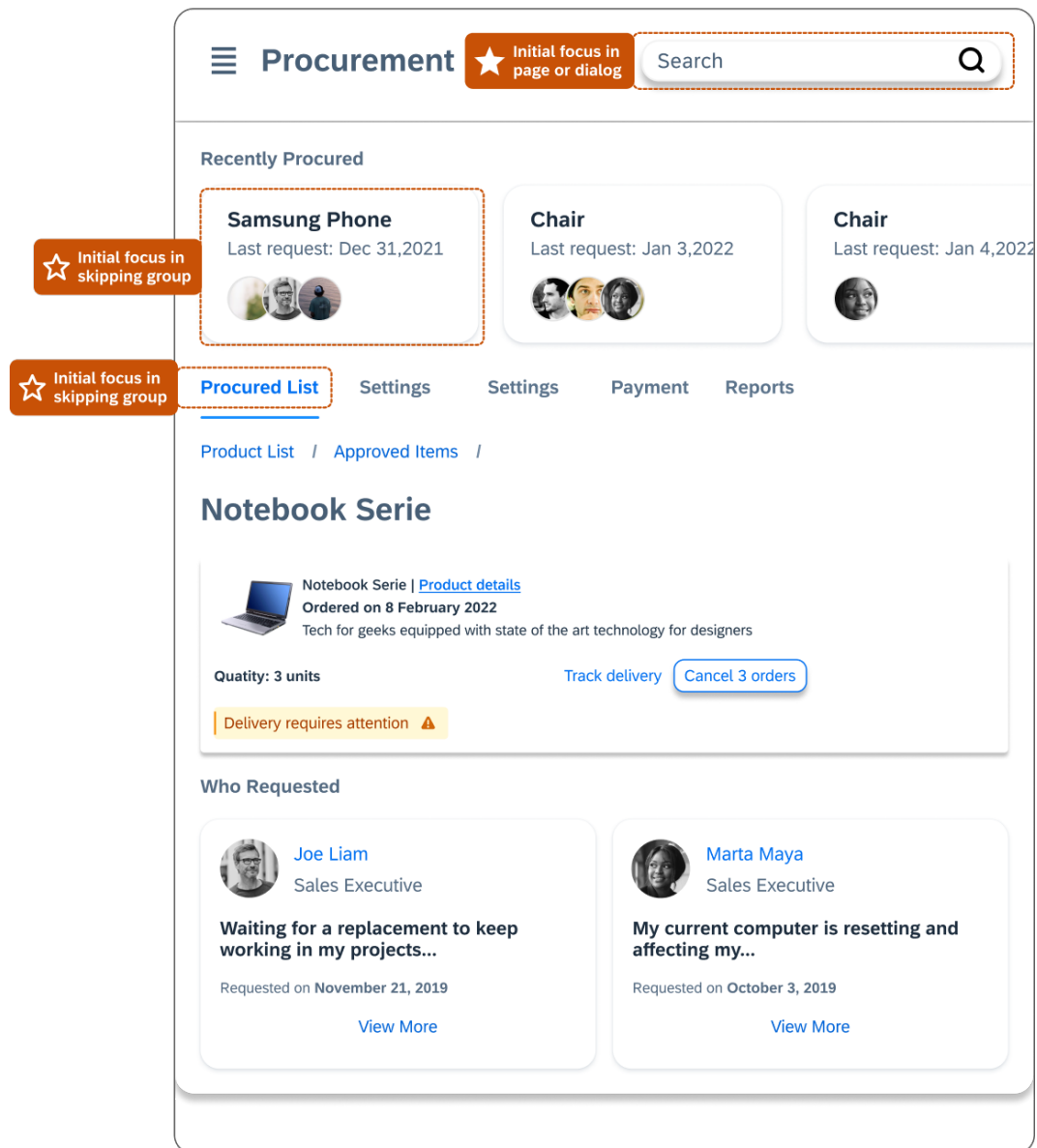


Figure 66: Initial focus position on page load and in skipping groups

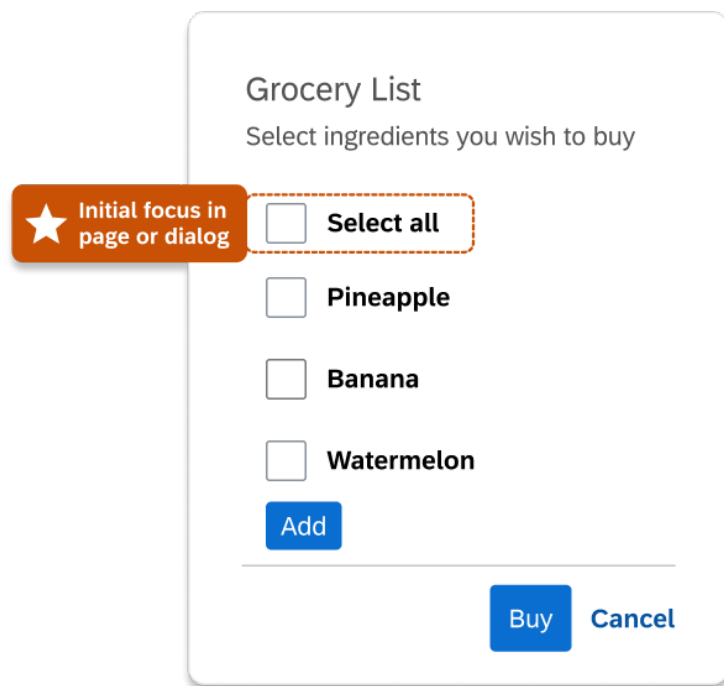


Figure 67: Initial focus position in a dialog

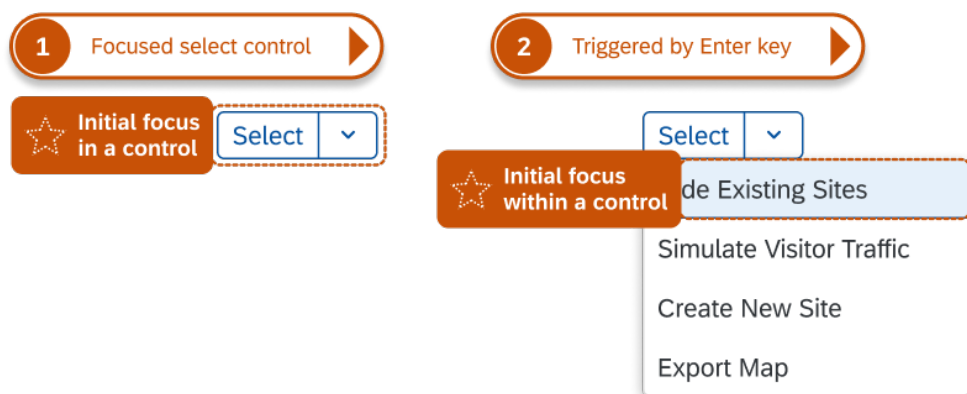


Figure 68: Initial Focus in controls
Menu Button Split Mode - Menu Opening



Shared Benefits



Temporary Disabilities

Everyone Benefits

Thoughtful initial focus design helps users stay oriented, act faster, and remain confident while navigating complex interfaces. When focus starts in the right place, such as the first actionable field in a form, or the close button of a modal, it does not only save time but reduces physical strain and cognitive effort for expert users, especially in enterprise or productivity environments where agility is key.

Developer Notes

Web Development: Developers should check respective documentation for setting focus on particular elements:

- [HTML autofocus global attribute - HTML | MDN](#)
- [HTMLElement: focus\(\) method - Web APIs | MDN](#)

For native mobile development, consult the documentation for the SDKs of the given platforms to set the focus and check the following code snippets:

iOS:

Use @FocusState for tracking or setting a focus position.

```
enum Field: hashable {
    case email
    case password }

@FocusState private var focusedField: Field?
TextField("Email", text:$email).focused($focusedField,
equals: .email)
SecureField("Password", text:$pwd).focused($focusedField,
equals: .password)

.onSubmit {
    if !isEmailValid {
        focusedField = .email
    }
}
```



See also:

- <https://developer.apple.com/videos/play/wwdc2021/10023/>
- <https://developer.apple.com/documentation/swiftui/focusstate>

Android:

Programmatically set focus on a view when an activity opens

```
private EditText mEdit;
mEdit = (EditText) findViewById(R.id.myEdit);
mEdit.requestFocus();
```

See also:

- [https://developer.android.com/reference/android/view/View#requestFocus\(int\)](https://developer.android.com/reference/android/view/View#requestFocus(int))
- [Change focus behavior | Jetpack Compose | Android Developers](#)

References

WCAG 2.2 Reference

[2.4.7 Focus Visible](#) (AA)

[2.4.11 Focus Not Obscured \(Minimum\)](#) (AA)

[2.4.12 Focus Not Obscured \(Enhanced\)](#) (AAA)

[2.4.13 Focus Appearance](#) (AAA)

SAP Product Standard Accessibility 4.3

[ACC 271 - Visible Focus](#)



Focus Restore

Once the user completes a task, the keyboard focus returns to the trigger or to the next logical UI element if trigger is not present any more.

Variants

Focus Restore to Trigger

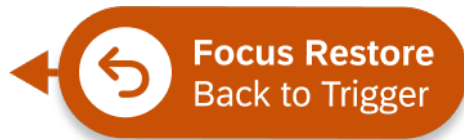


Figure 69: Use to inform which UI element was used to trigger and where the focus should return.

Focus Restore to Next Logical Element

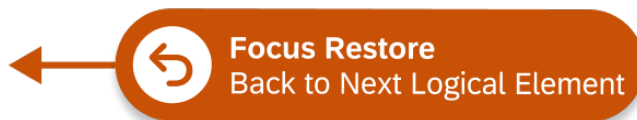


Figure 70: Use to inform where the next logical UI element should receive focus at the end of the flow.

About Focus Restore

Designing Predictable and Inclusive Interaction Flows is only complete if Focus Return is considered. Focus management is a recurring source of accessibility defects, particularly for keyboard and screen reader users. One critical, yet often overlooked, aspect is focus restore—the practice of returning keyboard focus to a logical and expected location after a task or interaction is completed. Proper focus restore ensures that users don't lose their place in the interface, especially after dismissing dialogs, completing forms, or removing elements from a list. Designing and annotating where focus should land after a flow demonstrates thoughtful attention to keyboard support and helps reinforce this requirement during implementation.

This annotation evolved from the concept of Predictable Context, previously featured in the Foundation chapter of the Accessibility Design Tools by SAP. As the toolkit matured, Predictable Context was split into two distinct annotations: Focus Restore, which addresses the interactive experience, and Wayfinding & Orientation, which supports the cognitive

experience. Together, these annotations help ensure users maintain context and continuity throughout an interaction.

Designing for focus restore requires an understanding of the predictability of user flows. However, there are scenarios where simply restoring focus to the original trigger may not be possible or logical—for example, when the trigger element (like a deleted list item) no longer exists, or when the item has been repositioned in a way that breaks continuity. In such cases, designers must plan alternative destinations for restored focus—such as the closest logical sibling element, the updated container, or a clearly labeled notification.

By planning for focus behavior early and documenting it through annotations, teams reduce the risk of disorientation and accessibility regressions. Focus restore is more than a technical fix—it is a user-centered practice that supports continuity, efficiency, and confidence for users who rely on keyboard navigation.

Focus Restore: Back to Trigger

When an interaction is completed—such as dismissing a dialog, popover, or overlay—focus should return to the element that originally triggered the interaction. This annotation is used to inform a pattern when the trigger is still relevant and present in the UI, and the user journey expects a return to the point of origin. For example, closing a modal that was opened by a button or icon within a list item should return focus to that same button. This helps users stay oriented and maintain continuity in their workflow.

Design tip: Use when the trigger remains visible, unchanged, and still represents the next logical step in the user's task. It reinforces predictability and supports reactivation or re-entry.

Focus Restore: Back to Next Logical Element

In some cases, returning focus to the original trigger isn't appropriate—especially if the trigger was deleted, moved, or is no longer relevant. In these cases, focus should move to the next logical element in the user's workflow. This annotation is used when the action modifies the structure of the page (e.g., deleting a row in a table). Instead of returning focus to a missing or moved trigger, designers should identify the most relevant next step in the task flow—such as the next row in a list, a message area, or an adjacent control. This approach prevents disorientation and supports task progression.

- **Design tip:** Requires awareness of the user's journey and intended task flow. Designers must anticipate what the user is likely to do next and



annotate the interface accordingly to maintain momentum and minimize confusion.

Examples

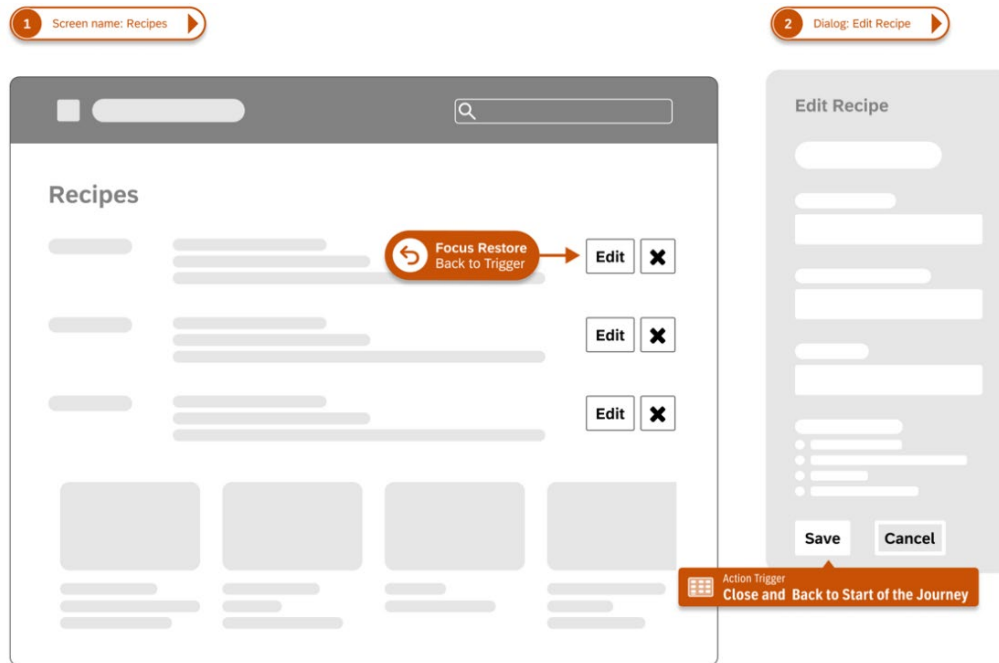


Figure 71: Focus Restore and Action Trigger Annotations in an Edit Recipe use case

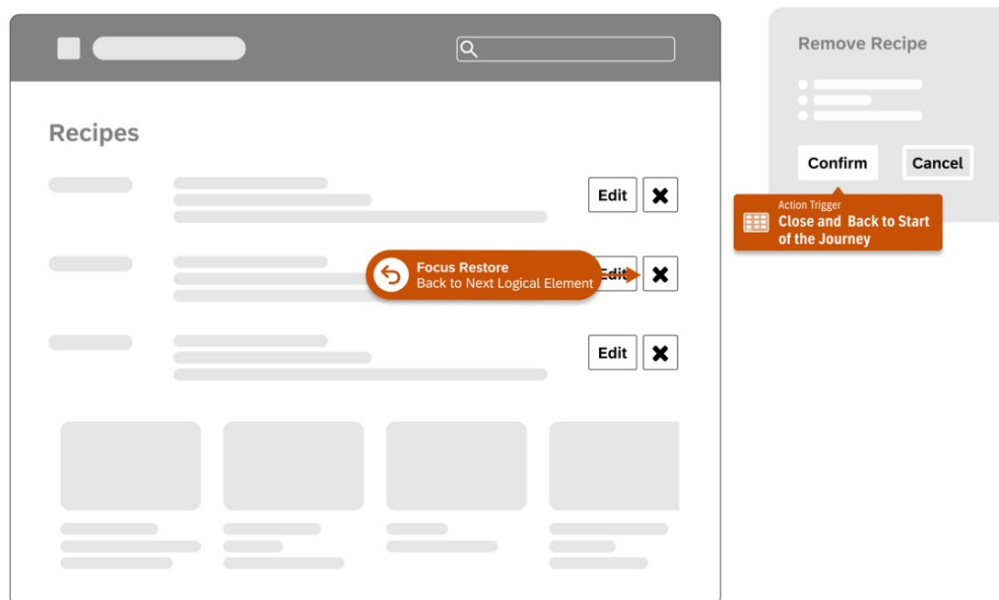


Figure 72: Focus Restore annotation applied on the trigger UI element after the invoked dialog was closed

Shared Benefits



Temporary Disabilities

Everyone Benefits

Older users may experience a combination of physical, cognitive, or sensory limitations. Clear, predictable navigation—including focus restore—supports confidence and independence in digital environments.



Without Vision

Screen Reading

These users rely entirely on focus order to understand where they are in an interface. Without proper focus restore, they may be dropped at the top of the page or in an unrelated area, causing confusion and disorientation. Predictable focus behavior helps maintain context and mental mapping of the UI.



Vision and Color Limitation

Visuals

Users with low vision often use screen magnifiers or high zoom levels. If focus is not restored predictably, they may lose their visual context, especially if the new focus is off-screen or unexpected. Proper focus placement avoids unnecessary panning or hunting for the next step.



Limited Cognition

Cognitive

Predictability and continuity are crucial for users who may struggle with memory, attention, or processing complex interactions. Sudden or illogical focus jumps can increase cognitive load, leading to task abandonment. Focus restore supports smoother interaction flows, reducing frustration.

Developer Notes

1. Check all info of last chapters as basics.
2. Develop a concept of storing and restoring the focus of the last element focused. Two cases apply:
 - a. The element previously focused is still visible. Put the focus back on it (typical case: a button opens a dialog, dialog closes, put focus back on button)
 - b. The last focused element has completely disappeared (page navigation scenarios). Remember last focused elements, query this info in page navigation in all “Go Back” cases and restore the focus on the element previously focused

References

WCAG 2.2 Reference

[3.2.1 On Focus](#) (A)

[3.2.3 Consistent Navigation](#) (AA)

SAP Product Standard Accessibility 4.3

[ACC 273 - Robust Context](#)

Skipping Group

Allow users to skip to the main content or skip big blocks of content that repeat on many pages to increase user productivity.

Variants

Skipping Group



Figure 73: Identifies a group of UI elements that can be skipped (bypassed tab chain) with a designated keyboard hotkey.

About Skipping Group

Skipping groups is a mechanism described by WCAG to bypass blocks of content that are repeated on multiple Web pages.

People with certain disabilities have difficulty reaching the main content of a page quickly and easily. For instance, screen reader users who visit several pages on the same site can listen to fewer announced words (imagine 200 words less to listen to) from heading and navigation links on every page and get directly to the main content. These users will appreciate skipping to the main content because they will feel more productive and efficient.

The same efficiency applies to keyboard users who can use fewer keystrokes to reach the main content avoiding extra keystrokes to reach the content they are looking for. Skipping to the main content also avoids severe physical pain for some users. Skipping to the main content is also convenient for users of screen magnifiers as they avoid long navigation paths, including headings or blocks of information, each time they enter a new page. These users appreciate a quick access to the main content to avoid a manual search for the location of the main body using a screen magnifier.

We propose Skipping Group annotation to bypass blocks framing skipping sequence regions. These groups speed navigation and is associated with navigation efficiency.

Annotate a skip block defining a group of contextual information. Use a logical order to guide users through Skipping navigation in the same way they would notice the visual layout hierarchy.



A mechanism to bypass blocks is using the F6 Key. Reverse navigation utilizes SHIFT + F6 Keys to bring the user back to the previous Skip Block. Microsoft originally used the shortcut F6 on browsers to switch focus between frames in a frameset and in Windows applications to switch between app sections.

Allow users to understand the hierarchy across groups during navigation. The direction of Skipping navigation is useful on complex layouts. Foresee where the Initial Focus will be when landing on a new group by using the Focus Placement annotation.

Examples

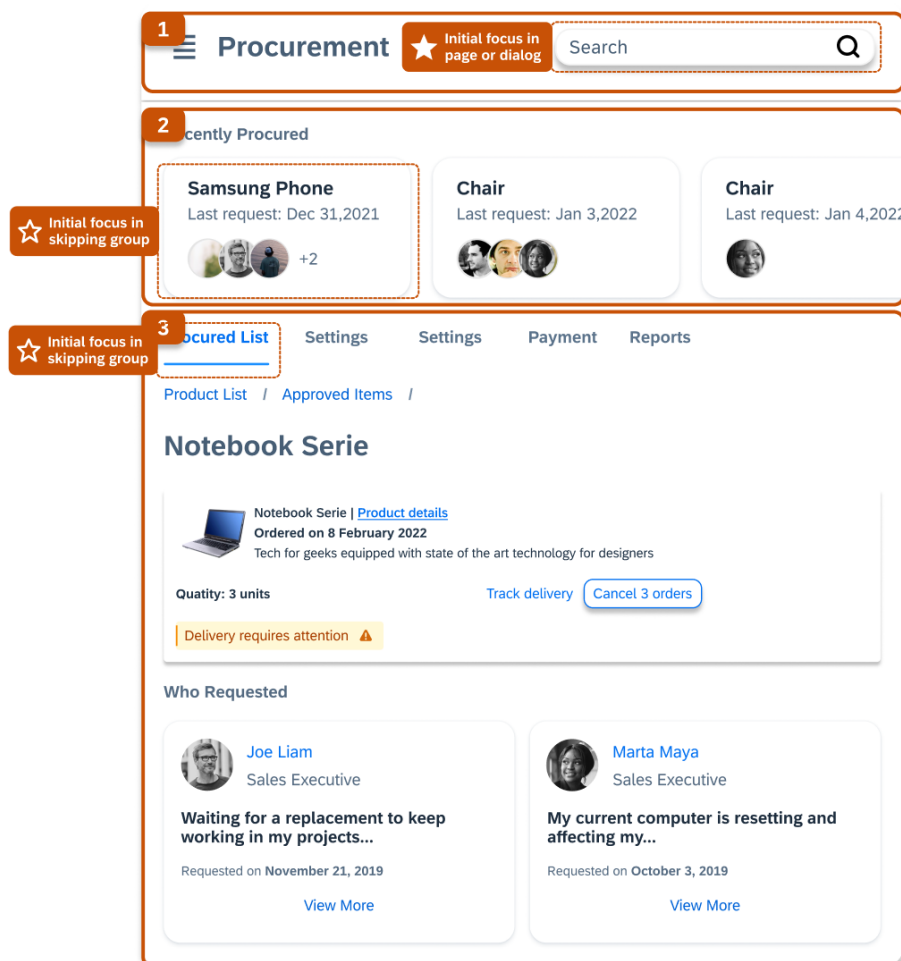


Figure 74: Skipping group sequence with initial focus positions in a Procurement page

Shared Benefits



Temporary Disabilities

Everyone Benefits

Expert users benefit from the Skipping Groups annotation because it supports faster navigation through large or repetitive content areas. By enabling shortcuts (such as F6) to jump between major interface regions, like toolbars, content areas, or side panels, expert users can quickly reach their target without unnecessary tabbing, enhancing workflow efficiency in complex applications.



Without Vision

Screen Reading

Blind users also rely heavily on group-skipping functionality to avoid navigating through every item in a dense interface. When screen readers announce the ability to skip to specific regions, it helps blind users maintain orientation and reduce the time spent moving sequentially through unrelated elements. This improves usability, supports independence, and aligns with the structure they expect from well-designed, accessible interfaces.

Developer Notes

Web Development:

Define a concept of “skippable” groups for framework components. “Groups” in this sense are large complex controls like tab strips and tables or control containers like panels and forms.

Technically, implement check JS keyup listener + `el.firstChild.focus()` with event bubbling cancelled like so:

```
document.addEventListener('keyup', (e) => {
  // forward navigation if (e.keyCode == 117 ) { // f6
  keycode
    e.preventDefault();
    e.stopPropagation();
    var el = findNextSkipContainer();
    el.firstChild.focus(); // default, can be
    overridden
  }
}, false);
```

For native mobile development, consult the documentation for the SDKs of the given platforms to set the focus and check the following code snippets:

iOS:

Options to structure a screen:

- Use heading trait to structure the screen (advances voice over navigation)
- Use focusSection to declare a group (advances keyboard- or remote control):

[https://developer.apple.com/documentation/swiftui/text/focussection\(\)](https://developer.apple.com/documentation/swiftui/text/focussection())

```
import SwiftUI

struct ContentView: View {
    @State private var isEnabled = false

    var body: some View {
        VStack {
            Button("Print message") {
                print("Hello World!")
            }

            Button("Delete message") {
                print("Message deleted.")
            }
        }.keyboardShortcut("p")
        .focusSection()
    }
}
```

- Use app shortcuts to move focus on your own (advances keyboard control)



Android:

For entire app and activities declare a custom “skipping” key using keyUp:

```
public static final int KEYCODE_F6 = 136;
@Override
public boolean onKeyUp(int keyCode, KeyEvent event) {
    switch (keyCode) {
        case KeyEvent.KEYCODE_F6:
            skipToNextPanel();
            return true;
        default:
            return super.onKeyUp(keyCode, event);
    }
}
```

References

WCAG 2.2 Reference

[2.4.1 Bypass Blocks](#) (A)

SAP Product Standard Accessibility 4.3

[ACC 271 - Visible Focus](#)



Shortcut

Give expert users shortcuts (hotkeys and access keys) to enhance productivity and increase work efficiency.

Variants

Hotkey



Figure 75: Associate a Hotkey to a button to enable users to trigger the action using a shortcut.

Access key



Figure 76: Associate an Access Key to a button to enable users to navigate to the button using a shortcut.

Transfer Focus - Navigation



Figure 77: Inform when a group control requires a switch navigation to continue exploring element within a container. Adding commonly used shortcut to switch navigation within groups speed annotations.

About Shortcut

Shortcuts are keyboard combinations that enhance productivity by providing an efficient way for the user to access functionality. Shortcuts are available as hotkeys or access keys.

Hotkeys

A hotkey allows users to trigger essential and frequently used functions from anywhere in the interface. It uses the CTRL + letter keys.

For example, you can run a search by pressing Enter key when on the search field or save data using CTRL+S.

If you want to inform available hotkeys on the screen to prevent users from accessing documentation, consider using an overlay popup or popover or show them in control context menus. Displaying shortcuts in native tooltip of browsers is a bit problematic since speech output of tooltips can be disabled by screen reader users. Instead, inform these users about shortcuts using respective control properties.

Access keys

An access key promotes more efficient navigation by making a control available anytime, anywhere in the UI. They typically consist of a combination of ALT + letter or number keys. An access key shortcut does not trigger essential and frequently used control; it just moves the focus to the control the access key is bound to.

Information of available access keys is typically visualized by underlining the first character of the control label the access key is associated with.

Transfer focus for inner navigation

When navigating a web interface using the Tab key, users typically move between interactive elements such as buttons, links, and form inputs. However, when they encounter group components, like lists, menus, or grids, the navigation behavior often shifts. To avoid excessive tabbing and improve efficiency, these groups commonly support arrow key navigation. For example, in a simple list or menu, users can move vertically using the Up and Down arrow keys, allowing them to explore content quickly and logically without losing their place in the interface. This strategy preserves logical flow and reducing unnecessary keystrokes

Things become more complex when list items or group elements include nested interactive controls, such as buttons, dropdowns, or editable fields. In these cases, users need a way to intentionally enter and exit the inner content. Here, shortcuts like F2 are used to transfer focus into the item, allowing interaction without disrupting the outer navigation flow. This technique ensures that users can browse lists or menus at a high level and



only dive deeper when needed, improving efficiency for keyboard users and reducing cognitive load by minimizing unintentional focus jumps. This approach prevents unintended focus shifts and ensures that users can efficiently navigate between higher-level items while accessing nested interactive elements only when needed.

Additional shortcuts, like F4 to expand items or F6 to skip between interface regions or groups, further support users in navigating complex interfaces. These keys allow users, especially those with limited mobility or who rely on assistive technologies, to move quickly and predictably across the UI. This structured and layered navigation model aligns with accessibility best practices by maintaining logical focus flow, preserving user orientation, and offering more control and flexibility in how users interact with nested or grouped components.

Examples



Figure 78: Hotkey and Transfer Focus annotations used to mark respective controls in an UI

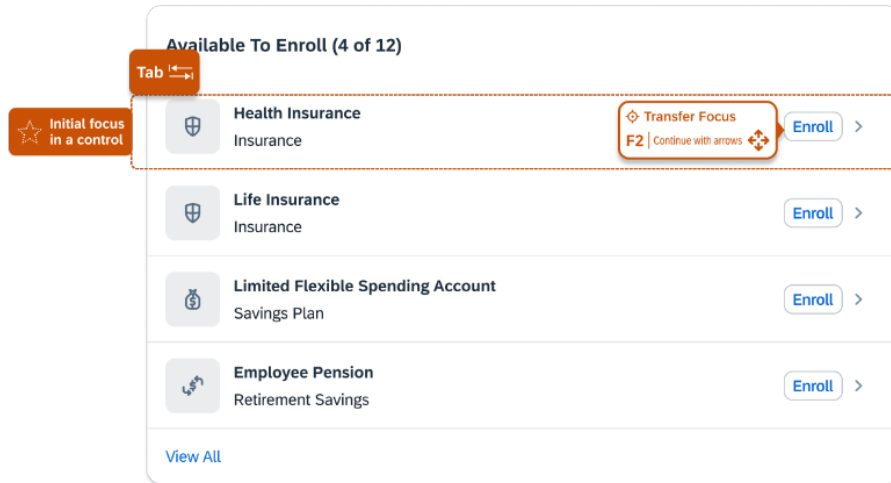


Figure 79: Example for Transfer Focus, Initial focus in a control and Tab Annotations.

Edit button shortcut can be activated using hotkey CTRL+E or accessed using ALT+E.

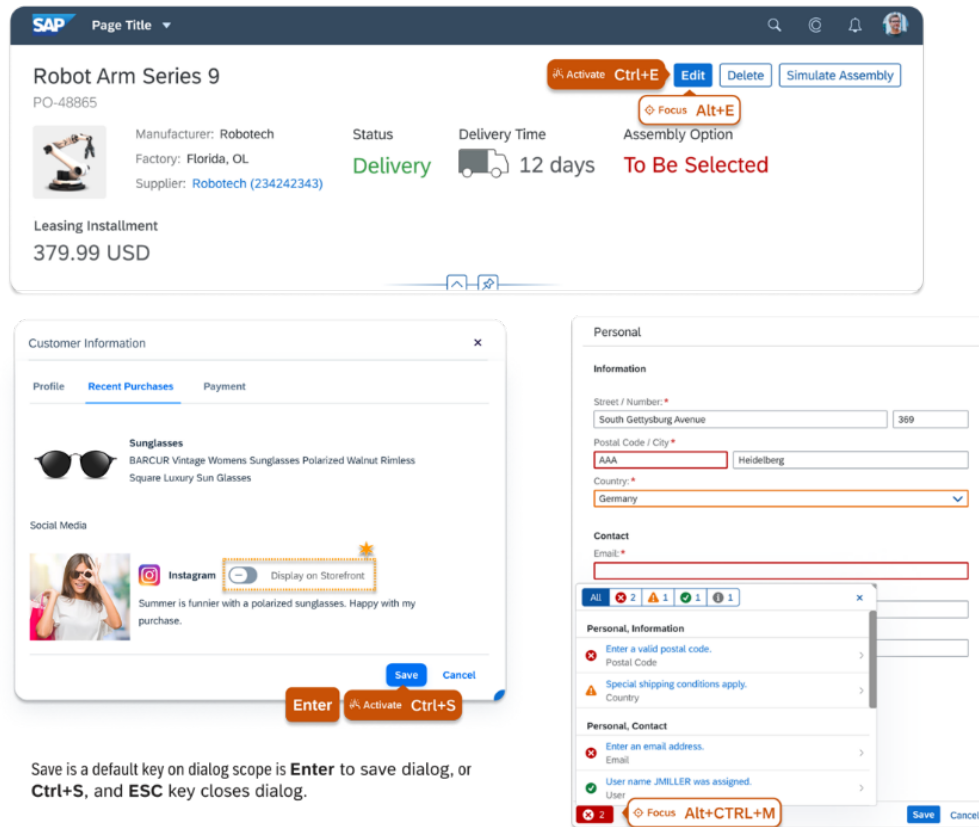


Figure 80: Examples for Hotkey and Access Key Annotations.

Shared Benefits



Temporary Disabilities

Everyone Benefits

Expert users benefit from shortcuts because they enable faster, more efficient interaction with digital interfaces. By bypassing multiple clicks or tab stops, shortcuts reduce friction and support productivity, especially in complex or repetitive workflows. This allows experienced users to stay in flow and complete tasks with greater speed and precision.



Without Vision

Screen Reading

Blind users rely on shortcuts to navigate interfaces without having to move through every element sequentially. Shortcuts provide direct access to key regions or actions, making the experience more efficient and predictable when using screen readers. This reduces cognitive load and improves orientation, especially in dense or dynamic interfaces.

Developer Notes

Web Development:

Hotkeys: By JS keyup listener with event bubbling cancelled

```
document.addEventListener('keyup', (e) => {
  if (e.ctrlKey &&
    String.fromCharCode(e.keyCode).toLowerCase() === 'e') {
    e.preventDefault();
    e.stopPropagation();
    // Do edit stuff...
  }
}, false);
```

Note: Register hotkeys and default key for a view at top level
 container^[1]_{SEP}(container collects all defined keys by a subscription model)

Access Keys: by `accesskey` property

```
<button accesskey="s"><span
class="underline">S</span>ave</button>
```

Note: accesskey declarations have local scope (managed by the browser)

Native Mobile Development:

iOS

For defining shortcuts, see for instance

<https://swiftwithmajid.com/2020/11/17/keyboard-shortcuts-in-swiftui/>

```
var body: some View {
    HStack {
        VStack {
            Button ("1") {}
                .keyboardShortcut("1", modifiers: [.control])
            Button ("2") {}
                .keyboardShortcut("2", modifiers: [.control])
            Button ("3") {}
                .keyboardShortcut("3", modifiers: [.control])
            Spacer()
        }
        .border(Color.white, width: 2)
        ....
    }
}
```

Android

Track Shortcuts

```
@Override
public boolean dispatchKeyEvent(KeyEvent event) {
    if (Build.VERSION.SDK_INT > 10 &&
        event.getAction() == KeyEvent.ACTION_DOWN &&
        event.isCtrlPressed()) {
        String actionType = "NONE";
        final int keyCode = event.getKeyCode();
        switch (keyCode) {
            case KeyEvent.KEYCODE_C:
                actionType = "COPY";
        }
    }
}
```



```

break;
case KeyEvent.KEYCODE_V:
    actionType = "PASTE";
    break;
...
}
return true;
}
return super.dispatchKeyEvent(event);
}

```

Populate Keyboard Shortcut Overview

```

override fun onProvideKeyboardShortcuts(
    data: MutableList<KeyboardShortcutGroup>?,
    menu: Menu?,
    deviceId: Int
) {
    super.onProvideKeyboardShortcuts(data, menu, deviceId)

    // Requires API 24
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.N) {
        val keyboardShortcutGroup = KeyboardShortcutGroup("Test
Group")
        keyboardShortcutGroup.addItem(KeyboardShortcutInfo("Shortcut
One", KeyEvent.KEYCODE_Z, KeyEvent.META_ALT_ON))
        data?.add(keyboardShortcutGroup)
    }
}

```

See for instance [Register hardware keyboard shortcuts in Android's help menu - Stack Overflow](#)

References

WCAG 2.2 Reference

[2.1.1 Keyboard](#) (A)

[2.1.2 No Keyboard Trap](#) (A)

[2.1.4 Character Key Shortcuts](#) (A)

SAP Product Standard Accessibility 4.3

[ACC 270 - Full Input Channel Support \(Keyboard, Mouse, Touch\)](#)



Trigger

Use a trigger to inform the beginning of a journey. Inform what's expected after button or link is triggered and respect semantics: a link should look like and behave like link - the same applies to buttons, although there may be exceptions. Clarity is key and every interactive element should communicate not just what it is, but what it will do.

Variants

Action Trigger

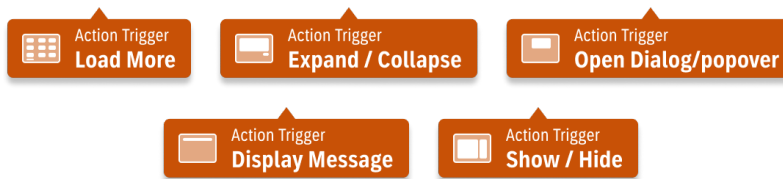


Figure 81: Inform what's expected from the button when triggered. Common pattern keeps the user in the same page.

Navigation Trigger



Figure 82: Inform what's expected from the link (navigation) when triggered. Common pattern is open a new page.

About Trigger

Providing annotations that explain triggers and their outcomes—such as buttons and links—is essential for meeting accessibility requirements and ensuring an inclusive user experience. Clear annotations help designers and developers define the purpose and expected behavior of interactive elements, reducing ambiguity and improving consistency in implementation. Therefore, designers should be clear about the designed experience and the intent of the UI element used on the mock-ups.

For users relying on assistive technologies, such as screen readers, properly annotated triggers ensure they receive meaningful information about what an action will do before they activate it. This includes specifying whether a link opens a new page in the same tab or a new tab, or if a button triggers a dialog or popover.

Without these annotations, users may experience unexpected changes in context, increased cognitive load, or navigation barriers. By documenting these details, teams can create more predictable, accessible, and user-friendly interfaces.

Annotations for triggers and outcomes significantly improve accessibility by supporting different user needs and preventing confusion. Screen reader users rely on descriptive labels and ARIA attributes to understand what will happen when they activate a button or link, ensuring they are not met with unexpected changes like a popover appearing or a new tab opening.

Similarly, users with cognitive disabilities benefit from clear expectations, as sudden context shifts can be disorienting or overwhelming. When annotations are unclear or missing, usability issues arise—such as a "Submit" button that unexpectedly opens a confirmation dialog, or a link that redirects to a new tab without warning, leaving users disoriented. These pitfalls can lead to frustration, increased cognitive load, and even prevent users from successfully completing tasks.

By proactively defining trigger behavior, designers and developers create more predictable and accessible experiences, reducing barriers for all users.

Examples

1 Recipes Home ▶

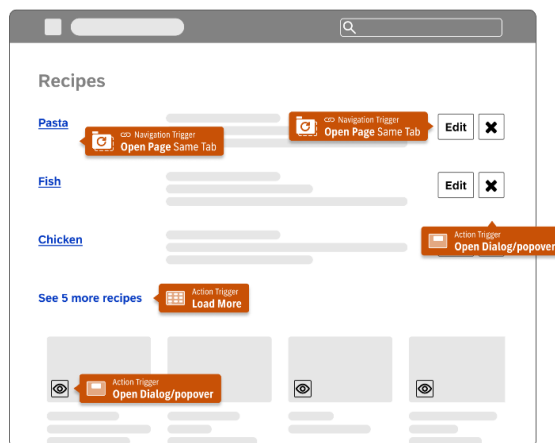


Figure 83: Example for indication of Action and Navigation triggers on an UI using respective annotations

2 Edit Recipe

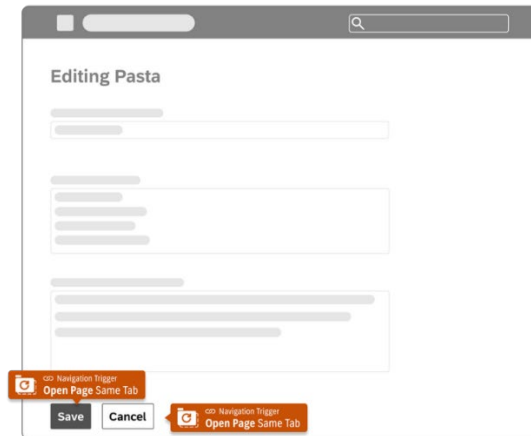


Figure 84: Example for Navigation Trigger Annotations

Shared Benefits



Vision and Color Limitation

Visuals

Clear trigger annotations help ensure that the trigger is visually distinguishable (through contrast, size, or spacing) and placed predictably, which is critical for users who rely on zoom or screen magnifiers to navigate interfaces.



Without Vision

Screen Reading

Trigger annotations support developers in correctly coding programmatic labels, roles, and states, so that screen reader users are informed of what the trigger is, what it will do, and whether it is currently active or disabled.



Limited Cognition

Cognitive

By reinforcing consistency and clear labeling, trigger annotations help users understand what action will occur when interacting with an element. This reduces ambiguity and supports decision-making during complex tasks.

Developer Notes

Consult the API documentation for typical trigger elements such as links and buttons in your framework.

References

WCAG 2.2 Reference

[1.3.5 Identify Input Purpose](#) (AA)

[3.2.2 On Input](#) (A)

[4.1.2 Name, Role, Value](#) (A)

SAP Product Standard Accessibility 4.3

[ACC 257 - Purpose and Target of a Reference](#)



Motion Alternative

If a feature is activated by moving the device (like shaking, tilting, or gesturing), then the same function must be available via standard interface controls, and the motion must be dialable, unless it's essential.

Variants

Motion Alternative



Figure 85: Use to inform where in the Interface there is an alternative trigger to do the same function triggered by the motion. User should not rely only on physical gestures.

About Motion Alternative

Designers (and developers) can meet the WCAG 2.5.4 Motion Actuation (A) requirement by informing that an interaction based on motion has a visual element alternative in the UI.

In plain terms, if a feature is activated by moving the device (like shaking, tilting, or gesturing), then the same function must be available via standard interface controls, and the motion must be dialable, unless it's essential.

The Goal is to ensure that users “can still fully use the interface without relying on physical gestures”. These users represent those who:

- Have limited motor abilities
- Cannot hold or move a device
- Use the interface in a fixed or mounted position
- Might accidentally trigger features via motion

Designers should observe the following:

1. Avoid Motion-Only Interactions. Do not make motion gestures (shake, tilt, etc.) the only way to:
 - Undo actions (e.g., shake to undo)
 - Navigate:
 - Trigger shortcuts or feature

Always pair motion interactions with a standard UI control (e.g., button or menu option).

2. Design Redundant, Discoverable UI Controls. For each motion-based trigger:
 - Include an on-screen button or accessible keyboard/touch equivalent
 - Make sure it's visible, labeled, and reachable
3. Make Motion Features Optional
 - Let users disable motion-based input in the app or via OS settings
 - Detect when motion sensors are unavailable or turned off
4. Consider Platform & Context
 - On iOS and Android, users can limit or turn off motion via system settings (like “Reduce Motion”)
 - Apps should respect these preferences and not require motion

Exceptions

When Motion Can Be Required? Motion can be the only method if:

- It's essential (e.g., augmented reality app requiring device movement)
- It's part of a physical simulation that loses meaning if replaced



Examples

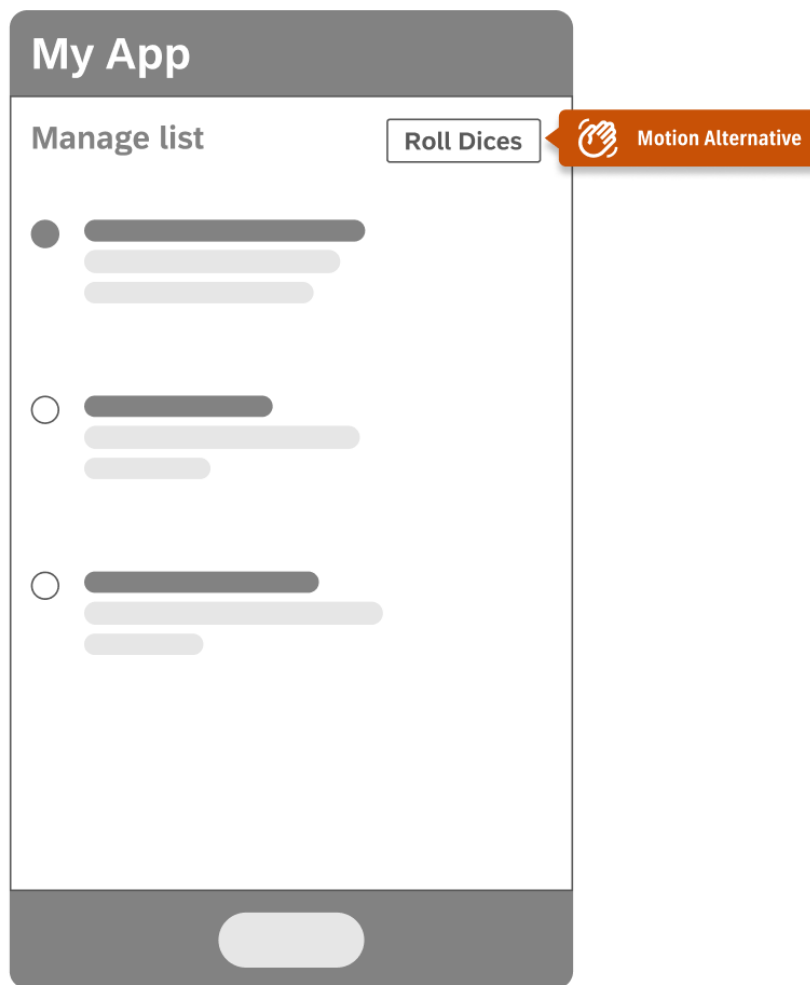


Figure 86: Example App Mockup, that offers the gesture to shake the device to roll dices and a button "Roll Dices"

DO: Visible "Shake the dices" button is always present on screen as an alternative.

DO NOT: Only way to "Shake the Dices" is shaking the device.

Shared Benefits

All users will benefit.

Developer Notes

Provide typical alternatives to motion triggering such as additional links and buttons in your app.

References

WCAG 2.2 Reference

[2.5.4 Motion Actuation](#) (A)

SAP Product Standard Accessibility 4.3

[ACC 270 - Full Input Channel Support \(Keyboard, Mouse, Touch\)](#)



Minimum Target Size

Make target sizes large enough for users to activate them using a pointing device, such as a mouse or stylus, or through touch. Aim for 24 pixels when minimum size is mandatory.

Variants

Minimum Target Size



Figure 87: The annotation is used on components to inform that the target size fulfills the minimum of 24 pixels in both directions (x, y).

About Minimum Target Size

What is the minimum size of a control to be accessible? According to WCAG2.1, it should be at least 44x44 pixels. However, WCAG 2.2 updated the success criteria 2.5.5 Target Size moving from AAA to AA and recommending a new minimum size 24x24 pixels.

A large size of a target helps users to easily activate actions using a mouse pointer, a pen, or touch. The larger a clickable area is, the easier it is for users to reach an interactive element. The target size is considered the region where the pointer (hit area) will reach the actionable element. The hit area of the interactive component can be larger than the component itself, including the surrounding white space i.e., icon button or checkbox. Overlapping hitting areas must be avoided.

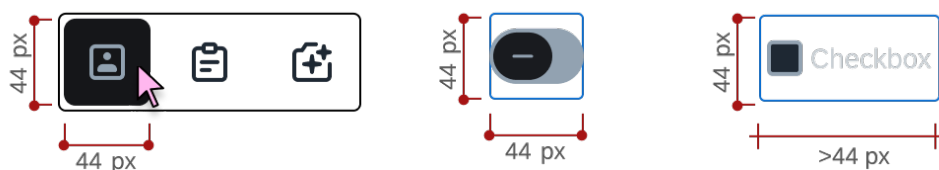


Figure 88: Examples of component size and their hitting areas larger than the minimum 24x24 pixels

However, size can vary depending on the context and some exceptions should be considered during the design. Text links in paragraph or text maintain the same size used for the entire content. Help icons are usually presented on labels or in sentences and are considered part of the content; hence not required to meet the minimum target size. Equivalent targets from multiple targets that perform the same

action on the same page - if one target's minimum size is met, the others are not required.

These are some of the user benefits:

Users with limited dexterity. Users with fine motor movements or hand tremors benefit from a large target as it allows them to interact easily and comfortably when using a mouse or touch device.

Users who use touch devices. Some users may have larger fingers or use their toes, knuckles, or parts of the fingers to access elements on touch devices. Still, others might access touch devices while in a moving vehicle. All will benefit from larger target sizes as it will help them with precision during an interaction.

Users with low vision. Discerning targets from content may require extra effort. Large targets facilitate easy identification of actionable elements.

MOBILE: Native applications may use SDKs with special and different requirements for minimum tappable. iOS is 44x44 points while Android is 48x48 pixels as recommended by their guidelines.

Examples

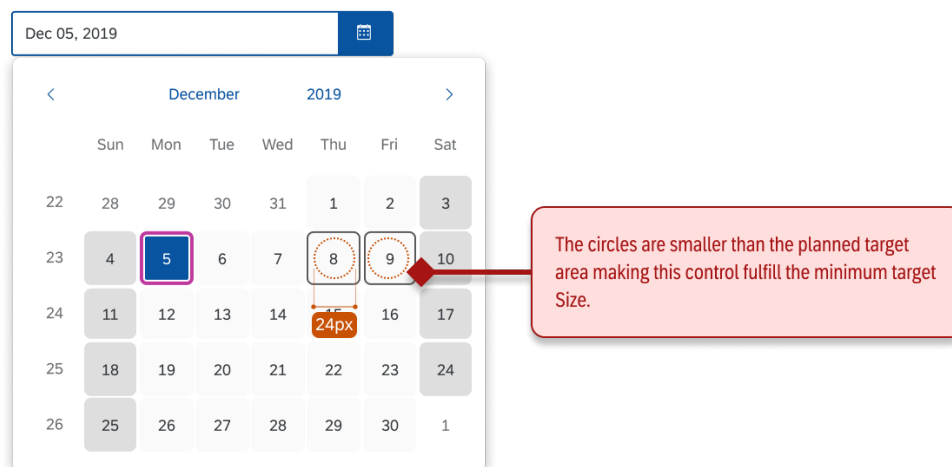


Figure 89: Calendar month view with two minimum target size annotations on day tiles, showing that the minimum target size is met.

DO KEEP CIRCLES TOUCHING OR SEPARATED.

- Adjust the size of the annotation to fit your components placed side to side or stacked vertically.

DO NOT allow the circles to cross. This will create a violation.

Shared Benefits



Temporary Disabilities

Everyone Benefits

Situations like using a device with one hand, wearing gloves, or interacting in motion (e.g., walking) reduce precision. Users with temporary or situational disabilities also benefit from larger targets. They are easier to hit in challenging real-world scenarios, improving usability for everyone.



Limited Cognition

Cognitive

Small or densely packed targets can create confusion or require greater focus and precision, which may increase errors. Users with Cognitive or Learning Disabilities also benefit from larger, well-spaced targets reducing cognitive load and help users make clear choices more confidently.



Vision and Color Limitation

Visuals

Larger targets are easier to locate visually, especially for those using screen magnification tools that reduce visible context. Users with low vision also benefit from this requirement because larger targets reduce effort in aiming or hunting for interactive elements and helps avoid accidental taps or clicks.

Developer Notes

Check if your object dimensions of all your active framework elements are within the allowed range (following the recommendations for minimum size in iOS and Android development kits) and make sure that the distance

between two active framework elements in the layout meets the minimum distance requirements.

References

WCAG 2.2 Reference

[2.5.8 Target Size \(Minimum\)](#) (AA)

SAP Product Standard Accessibility 4.3

[ACC-287 - Target Size](#)



Journey

Plan an exploratory journey and address the steps required to interact and navigate with the component and across pages.

Variants

Journey Flow Step Name - Component



Figure 90: Informs the name of a step in a component flow designed to be applied consistently in various design use cases.

Journey Flow Step Name – Page/Dialog



Figure 91: Informs the name of a step in a business use case flow designed for a user to start, conduct, and conclude a task across pages and overlays.

Journey Direction Flow



Figure 92: Informs the name of a step in a direction flow designed to be applied consistently in various design use cases.

About Journey

Journeys are an excellent way to describe the navigation flow and interactions, both on the control and application level. If an element is interactive, it triggers a flow of actions that should be documented.

Users may use different input mechanisms to interact or navigate inside a control, like tabs, lists, tables, or menus. Exploring them during the design phase contributes to an inclusive experience across devices for all users.

Actions should be intuitively mapped to meet visual requirements for sighted users (i.e., focus and states) and interactive requirements for all user types (i.e., vertical and horizontal navigation on pages or tables and announcements for screen reading users).

Journeys also help explain flows among screens. Before checking all the elements used to design a page, it's essential to understand the journey

and triggers. This contributes to a more accurate design review of accessibility for the screens and their content.

Examples

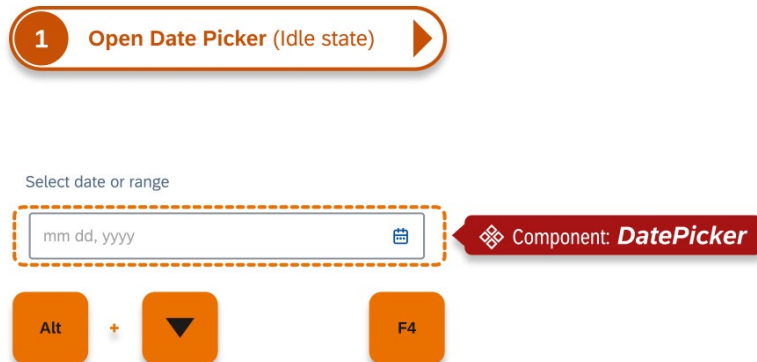


Figure 93: Step Name annotation used to indicate that ALT+ Arrow Down keys and F4 key will open a date picker helper dialog

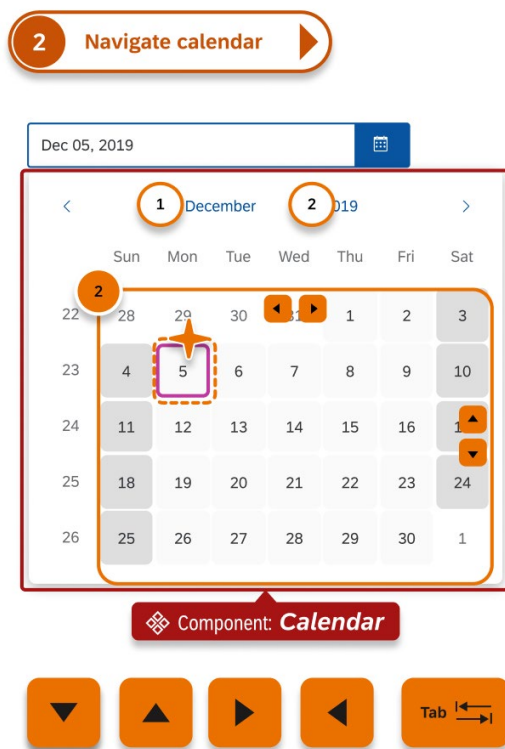


Figure 94: Step Name annotation used to indicate that Tab navigates between functionalities of the calendar. Arrow keys navigate dates within the calendar.

3 Select date range or
Select a single date

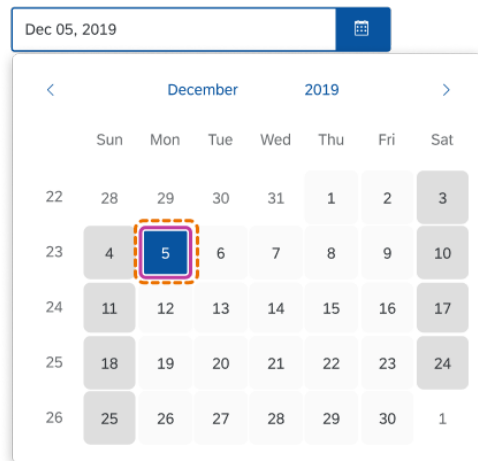


Figure 95: Step Name annotation used to indicate that Shift + Arrow selects a range. Enter selects one element. The initial focus position should be put to “today” by default.

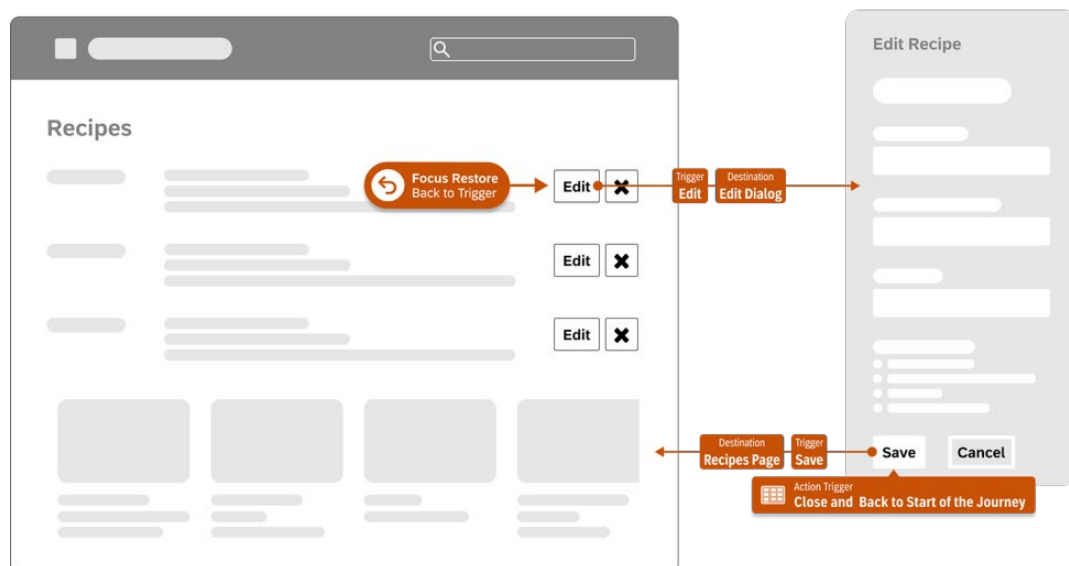


Figure 96: Journey Direction Flow annotation used to indicate invocation and closing of dialog screens

Shared Benefits

All users will benefit from using this annotation.

Developer Notes

Scrutinize workflows: discuss with the designer the trigger elements to go to the individual journey steps.

References

WCAG 2.2 Reference

[1.3.5 Identify Input Purpose](#) (AA)

[2.4.8 Location](#) (AAA)

[2.5.3 Label in Name](#) (A)

[4.1.2 Name, Role, Value](#) (A)

[4.1.3 Status Messages](#) (AA)

SAP Product Standard Accessibility 4.3

[ACC 263 - Information, Structures and Relationships](#)

[ACC 272 - Tab/Reading Order](#)



*ḐḐḐḐḐ ḐḐḐ ḐḐḐḐḐḐḐḐ ḐḐḐḐḐ ḐḐḐḐḐ ḐḐḐḐḐ ḐḐḐ
ḐḐḐḐḐ ḐḐḐḐḐḐḐḐ ḐḐḐḐḐḐḐḐ ḐḐḐ ḐḐḐ
ḐḐḐḐḐ ḐḐ ḐḐḐ ḐḐḐḐḐḐ ḐḐḐ ḐḐḐḐ ḐḐḐḐḐ ḐḐḐḐḐ*

Aisling Noone – User Experience Designer



Screen Reading Experience

Imagine a screen reader is your only way to know what has been designed for a screen. How many words are you prepared to listen to? How much information are you ready to process so that you can understand the content and context and take action?

This chapter helps designers to address requirements for screen reader support to include blind users.

The Screen Reading Experience

Meeting the needs of people who are blind.

Prepare your application and UI components to allow blind users to listen to the screen reading announcements of all relevant content.

Content is any type of information: form fields, title, link, image, table, page title, descriptions, messages, labels and landmarks.

A solid page structure leverages navigation by headings, landmarks and reading order.

Because blind users use screen reading to scan the content structure on a page quickly, the user selects a navigation strategy based on needs (e.g., explore all content, explore titles or groups).

Screen reading navigation varies among assistive technologies, but arrow keys are commonly used to navigate reading order.

Assistive Technologies may support hybrid navigation, including Tab (or focus order) and screen reading capabilities to work in synchronization.

Principles

Blindness



Persona: Bob

"I am legally blind, and I cannot see light or shapes. I use digital products with a screen reader to explore and operate digital products. My preferred interactive tools are the keyboard & touch in mobile devices."

This user group represent individuals who are legally blind and others with significant vision impairments. They rely on screen readers and assistive technologies that convert on-screen content into speech or braille readers and prefer to interact via keyboard and touch interactions to navigate and operate the digital product. A highly sequential, linear, and well-structured content is key to reach usability and comply with accessibility. These users receive clues from the system by listening announcements made by the screen reading. As a result, these users listen a large number of words per day to interact with digital product which could stressful.

Some users experience vision loss that worsens over time. The progressive or temporary vision loss may result from glaucoma, diabetes, or retinopathy. There are two types of blindness:

- **Total blindness:** These users perceive no visual input and rely entirely on-screen readers, braille, or audio/tactile feedback to navigate digital products.
- **Legal Blindness:** Often defined as 20/200 vision or worse in the better eye (with correction) or a visual field of 20 degrees or less, these users can detect shapes and brightness but struggle with details and often rely on screen magnifiers or screen readers.

Statistics:

- Global: 2.2 billion people have a near or distance vision impairment, where 36 million people are blind, and 217 million have moderate to severe vision impairment. WHO World Report on Vision, 2019

Situational & Temporary Disabilities

By integrating situational and temporary disabilities into this disability group, we help design teams address challenges that might not be permanent but still impact user experience under certain conditions.

Situational and temporary:

- Dark or bright environments: Difficulty seeing due to temporary lighting conditions, such as extreme glare or darkness.
- Temporary blindness: Due to events like a sudden injury or temporary eye condition (e.g., eye infection or trauma).

General Design Tips

1. **Accurate and Complete Announcements:**

Screen readers depend on accurate labels and programmatic associations to convey content. Missing, unclear, or duplicate labels can leave users confused or unaware of available options.

Design Tip: Ensure every interactive or informative element (e.g., buttons, links, form fields, icons, images) has a clear, unique accessible name using appropriate attributes like `aria-label`, `aria-labelledby`, `alt`, or semantic HTML elements (e.g., `<label>`).

2. **Clear Page Structure:**

Without a well-defined heading hierarchy or landmarks, screen reader users may struggle to understand the layout or jump to relevant sections.

Design Tip: Use semantic headings (`<h1>`–`<h6>`) to establish a clear content structure and add ARIA or native landmarks (`<main>`, `<nav>`, `<header>`, `<footer>`) to support orientation and quick navigation.

3. **Logical Reading Order:**

If content is visually arranged in one way but coded in another, the screen reader may read it out of sequence, leading to confusion.

Design Tip: Ensure the DOM (document order) reflects the visual and logical order of content. Avoid using CSS tricks that misalign reading and focus order.

4. **Hybrid Navigation (Focus + Reading Order):**

Challenge: Blind users often combine keyboard (Tab/Shift+Tab) with arrow-key reading. Inconsistent focus management or skipped elements can break this experience.

Design Tip: Maintain a predictable focus order and supplement it with reading-friendly design. Announce state changes, use ARIA roles (role="alert", aria-live) to surface updates without requiring focus shifts.

5. **Unique and Contextual Labels for Repeating Elements:**

Repeating components (e.g., icon buttons, links, or lists) with identical labels confuse screen reader users.

Design Tip: Provide context to each instance—e.g., "Edit profile" vs. "Edit address"—using visible or aria-labeledby text tied to neighboring content.

6. **Supplementary Descriptions for Complex Elements:**

Visual-only cues like color or icons aren't announced by screen readers unless explicitly labeled.

Design Tip: Use aria-describedby or to associate additional info with elements like buttons or charts, ensuring screen reader users receive the same guidance as sighted users.

Annotations

Screen Reading Annotations

The checklist helps designers to remember the important aspects of accessibility that should be addressed in the design phase.



Screen Reading Experience

- ☐ Reading Order
- ☐ Label
- ☐ Description
- ☐ Live Message
- ☐ Heading
- ☐ Landmark
- ☐ Page Title

Reading Order

Inform and number the sequence of UI elements on the layout, including hidden elements when applicable, that must be logically perceived during screen reading announcements.

Variants

Reading Order



Figure 97: Indicate order of UI elements to be announced by the screen reader in reading navigation by this annotation with numbers that reflect the reading order.

Reading Order (with add. inner order)



Figure 98: UI elements that require further inner navigation (like toolbars, breadcrumbs and tables) should be annotated with the Reading Order with additional inner order variant

Reading Scope



Figure 99: When appropriate, enclose multiple UI elements to be read together by the screen reader in reading navigation (reduces reading navigation effort)

Reading Flow



Figure 100: Indicates a general reading navigation flow direction when individual annotation may not be required

About Reading Order

Every UI element visually available on the screen should be announced during screen reading navigation. The reading order is the sequence of UI elements announced.

On desktops, this will be done in a special reading mode of assistive technologies (called “virtual mode”, “browse mode” etc.). The user will use an assistive tool to navigate the UI elements using arrow keys up and down. On mobile, the reading order is known as swipe order and navigation is done using swipe gestures.

Identify the logical Reading Order of elements including texts, images and interactive controls. A logical order of elements enhances the understanding of the content and its context. Use the dashed rectangular area to identify a group of content that should be announced all at once by the screen reading. This helps to form logical contexts (what belongs together is spoken together) and reduces reading navigation effort and forms.

The Reading Scope annotation should be used to represent UI elements that require further inner navigation (like toolbars, breadcrumbs and tables) where the total number of steps inside is unknown or not important in the current context.

The two variants of Reading Flow represent the default navigation directions. Use these when annotating individual annotation sequences is not required in your design case.

SAP Figma Plugin

The [SAP Figma Plugin “Reading Order”](#) helps designers annotate their screens identifying relevant focus order sequence.

Examples

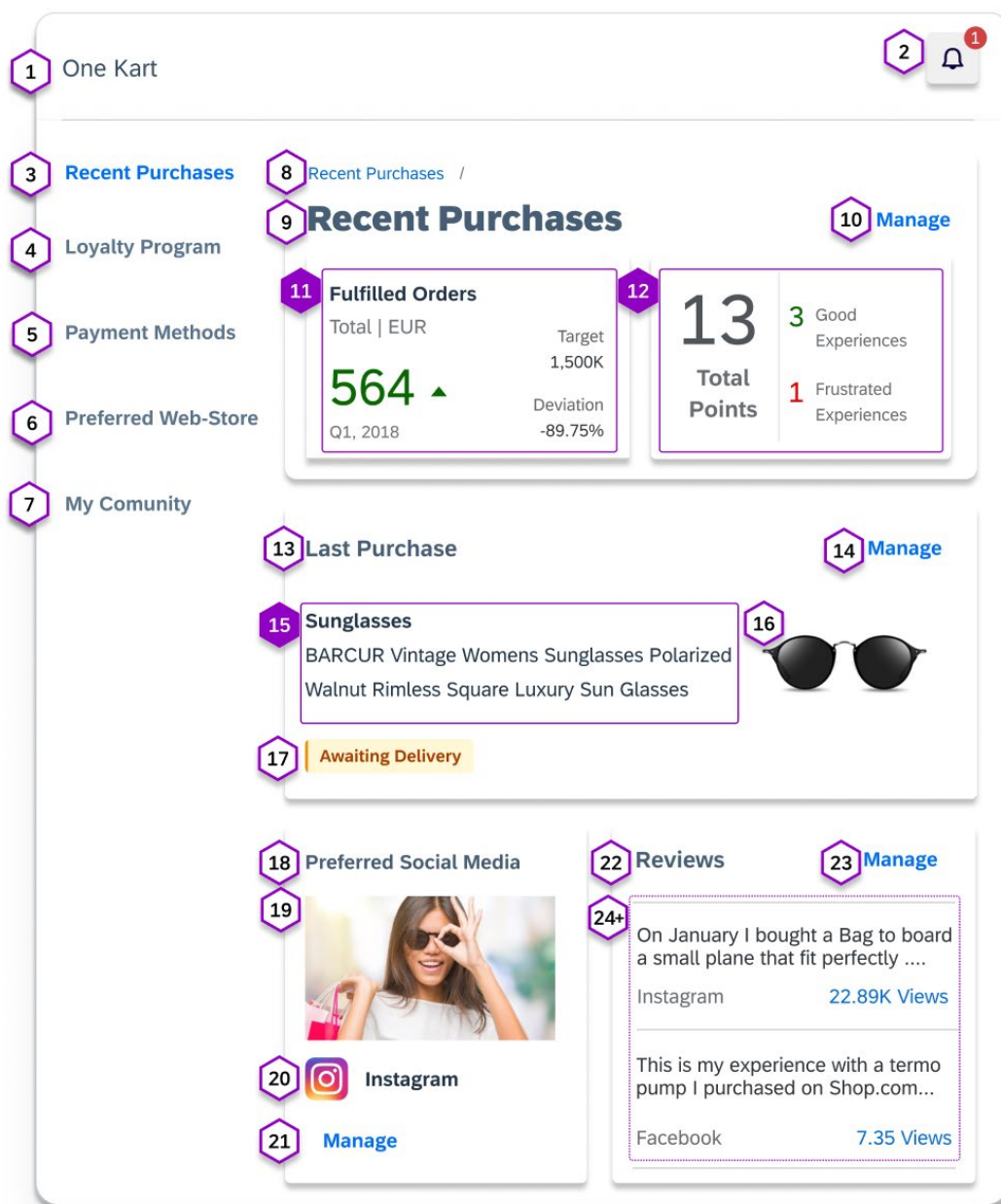


Figure 101: Reading sequence using reader keys or swipe gestures should follow a logical order and should respect the logical nesting of grouping containers in UI. Text belonging to a common context should be read at once (11, 12, 15). Components with predefined inner reading orders are indicated with the Reading Order + annotation (24).

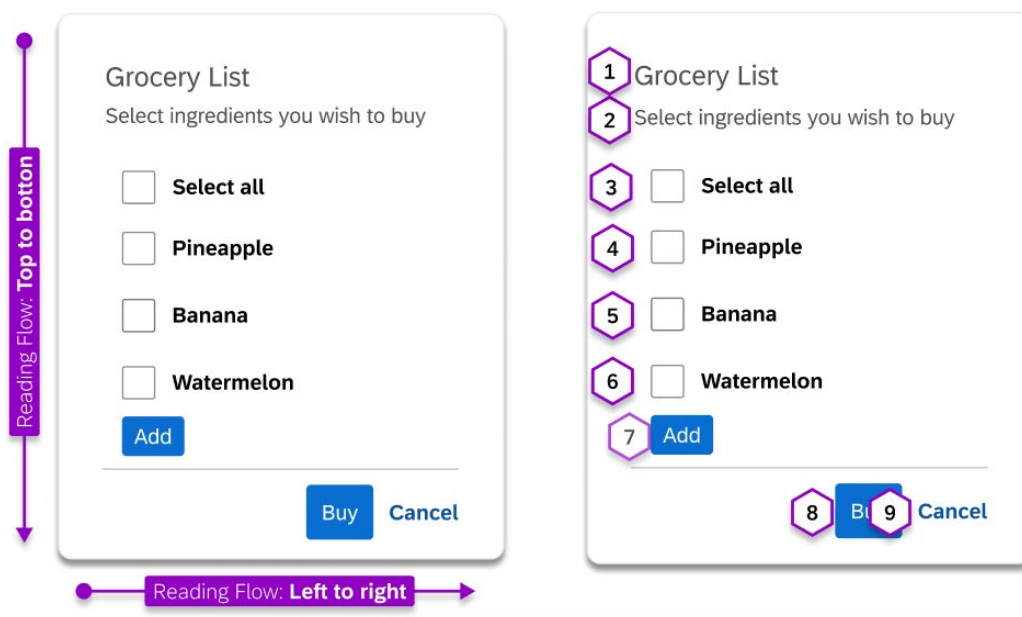


Figure 102: Default Reading order flow indication for a simple dialog. In this case, using Reading Flow annotation is like annotating individual Reading Order steps and saves a lot of work, therefore preferred and recommended instead.

Shared Benefits



Temporary Disabilities

Everyone Benefits

A clearly structuralized and well-perceivable easy to read UI is of benefit for all users. Audio feedback is generally helpful in all situations when users are not looking at the user interface since they are busy with other tasks.

Developer Notes

Reading order (during arrowing with screen readers in HTML or swipe next operations on mobile devices) is established in all technologies by layout and content node element order and nesting.

Always make sure that elements that form a common context are adjacent in layout and not distributed between different containers, except if presented in an extra control context for this purpose (like row/columns in tables).

Examples:

Web Development:

By combining text that belongs together in one larger DOM Node

```
<p>
<span>Text 1</span>
<span>Text 2</span>
<span>Text 3</span>
</p>
```

Note: A swipe or arrowing in reading mode to the div will read multiple items at once

```
<div role="group" aria-label="Text 1 Text 2 Text 3">
<span>Text 1</span>
<span>Text 2</span>
<span>Text 3</span>
</div>
```

Fallback: Concatenating all inner info in an aria-label of the container

Mobile Framework Developer:

iOS

Reading Order depends on view construction

```
VStack(alignment: .leading) {
  Text(landmark.description)
  .font(.subheadline)
  HStack(alignment: .top) {
    HStack(alignment: .top) {
      Button(action: {}) {
        Text("Route")
      }
      .accessibility(label: Text("Plan a route"))
    }
    Spacer(minLength: 10)
    Button(action: {}) {
      Text("Website")
    }
    .accessibility(label: Text("Visit website"))
  }
}
.accessibilityElement(children: .combine)
```

Accessibility for native iOS Apps

https://video.sap.com/media/t/1_pbxervj7

(see from minute 15)

Android

Group content to be read at once in ConstrainedLayout or LinearLayout

```
<ConstraintLayout
  android:id="@+id/song_data_container" ...
  android:screenReaderFocusable="true">
  <TextView
    android:id="@+id/song_title" ...
    android:focusable="false"
    android:text="@string/my_song_title" />
  <TextView
    android:id="@+id/song_artist"
    android:focusable="false"
    android:text="@string/my_songwriter" />[SEP] ...
</ConstraintLayout>
```

Use AccessibilityTraversal methods to change default left to right swipe order

```
volupButton.setAccessibilityTraversalAfter(myView.findViewById(R.id.remote0).getId());  
voldownButton.setAccessibilityTraversalAfter(myView.findViewById(R.id.volup).getId());  
chaineplusButton.setAccessibilityTraversalAfter(myView.findViewById(R.id.voldown).getId());  
chainemoinsButton.setAccessibilityTraversalAfter(myView.findViewById(R.id.chaineplus).getId());
```

References

WCAG 2.2 Reference

[1.3.1 Info and Relationships](#) (A)

[1.3.2 Meaningful Sequence](#) (A)

SAP Product Standard Accessibility 4.3

[ACC 263 - Information, Structures and Relationships](#)

[ACC 272 - Tab/Reading Order](#)



Label

Provide labels, visible or invisible, to identify and describe the purpose of UI elements for all users and screen readers.

Variants

Visible Label (Relationship)



Figure 103: Use this variant to reinforce the presence of a visible label that is tied to another UI element, like input field and label

Invisible Label



Figure 104: When a visual text cannot be used in the design, the Invisible Label variant is used to define an element purpose and context to blind users

About Label

A label is the text or visual indicator that names or describes the purpose of a user interface element, such as a form field, button, or checkbox. It helps all users, including those using assistive technologies, understand what the element does or what information is expected.

Labels can be visible or invisible. Visible labels are easily recognized in forms as they are positioned typically above or adjacent to its interactive component. If you have a choice, use a visible label. If this is not possible, use an invisible label associated with a control to ensure that screen readers will announce the element's purpose. An invisible label is used in minimalist designs to reduce visual clutter—such as when icon-only buttons are shown to sighted users—but it is essential to include an invisible label so that screen reader users (who are blind or have low vision) can perceive the purpose of the control. It ensures that the functionality remains understandable and operable for all users, even when visual text is hidden.

For sighted users, visible labels should be straightforward and aligned with common expectations, while for blind users, invisible labels should

supplement visual context without redundancy. This approach ensures an inclusive experience, enabling all users to navigate interfaces efficiently and confidently.

Use cases where invisible labels are accepted are:

- Combined input fields like Address (number and street)
- List of checkboxes
- List radio buttons
- Icon button
- Semantic buttons

The content used in an invisible label should also be provided as a tooltip to ensure it's accessible to sighted users as well—so that essential context is not exclusive to blind users but available to everyone.

Good labels improve usability, accessibility, and reduce user errors. In design, always ensure labels are:

- Visible and placed close to the element they describe (e.g., above or beside a text field).
- Consistent in wording and placement.
- Programmatically associated with the element for screen readers (via label for in HTML or aria-label when appropriate).

Do not use placeholders as labels. Once the user enters a value, the “label-placeholder” will be gone leaving users to wonder what the purpose of the information was requested in the input field.

Examples

Single Reference Annotations

Form Title

Group Header

Label For: **Textbox** Name: * John Miller

Postal Code / City: 12345

Street / No.: Mainstreet 1618

Group Header

Label For: **Textbox** Purchase Order: Valid from Date

Valid From Date/Time: 01/01/1970 00:00

Figure 105: Different fields of a combined input field are presented with one unique visible label. Invisible labels help to clarify the purpose of the single fields.

Scope Annotations

Label For: **CheckboxGroup**

Banana cake

- ☐ Banana
- ☐ Flour
- ☐ Eggs
- ☐ Chocolate
- ☐ Baking powder

Banana cake

- ☐ Banana
- ☐ Flour
- ☐ Eggs
- ☐ Chocolate
- ☐ Baking powder

Invisible Label **Banana Cake**

Figure 106: Group of checkboxes identified by visible label relation or invisible label scope annotations assigning in both cases group header text

Label
For: Table

Core Competency

Competency Name	Weight	Target	Weight	Rating	Gap	Comment
Time management	20%	3.0	30%	Unrated	0	>
Time management	10%	3.0	50%	Unrated	0	>

Figure 107: Visible table titles must be associated as label with corresponding table

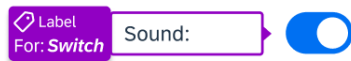
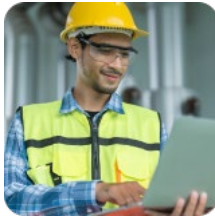


Figure 108: Label-control association for a custom Android switch component

Shared Benefits



Temporary Disabilities

Everyone Benefits

Sighted users must be supported with tooltips to understand the purpose of icons or buttons that lack visible text. Tooltips provide additional context on hover or focus, helping users avoid confusion, especially in complex interfaces or when icons alone are ambiguous.



Limited Mobility

Interactions

Keyboard users must be supported with tooltips that appear on focus—not just hover—so they can access the same contextual information as mouse users and blind users using the screen reader. This ensures equal understanding of icon-only controls or complex actions without needing a mouse.



Limited Cognition

Cognitive

Neurodivergent users may rely on tooltips to reduce cognitive load by providing clear, concise explanations of unfamiliar icons or functions. Tooltips support memory, comprehension, and decision-making—especially when the interface is complex or uses abstract visuals.

Developer Notes

Labels support the Info and Relationships principle by clearly linking UI controls to their purpose. They serve as an accessible name or description—whether visible or hidden (e.g., using aria-label)—ensuring that form controls and inputs are understandable to all users.

Ensure an invisible label is informed where the design uses a limited number of words to provide context.

When crafting labels, UX writers should prioritize clarity, ensuring that labels are immediately understandable without requiring users to interpret vague or ambiguous wording. Avoid abbreviations unless they are widely recognized, as unfamiliar shorthand can create confusion and increase cognitive load. Every label should be concise yet descriptive enough to provide clear meaning, reducing the mental effort needed to understand its purpose. Tips for UX writers to deliver a clean screen reading speech output:

Use plain text on labels.

Do not use a colon (":") as this is a decorative style - If needed, use it separated from the label.

Use caps only on the first letter of the first word to secure a smooth tone of voice.

Another place where contextualization is required is on repeating Landmarks. The Landmark Region is often repeated in a page, which requires a contextualization to help blind users distinguish them.

Accessible Name

Accessible Name is the Primary Identifier used by assistive technologies to identify an element. It is given to UI elements as a supplement to create

meaningful labels associated to meaningful visual cues where needed (like icons or section areas). Examples of Accessible Names are “aria-label” for custom names when there’s no good visible label, “aria-labelledby” to reference repeating instances with an existing text to make them unique in the UI - instead of duplicating content.

Key Differences Between ‘aria-label’ and ‘aria-labelledby’

Feature	aria-label	aria-labelledby
When to use	When there’s no visible text to describe the element	When you can reuse an existing visible element as the label
How it works	Provides a custom, standalone name	References another element’s text
Example usage	<code><nav aria-label="Main Navigation"></code>	<code><nav aria-labelledby="menu-heading"></code>
Downside	Can create inconsistencies if not aligned with visible UI	Requires maintaining correct ID references

UI technologies have different labeling techniques but there are only three basic principles to denote the relationship between a label and a control:

1. Inclusion: Label and control form a common context (mostly represented by control having a label text attribute or markup that encloses label and control together)
2. Forward labelling: Separate label control has an id pointer to id of a different control (classic “label for=” approach in HTML)
3. Backward labelling: Control has an id pointer to id of a control used as a label (WAI-ARIA approach)

To apply correct labelling techniques for a number of base technologies, read the respective info in the links given below:

Web Development

HTML:

[H44: Using label elements to associate text labels with form controls | WAI | W3C](#)

WAI-ARIA:

[ARIA16: Using aria-labelledby to provide a name for user interface controls | WAI | W3C](#)

Mobile Development

iOS:

[accessibilityLabel | Apple Developer Documentation](#)

Android:

[Principles for improving app accessibility | App quality | Android Developers](#)

If you are using an UI framework that possibly encapsulates one or more of the methods above, look for respective framework API property documentation instead.

References

WCAG 2.2 Reference

[1.1.1 Non-Text Content](#) (A)

[1.3.1 Info and Relationships](#) (A)

[1.3.5 Identify Input Purpose](#) (AA)

[2.4.6 Headings and Labels](#) (AA)

[2.5.3 Label in Name](#) (A)

[3.3.2 Labels or Instructions](#) (A)

SAP Product Standard Accessibility 4.3

[ACC 253 - Labels for Input and Output Fields](#)

[ACC 255 - Text Alternative for Non-Text Content](#)

[ACC 258 - Consistent Use of Identical UI-Elements](#)

[ACC 259 - Accessible Alternatives](#)

[ACC 263 - Information, Structures and Relationships](#)

Description

Describe images, add invisible descriptions for screen reader users and relate visible descriptions to other UI elements.

Variants

Visible Description (Relation)

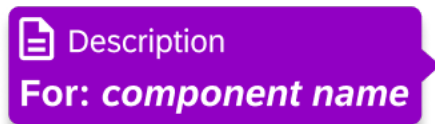


Figure 109: Use this variant to describe the invisible description and its relationship to the element.

Invisible Description



Figure 110: Use this variant to describe the invisible description for an element

Decorative Element



Figure 111: Use this variant on visual UI elements to indicate that no screen reader output should be generated for those

About Description

Use Description to explain images. These descriptions are very popular and widely used to fulfill accessibility for a long time. They are known as Alternative Text (ALT Text).

Most of the time the description will be an Invisible Description to be announced by the screen reading.

But a description can be also a Visual Description which makes all users have access to same content avoiding privileged information shared only with blind users.

Besides images, visible descriptions can be also used as complementary information next to form fields. This description appears as a static text part

of the interactive UI control. For example: Label <Quantity>, Value <3>, Description <kg>.



Decorative images do need an explicit empty description and are to be hidden from screen readers.



When photos, illustrations or similar non-text content has an important part in the context, it must be described to allow users of screen readers to understand the whole information.



Image of text must be described containing the visible text to allow users of screen readers to have equal access to information seen by sighted users.

Examples

The screenshot shows a 'My Profile' form with the following elements and annotations:

- Name ***: A text input field with the placeholder 'Enter name'.
- Height**: A text input field with a unit selector 'CM' and a description box: 'Description For: TextBox'.
- Weight**: A text input field with a unit selector 'KG' and a description box: 'Description For: TextBox'.
- Address**: A text input field with the placeholder 'Enter name'.
- City**: A text input field.
- Zip Code**: A text input field.
- Coaching Time**: A text input field with a unit selector '€ / Hour' and a description box: 'Description For: TextBox'.
- Preferred Attractions**: A row of five image thumbnails. The third thumbnail (Eiffel Tower) has an 'Invisible Description' box: 'Eiffel Tower seen from the bridge framed by fall trees with yellow leaves'.
- Buttons**: 'Save' and 'Cancel' buttons at the bottom right.

Figure 112: Description relations and image descriptions

Custom object status descriptions can be added as invisible description.



Figure 113: Status



Figure 114: Assembly Option

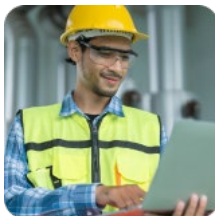


Figure 115: Decorative images must not have any descriptions and are to be hidden from screen readers by technical means.



Figure 116: Image of text must be described including the visible text

Shared Benefits



Temporary Disabilities

Everyone Benefits

Clear descriptions help all users to understand current contexts in user interfaces. Audio feedback is generally helpful in all situations when users are not looking at the user interface since they are busy with other tasks.

Developer Notes

UI technologies have different techniques for assigning descriptions but there are only two basic principles to denote the relationship between a description and a control:

1. Inclusion: Description and control form a common context (mostly represented by control having a description text attribute or markup that encloses description and control together)
2. Backward describing: Control has an id pointer to id of a control used as a description (WAI-ARIA approach)

If you are using an UI framework that possibly encapsulates one or more of the methods below, look also for respective framework API property documentation.

Web Development

Visible/Invisible descriptions:

By using the “aria-describedby” reference attribute (“backward” describing)

```
<label for="i1">Weight</label>[L][SEP]<input id="i1" aria-  
describedby="d1"/>  
<span id="d1">kg</span>
```

Invisible descriptions: By using the HTML alt attribute OR the “aria-description” attribute OR by additional “hidden” DOM text (“direct” describing)

```
[L][SEP]<span  
style="background-image: eiffel.png"  
aria-description="Eiffel tower in autumn on a sunny day">[L][SEP]  
</div>  
<span>Assembly Option</span>  
<span class="urgent">To be selected [L][SEP] <span  
class="hiddentext">Negative Status</span>[L][SEP] </span>  
</div>
```

Mobile Development

iOS:

Descriptions via accessibility hint

```

VStack(alignment: .leading) {
  Text(landmark.description)
  .font(.subheadline)
  HStack(alignment: .top) {
    HStack(alignment: .top) {
      Button(action: {}) {
        Text("Route")
      }
      .accessibility(hint: Text( "Navigates to next screen to plan
a tracking route"))
      Spacer(minLength: 10)
      Button(action: {}) {
        Text("Website")
      }
      .accessibility(hint: Text("Open the website of the park's
visitor information"))
    }
  }
  .accessibilityElement(children: .combine)

```

Android: Describe images

```

<ImageView
...
android:src="@drawable/my_image"
android:contentDescription="@string/my_image_description"
/>

```

Give additional object information

```

<TextView
...
android:contentDescription="To be selected - Negative Value"
/>

```

Identify decorative images

```

<ImageView
...
android:importantForAccessibility="no"[1]SEP
android:contentDescription="null"
/>

```

Describe a complex context

```
containerView.setContentDescription("3 mails not read,  
button");  
containerView.setImportantForAccessibility(View.IMPORTANT_FO  
R_ACCESSIBILITY_YES);  
mailImageView.setImportantForAccessibility(View.IMPORTANT_FO  
R_ACCESSIBILITY_NO);  
infobulleTextView.setImportantForAccessibility(View.IMPORTAN  
T_FOR_ACCESSIBILITY_NO;
```

References

WCAG 2.2 Reference

[1.1.1 Non-Text Content](#) (A)

[1.4.5 Images of Text](#) (AA)

SAP Product Standard Accessibility 4.3

[ACC 255 - Text Alternative for Non-Text Content](#)

Live Message

Communicate with messages while user maintaining focus on the where it is

Variants

Visible Message



Figure 117: Use this variant to inform that this section on the UI represents a visible live updated message

Invisible Live Message

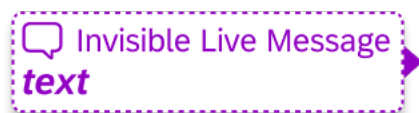


Figure 118: Use this annotation to inform an invisible message and its relationship.

About Live Message

Visible messages are also perceived by non-blind people and contribute to a usable experience. Messages are triggered and presented after user interaction or system updates. They are usually visible but often fade away in a few seconds.

An essential aspect of toast messages is that, whenever it appears, the focus remains where it was before the message was triggered. Dialog messages will change focus.

Screen readers should be prepared to announce visible messages (like in message areas of business applications) while keeping the focus on the current UI element; Focus does not move to the message. This is important, especially when a message disappears in a few seconds.

Invisible Messages happen on a few occasions. Some examples are messages that should announce search results, filter results, feed list additions or auto-save. These use cases may not provide one specific visual feedback. Instead, the state of existing UI elements should react and be announced in a way that is easily understood by both: blind and visually abled users.

Remember:

- When messages pop up, focus does not change. This applies to toast messages. Focus remains on the element the user was working on before the message appears. This rule does not apply to dialog messages.
- Message that fades away in a few seconds is not recommended. Neurodivergent users need extra time to read the message. It is best to give control to the user to dismiss the message when the user ends the reading.

Alert sounds combined with invisible (non-visible) messages present a barrier for blind users. WCAG addresses this indirectly. Blind users using screen readers expect that an alert sound (like a beep) happens without a visible or programmatically exposed message, a screen reader user won't be aware of it. The SC 4.1.3 requires that status updates, like "message sent," "connection lost," or "new message received," be exposed to assistive tech (e.g., using aria-live).

ARIA live regions are critical for accessibility, but can be overlooked during the design phase, can cause confusion or loss of important updates for screen reader users. Application and component designers should specify ARIA Live Regions for these three reasons:

1. **Screen readers don't "see" visual changes.** If a success message, error, or dynamic update appears visually (e.g., a toast, notification bar, or inline message), it won't be announced unless it's placed inside a properly configured ARIA live region.
2. **Developers can't guess your intent.** If your design doesn't explicitly indicate which dynamic updates are important for screen reader users, they may not implement aria-live, or may use the wrong priority (e.g., polite vs assertive).
3. **It ensures parity in user experience.** By planning for ARIA live regions in design, you ensure that blind and screen reader users receive timely feedback about system responses, errors, confirmations, or live updates—just like sighted users do.

Interruptions such as system alerts, popups, notifications, banners, or auto-refreshing content should be avoidable or user-controlled, except when urgent. This requirement is aligned with WCAG 2.2.4 Interruptions (AAA) which ensures users can stay focused, not lose progress, and control or postpone unexpected interruptions. This is also important for users with cognitive disabilities, attention-related challenges, or limited motor control,

who may find it difficult to return to their task or regain focus after an unplanned disruption.

Interruptions can break the flow of interaction, causing users to lose their place or input. To prevent this, designers should use non-intrusive alerts that don't steal focus and provide settings to postpone or manage notifications. This will help users stay oriented and maintain control without unnecessary distraction.

Examples

1. Invisible Message raised as a result of a search
2. Message "Draft saved" added to a toolbar as a result of a save operation.

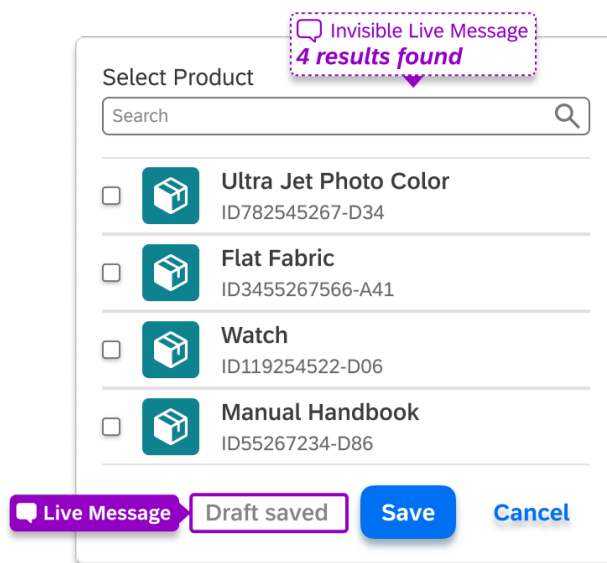


Figure 119:

1. Invisible Message raised as a result of a search,
2. Message "Draft saved" added to a toolbar as a result of a save operation.

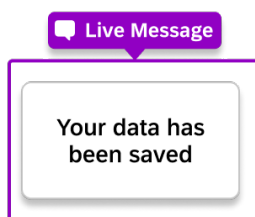
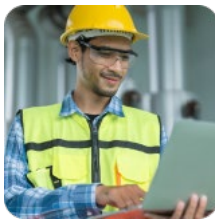


Figure 120: Message raised by showing a message toast temporarily.



Figure 121: Sometimes an invisible message has to be raised differently from a visual message. It has to include extended information, such as a description of the meaning of visual cues visual (error sign) and other options offered.

Shared Benefits



Temporary Disabilities

Everyone Benefits

Audio messages are generally helpful in all situations when users are not looking at the user interface since they are busy with other tasks.

Developer Notes

If you are using an UI framework that possibly encapsulates one or more of the methods below, look also for respective framework API property documentation.

Web Development

By declaring live regions (self-announcing on DOM update)

```
<div role="group" aria-live="assertive">
<span>Error: Cannot perform save. Reason: Object locked by
other user</span>
</div>
```

Mobile Development

iOS

- Notifying about the exceptional case
- Notify user about errors and allow user to move focus to the point where the notification's context is described.
- Create a notification area (e.g. a banner) which contains/displays the context
- Create a notification and notify user

- Allow user to set focus on notification area and perceive the context information.

```
.onChange(of: notification) { notification in
    if notification?.priority == .high {
        isNotificationFocused = true
    } else {
        postAccessibilityNotification() // notification.text
    }
}
```

SwiftUI Accessibility: Beyond the basics

<https://developer.apple.com/videos/play/wwdc2021/10119/>

(see from minute 24)

Android

Announce error messages

```
<TextView
[...]
android:accessibilityLiveRegion="polite" />
```

Give instructions

```
if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.JELLY_BEAN) {
    myView.announceForAccessibility(R.string.SearchResultInfo);
}
```

References

WCAG 2.2 Reference

[1.3.5 Identify Input Purpose](#) (AA)

[2.5.3 Label in Name](#) (A)

[2.2.4 Interruptions](#) (AAA)

[4.1.2 Name, Role, Value](#) (A)

[4.1.3 Status Messages](#) (AA)

SAP Product Standard Accessibility 4.3

[ACC-255 - Text Alternative for Non-Text Content](#)

Heading

Prepare a meaningful headings structure to enable all users and screen readers to find relevant blocks of content quickly.

Variants

Visible Heading



Figure 122: Use this variant to identify a visible heading pointing to text that appears on the screen

Invisible Heading



Figure 123: Use this variant to identify an invisible heading and informing the content of the hidden heading

About Headings

The Headings navigation strategy is used with screen readers to skim the page as they communicate the organization of the content.

Visible headings are directly tied to best practices on usability. The layout and headings must respect the page's logical and consistent structure to provide predictability to the application exploration.

Invisible headings replace a page's visual structure in the page that does not present a visible heading in the layout. Inform the invisible heading using the annotation pointing to where it should be announced during Reading Order navigation (see respective annotation for details).

HINT: Exclusive Headings navigation is possible on Jaws and NVDA using the "H" key.

Reserve H1 for page titles to ensure the visible headings hierarchy matches the code <h1>, <h2> and so on. Headings with a lower rank form new subsections that are part of the higher ranked section. Multiple H1s are allowed, provided they are NOT nested within each other.

SAP Figma Plugin

The [SAP Figma Plugin “Headings”](#) helps designers annotate their screens identifying relevant focus order sequence.

Examples



Figure 124: Complex Main Page with different heading levels. Additionally, invisible headings have been defined for enhanced navigation within page structure.

Shared Benefits



Temporary Disabilities

Everyone Benefits

A concise and clear heading structure helps all users to orient in any user interface. Audio feedback is generally helpful in all situations when users are not looking at the user interface since they are busy with other tasks.

Developer Notes

If you are using an UI framework that possibly encapsulates one or more of the methods below, look for respective framework API property documentation instead.

Web Development

By adding heading role with level info to HTML element

```
<div role="heading" aria-level="2">
My Heading
</div>
```

By using respective HTML element directly

```
<h2>My Heading </h2>
```

Hidden headings that can be read by screen readers can be created using respective css on the heading like so:

```
.pseudoInvisibleText {
  position:absolute!important;
  clip:rect(1px,1px,1px,1px);
}
```

Don't use display:none since this will remove the heading from the DOM and it won't be detected anymore by screen readers.

Moblle Framework Development

iOS

Declare headings as .isHeader trait

Screen reader 'voice over' allows Rotor-based navigation to headings.

```
VStack(alignment: .leading) {
  Text(currentTile.name)
  .font(.title)
  .accessibility(addTraits: .isHeader)

  VStack(alignment: .left) {
    Text(listItem.label)
    .font(.subheadline)
    Spacer()
    Text(listitem.value)
    .font(.subheadline)
  }
}
```

Android

Use accessibilityHeading on TextViews

```
<TextView
[...]
android:id="@+id/myTextLevel1viewId"
android:accessibilityHeading="true"
/>
```

[Android TalkBack Gestures](#)

[Samsung Voice Assistant Gestures](#)

References

WCAG 2.2 Reference

[1.3.1 Info and Relationships](#) (A)

[1.3.2 Meaningful Sequence](#) (A)

[2.4.2 Page Title](#) (A)

[2.4.6 Headings and Labels](#) (AA)

[2.4.10 Section Headings](#) (AAA)

SAP Product Standard Accessibility 4.3

[ACC-263 - Information, Structures and Relationships](#)

[ACC-264 - Interoperability with Assistive Technologies](#)

Landmark

Define semantic groupings in your layout and assign appropriate labelled landmarks around them to enhance screen reading navigation.

Variants

Landmark



Figure 125: Use this annotation to define and label semantic groupings in your layout

About Landmark

Currently, landmarks do not exist on mobile devices. Navigation by landmarks is used by users of assistive technology as an alternative to “skip to main content.” They are used to speed navigation, similar to skipping groups. Use them to indicate a region or a group within a page to enhance screen reading navigation strategies.

Sighted users find it helpful to understand the structure of a page by identifying groups of information visually laid out. Blind users understand the page’s structure using landmarks embedded in screen reader tools.

Landmarks are only available for screen readers and are NOT a substitute to skip blocks (F6). Landmark navigation on JAWS uses the “R” Key. In NVDA, landmark navigation uses the “D” Key; the user uses shift + Key to navigate backward. Both technologies (or tools) offer an interface listing the landmarks, something not available on skip groups.

Although landmarks may overlap with skipping groups, the keyboard navigation strategies differ. The same general structure of landmark blocks (L & R keys in Jaws and NVDA) can match skipping groups (F6).

Recommendations: (1) Use the “Main” landmark only once on the page. (2) Avoid repeating the same landmark as it decreases the ability to outline unique regions of a page. (3) Region, form, content info, complementary,

navigation, and search can be repeated. But you must add a label to make them distinguishable. (4) Use them sparingly as too many landmark roles create "noise" in screen readers, making it difficult to understand the page's overall layout. Complementary is used to group tertiary information that will help users understand the context of the content.

This list helps UX designers and developers to map HTML elements to their ARIA roles correctly while minimizing unnecessary ARIA attributes.

- **Banner:** Represents introductory content or site-wide branding. It is typically at the top of a page but should not be inside `<article>` or `<section>` unless unique. ARIA landmark role `<header>`
- **Navigation:** Contains primary or secondary navigation links. It needs `aria-labelledby` when there are multiple navigation regions. ARIA landmark role `<nav>`
- **Search:** Identifies a section dedicated to search functionality. It acts as an implicit landmark without needing `role="search"`. ARIA landmark role `<search>`
- **Main:** The primary content of the page, one per page. It helps screen reader users skip repetitive elements like headers and sidebars. ARIA landmark role `<main>`
- **Complementary:** Holds related, but non-essential content (e.g., sidebars, ads, or widgets). It needs `aria-labelledby` if context is needed. ARIA landmark role `<aside>`
- **Region:** Defines a section of content that is meaningful on its own. It needs `aria-labelledby` if it needs a name for navigation. ARIA landmark role `<section>`
- **Contentinfo:** Contains site-wide or section-wide footer information. It shouldn't be used inside `<article>` unless unique to that article. ARIA landmark role `<footer>`
- **Form:** Becomes a landmark only when it has an accessible name (e.g., `aria-labelledby` or `<legend>` in fieldsets). ARIA landmark role `<form>`
- **Dialog:** Represents a self-contained interaction, such as a pop-up or modal. It becomes a landmark when it has `role="dialog"` or `aria-labelledby`. ARIA landmark role `<dialog>`
- **Article:** Represents self-contained content (e.g., news articles, blog posts). It typically used in feeds or lists of independent content pieces. ARIA landmark role `<article>`

The notation `<>` represents the HTML element that naturally map to each ARIA landmark role. These elements are semantic by default, meaning they automatically act as landmarks without needing extra ARIA roles (unless there are multiple, and they need differentiation).

For example:

- `<header>` automatically acts as a banner landmark (unless inside `<article>` or `<section>`).
- `<nav>` automatically acts as a navigation landmark (but might need aria-label if multiple exist).
- `<main>` automatically acts as a main landmark (only one per page).

Design Tip: Use the Figma Auto Layout alignment to position the content to the right, left, top, bottom or middle.



Figure 126: Screen readers users may use these keys to navigate landmarks.

SAP Figma Plugin

The [SAP Figma Plugin “Landmarks”](#) helps designers annotate their screens identifying relevant focus order sequence.

Examples



Figure 127: Scheme for applying different types of landmarks for a complex page. Regions and Forms require extra contextualization by labels to help blind users distinguish between them.

Shared Benefits



Temporary Disabilities

Everyone Benefits

Audio feedback is generally helpful in all situations when users are not looking at the user interface since they are busy with other tasks.

Developer Notes

UI technologies have different message raising techniques:

If you are using an UI framework that possibly encapsulates one or more of the methods below, look for respective framework API property documentation instead.

Web Development

By adding landmark role to HTML element

```
<div id="content" role="main">
<!-- ... Main Content goes here ... -->
</div>

<div id="nav" role="navigation">
<!-- ... navigation entries ... -->
</div>

<form role="search" ...>
<input type="search" name="searchinput" />
...
</form>
```

By using respective HTML element directly

```
<main id="content">
<!-- ... Main Content goes here ... -->
</main>
```

Mobile Development

Currently there is no API for landmarks in iOS and Android.

References

WCAG 2.2 Reference

[1.3.1 Info and Relationships](#) (A)

[2.4.1 Bypass Blocks](#) (A)

[2.4.1 Bypass Blocks](#) (A)

SAP Product Standard Accessibility 4.3

[ACC-263 - Information, Structures and Relationships](#)

[ACC-263 - Multiple Ways to Find Content](#)

[ACC-269 - Group Skipping](#)

Page Title

All users must be able to identify unique app contexts. Blind users with screen readers must recognize the page they landed on.

Variants

Web Browser Title



Figure 128: Indicate the page title on user agents (browser tab text) using a standard pattern all users can perceive to identify unique app contexts and screen readers announce correct title when a page loads.

Mobile App Title



Figure 129: Indicate the app title using a standard pattern all users can perceive to identify apps in mobile app switchers

About Page Title

Page Title is an unique identification text that helps users orient themselves when page titles are presented on browser tabs or in OS app/task switchers. The page title is the first announcement from the screen reader when a page or a view is loaded.

A visible page title helps all users locate where they are amongst the many pages of a product. Page titles in mobile devices are identical to the current view titles.

A Page Title is a combination of various identifiers, for instance one or more visible section titles, product name, and application name.

Sighted users easily recognize all these elements, but the information may be scattered or ungrouped on the page on desktops. Also, mobile apps

save space and usually hide some of this information. Therefore you have to use Page Title annotation to form up the complete text to be put e.g. on browser tabs and to be presented while screen reading.

Page Titles must be planned and informed during the design of a screen to allow the developer to code the title of the page correctly for all users.

Sometimes a page title is composed of invisible text. The Invisible Page Title variant should be used in all cases where some titles or names could be invisible. This will create a complete information for users of screen readers, especially in mobile scenarios for pages without visible titles.

A variety of combinations are possible to present a page title pattern. The sequence and presentation format of the individual parts has to be defined by company guidelines.

Examples for page titles:

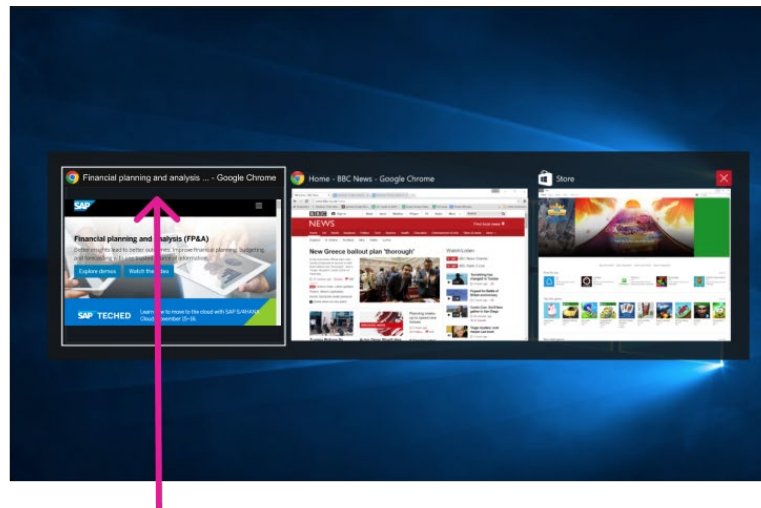
Page Title = Section Title(s) / App Name / Company Name

OR

Page Title = Product Name / Section Title(s)

Note: The OS or the browser may append to that the OS process name automatically, such as “Google Chrome” for Chrome browser or the app name such as “Microsoft Word”.

Examples



Web Browser Title

Financial Planning and Analysis | FP&A Software | SAP
Page Title + Product Title + Company Name

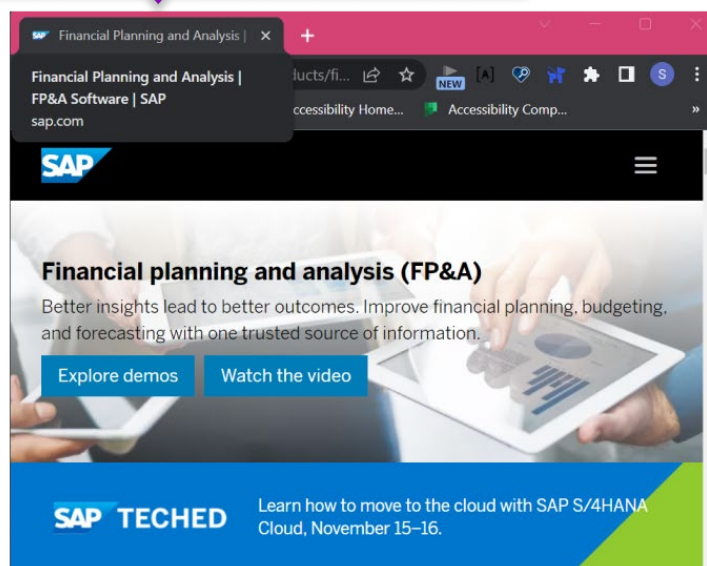


Figure 130: Web Browser window as part of open windows in OS Task Switcher overview. The title of the browser tab is labelling the respective task switcher item.

Browser Tab Text as Page Title

Here it is composed of:

Page Title + Product Title + Company Name

What matters here is that you define the different text elements to be put in as part of your design according to respective central company-wide rules.

Once defined, this text is also used to identify the Window in the OS App switcher.

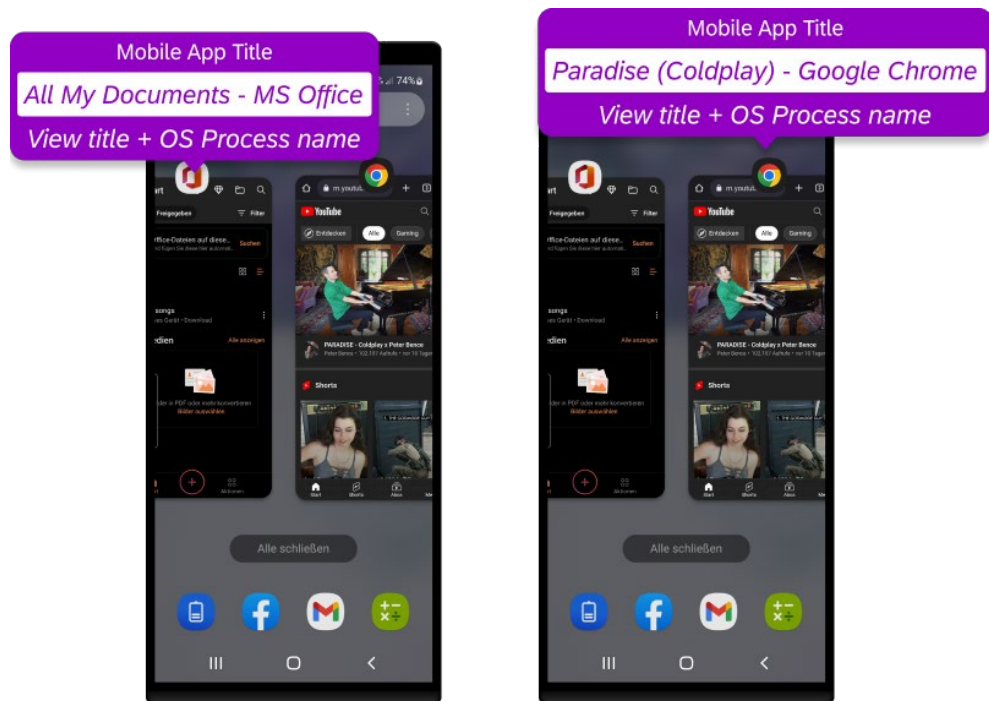


Figure 131: App window as part of open windows in OS Task Switcher overview. The title of the App window is labelling the respective task switcher item.

Mobile App “Page” Titles

Main view titles should be used to indicate the app in the task switcher.

Here the view title is composed of

View Title + OS Process name

(“OS Process name” content may be appended by the OS automatically)

Shared Benefits



Temporary Disabilities

Everyone Benefits

A visible page title helps all users locate where they are amongst the many pages of a product. Page titles in mobile devices are identical to the current view titles. Audio feedback is generally helpful in all situations when users are not looking at the user interface since they are busy with other tasks.

Developer Notes

UI technologies have different message raising techniques:

If you are using an UI framework that possibly encapsulates one or more of the methods below, look for respective framework API property documentation instead.

Web Development

By adding title element to head of page

```
<!DOCTYPE html>
<html>
<head>
<title>Customer Sales Orders - SAP CRM</title>
</head>
<body> ...
```

By modifying title element content dynamically via JS

```
top.document.title = "Customer Sales Orders - SAP CRM";
```

Mobile Development

iOS

For entire app and single screen views

```
var body: some View {
  NavigationView {
    List(landmarkData) { landmark in
```

```

NavigationLink(destination: LandmarkDetail(landmark:
landmark)) {
LandmarkRow(landmark: landmark)
}
}
.navigationBarTitle(Text("<your screen title>"))
}

```

Android

For entire app and single screen activities/views

```

<application
android:label="@string/turns_up_in_manage_apps" >
<activity
android:name=".MainActivity"
android:label="@string/mainViewTitle" >
...
</activity>
</application>

```

[L1][L2][L3][L4][L5][L6][L7][L8][L9][L10][L11][L12][L13][L14][L15][L16][L17][L18][L19][L20][L21][L22][L23][L24][L25][L26][L27][L28][L29][L30][L31][L32][L33][L34][L35][L36][L37][L38][L39][L40][L41][L42][L43][L44][L45][L46][L47][L48][L49][L50][L51][L52][L53][L54][L55][L56][L57][L58][L59][L60][L61][L62][L63][L64][L65][L66][L67][L68][L69][L70][L71][L72][L73][L74][L75][L76][L77][L78][L79][L80][L81][L82][L83][L84][L85][L86][L87][L88][L89][L90][L91][L92][L93][L94][L95][L96][L97][L98][L99][L100][L101][L102][L103][L104][L105][L106][L107][L108][L109][L110][L111][L112][L113][L114][L115][L116][L117][L118][L119][L120][L121][L122][L123][L124][L125][L126][L127][L128][L129][L130][L131][L132][L133][L134][L135][L136][L137][L138][L139][L140][L141][L142][L143][L144][L145][L146][L147][L148][L149][L150][L151][L152][L153][L154][L155][L156][L157][L158][L159][L160][L161][L162][L163][L164][L165][L166][L167][L168][L169][L170][L171][L172][L173][L174][L175][L176][L177][L178][L179][L180][L181][L182][L183][L184][L185][L186][L187][L188][L189][L190][L191][L192][L193][L194][L195][L196][L197][L198][L199][L200][L201][L202][L203][L204][L205][L206][L207][L208][L209][L210][L211][L212][L213][L214][L215][L216][L217][L218][L219][L220][L221][L222][L223][L224][L225][L226][L227][L228][L229][L230][L231][L232][L233][L234][L235][L236][L237][L238][L239][L240][L241][L242][L243][L244][L245][L246][L247][L248][L249][L250][L251][L252][L253][L254][L255][L256][L257][L258][L259][L260][L261][L262][L263][L264][L265][L266][L267][L268][L269][L270][L271][L272][L273][L274][L275][L276][L277][L278][L279][L280][L281][L282][L283][L284][L285][L286][L287][L288][L289][L290][L291][L292][L293][L294][L295][L296][L297][L298][L299][L300][L301][L302][L303][L304][L305][L306][L307][L308][L309][L310][L311][L312][L313][L314][L315][L316][L317][L318][L319][L320][L321][L322][L323][L324][L325][L326][L327][L328][L329][L330][L331][L332][L333][L334][L335][L336][L337][L338][L339][L340][L341][L342][L343][L344][L345][L346][L347][L348][L349][L350][L351][L352][L353][L354][L355][L356][L357][L358][L359][L360][L361][L362][L363][L364][L365][L366][L367][L368][L369][L370][L371][L372][L373][L374][L375][L376][L377][L378][L379][L380][L381][L382][L383][L384][L385][L386][L387][L388][L389][L390][L391][L392][L393][L394][L395][L396][L397][L398][L399][L400][L401][L402][L403][L404][L405][L406][L407][L408][L409][L410][L411][L412][L413][L414][L415][L416][L417][L418][L419][L420][L421][L422][L423][L424][L425][L426][L427][L428][L429][L430][L431][L432][L433][L434][L435][L436][L437][L438][L439][L440][L441][L442][L443][L444][L445][L446][L447][L448][L449][L450][L451][L452][L453][L454][L455][L456][L457][L458][L459][L460][L461][L462][L463][L464][L465][L466][L467][L468][L469][L470][L471][L472][L473][L474][L475][L476][L477][L478][L479][L480][L481][L482][L483][L484][L485][L486][L487][L488][L489][L490][L491][L492][L493][L494][L495][L496][L497][L498][L499][L500][L501][L502][L503][L504][L505][L506][L507][L508][L509][L510][L511][L512][L513][L514][L515][L516][L517][L518][L519][L520][L521][L522][L523][L524][L525][L526][L527][L528][L529][L530][L531][L532][L533][L534][L535][L536][L537][L538][L539][L540][L541][L542][L543][L544][L545][L546][L547][L548][L549][L550][L551][L552][L553][L554][L555][L556][L557][L558][L559][L560][L561][L562][L563][L564][L565][L566][L567][L568][L569][L570][L571][L572][L573][L574][L575][L576][L577][L578][L579][L580][L581][L582][L583][L584][L585][L586][L587][L588][L589][L590][L591][L592][L593][L594][L595][L596][L597][L598][L599][L600][L601][L602][L603][L604][L605][L606][L607][L608][L609][L610][L611][L612][L613][L614][L615][L616][L617][L618][L619][L620][L621][L622][L623][L624][L625][L626][L627][L628][L629][L630][L631][L632][L633][L634][L635][L636][L637][L638][L639][L640][L641][L642][L643][L644][L645][L646][L647][L648][L649][L650][L651][L652][L653][L654][L655][L656][L657][L658][L659][L660][L661][L662][L663][L664][L665][L666][L667][L668][L669][L670][L671][L672][L673][L674][L675][L676][L677][L678][L679][L680][L681][L682][L683][L684][L685][L686][L687][L688][L689][L690][L691][L692][L693][L694][L695][L696][L697][L698][L699][L700][L701][L702][L703][L704][L705][L706][L707][L708][L709][L710][L711][L712][L713][L714][L715][L716][L717][L718][L719][L720][L721][L722][L723][L724][L725][L726][L727][L728][L729][L730][L731][L732][L733][L734][L735][L736][L737][L738][L739][L740][L741][L742][L743][L744][L745][L746][L747][L748][L749][L750][L751][L752][L753][L754][L755][L756][L757][L758][L759][L760][L761][L762][L763][L764][L765][L766][L767][L768][L769][L770][L771][L772][L773][L774][L775][L776][L777][L778][L779][L780][L781][L782][L783][L784][L785][L786][L787][L788][L789][L790][L791][L792][L793][L794][L795][L796][L797][L798][L799][L800][L801][L802][L803][L804][L805][L806][L807][L808][L809][L810][L811][L812][L813][L814][L815][L816][L817][L818][L819][L820][L821][L822][L823][L824][L825][L826][L827][L828][L829][L830][L831][L832][L833][L834][L835][L836][L837][L838][L839][L840][L841][L842][L843][L844][L845][L846][L847][L848][L849][L850][L851][L852][L853][L854][L855][L856][L857][L858][L859][L860][L861][L862][L863][L864][L865][L866][L867][L868][L869][L870][L871][L872][L873][L874][L875][L876][L877][L878][L879][L880][L881][L882][L883][L884][L885][L886][L887][L888][L889][L890][L891][L892][L893][L894][L895][L896][L897][L898][L899][L900][L901][L902][L903][L904][L905][L906][L907][L908][L909][L910][L911][L912][L913][L914][L915][L916][L917][L918][L919][L920][L921][L922][L923][L924][L925][L926][L927][L928][L929][L930][L931][L932][L933][L934][L935][L936][L937][L938][L939][L940][L941][L942][L943][L944][L945][L946][L947][L948][L949][L950][L951][L952][L953][L954][L955][L956][L957][L958][L959][L960][L961][L962][L963][L964][L965][L966][L967][L968][L969][L970][L971][L972][L973][L974][L975][L976][L977][L978][L979][L980][L981][L982][L983][L984][L985][L986][L987][L988][L989][L990][L991][L992][L993][L994][L995][L996][L997][L998][L999] You can also use
binding.rootView.announceForAccessibility("My fragment
title")
in onCreateView() function of your fragment.[L1][L2][L3][L4][L5][L6][L7][L8][L9][L10][L11][L12][L13][L14][L15][L16][L17][L18][L19][L20][L21][L22][L23][L24][L25][L26][L27][L28][L29][L30][L31][L32][L33][L34][L35][L36][L37][L38][L39][L40][L41][L42][L43][L44][L45][L46][L47][L48][L49][L50][L51][L52][L53][L54][L55][L56][L57][L58][L59][L60][L61][L62][L63][L64][L65][L66][L67][L68][L69][L70][L71][L72][L73][L74][L75][L76][L77][L78][L79][L80][L81][L82][L83][L84][L85][L86][L87][L88][L89][L90][L91][L92][L93][L94][L95][L96][L97][L98][L99]

If you are not using viewBinding then just use the root view of the [L1][L2][L3][L4][L5][L6][L7][L8][L9][L10][L11][L12][L13][L14][L15][L16][L17][L18][L19][L20][L21][L22][L23][L24][L25][L26][L27][L28][L29][L30][L31][L32][L33][L34][L35][L36][L37][L38][L39][L40][L41][L42][L43][L44][L45][L46][L47][L48][L49][L50][L51][L52][L53][L54][L55][L56][L57][L58][L59][L60][L61][L62][L63][L64][L65][L66][L67][L68][L69][L70][L71][L72][L73][L74][L75][L76][L77][L78][L79][L80][L81][L82][L83][L84][L85][L86][L87][L88][L89][L90][L91][L92][L93][L94][L95][L96][L97][L98][L99] fragment's layout to call the function announceForAccessibility()

References

WCAG 2.2 Reference

[2.4.2 Page Title](#) (A)

SAP Product Standard Accessibility 4.3

[ACC 254 - Screen Titles](#)

Role and Properties

Define the meaning and purpose of each design element used during UI component design for assistive technologies (like screen readers). Every design element must have a role name that follows common terminology. Design or review the UI component properties, including their states and attributes.

Variants

WAI-ARIA Widget Role



Figure 132: These interactive roles act as standalone user interface widgets/elements or as larger composite widgets/elements

WAI-ARIA Structure Role



Figure 133: These roles describe structures that organize content in the user interface and are usually not interactive

WAI-ARIA Landmark Role



Figure 134: These roles are regions of the page intended as navigational landmarks

WAI-ARIA Live Region Role



Figure 135: These roles indicate live (=self-updating) regions in the user interface

WAI-ARIA Window Role



Figure 136: These roles act as windows within the browser or application

iOS UIAccessibilityTraits



Figure 137: Set these traits in native iOS apps to tell assistive apps how an UI element behaves or how to treat it

Android Roles



Figure 138: Set these roles in native Android apps to tell assistive apps how an UI element behaves or how to treat it

About Role and Properties

Connecting Roles and Properties Across Platforms

In both web and mobile interfaces, accessibility begins with clarity—clarity of purpose, structure, and interaction. Roles and properties are foundational tools that help assistive technologies interpret and communicate the purpose of each element to users. Whether it's a button, a status update, or a live region, assigning the right role ensures that the element behaves as expected across platforms. Properties, such as labels or state indicators, enrich that experience by adding meaningful context.

The annotations proposed in this section explores how roles and properties work together to provide accessible, predictable, and inclusive user experiences—on both the web and native mobile applications. While the syntax may vary between platforms, the principles of semantics, clarity, and user support remain constant. We cover common patterns, key differences, and how to annotate these elements effectively for diverse user needs so that they can be implemented for accessibility.

Combined annotation

The Role & Properties annotation is used to specify the semantic role name assigned to an implemented control, along with a space for component designers to list the relevant ARIA properties or mobile traits developers should apply. This ensures clarity in the expected functionality, behavior, and accessibility support of each component.

While scope annotations help frame larger containers or groupings, arrow annotations are better suited for pointing to simple or atomic elements that require semantic clarity—such as a button, input, or slider.

Component designers are encouraged to reference commonly used WAI-ARIA roles for web and mobile platform traits and roles (iOS and Android) to guide their annotation process. This alignment ensures accessibility is planned early and correctly implemented across platforms.

For example, consider a navigation control that appears as a menubar on desktop in full-screen view. On mobile, you might switch to a toggleable menu pattern. In such cases, the entire markup structure—and thus the role—changes. Rather than simply replacing a menubar role with a menu role, you'd likely restructure the component entirely to reflect the new interaction pattern. Annotating this shift early ensures the appropriate roles and properties are applied for both layouts, supporting accessibility without confusion.

Let's go through the example and identify ARIA properties (and mobile traits and roles, where applicable) that would typically be associated with a menubar on desktop and a menu or toggleable navigation on mobile:

1. Desktop View – Menubar

Role: menubar - A menubar is a container for menu items, typically used in desktop-style navigation.

Associated ARIA properties and states:

- `aria-haspopup="true"` (if a menu item opens a submenu)
- `aria-expanded="false"` or `"true"` (used dynamically on items that can be toggled open/closed)
- `aria-controls` (references the ID of the submenu controlled by the menu item)
- `aria-label` or `aria-labelledby` (gives an accessible name to the menubar)
- `aria-orientation="horizontal"` (optional but clarifies layout)

Child roles: `menuitem`, `menuitemcheckbox`, `menuitemradio`

2. Mobile View – Toggleable Menu

Role: menu (or simplified navigation role inside a toggleable container) - On mobile, navigation is often hidden behind a "hamburger" icon and expanded via user interaction.

Associated ARIA properties and states:

- `aria-expanded="false"` (on the toggle button, changes to true when menu is open)
- `aria-controls="menuID"` (to associate the toggle button with the menu container)
- `aria-label="Main menu"` (on the toggle button, describing its function)
`aria-hidden="true"` (used on the menu itself when it is visually and semantically hidden)
- `aria-labelledby` (used to associate the menu with a visible label, if applicable)

Mobile platform traits:

- iOS: `accessibilityTraits` like `button`, `selected`, `header`
- Android: `button` role, `android:contentDescription` with structural hierarchy hints

Role

A well-designed component communicates its function through both form and semantics. One of the most critical elements in this process is the role—a standardized term that identifies the type of user interface element. Component designers should rely on familiar, commonly understood role names to reinforce expected behaviors. According to the W3C, roles serve as the primary indicator of an element's purpose. This semantic association enables assistive technologies to present and interact with components in ways that align with user expectations, enhancing both usability and accessibility.

When designing accessible components, assigning the correct ARIA role is not just a developer concern—it's a critical part of the design specification. Roles define the semantic purpose of UI elements, enabling assistive technologies to communicate what an object is and how users can interact with it.

For component designers, observing roles involves more than naming:

- Match roles to expected behaviors. Roles must always align with user expectations and keyboard interaction patterns. If a component looks and behaves like a button, it must have the button role, regardless of its visual design.
- Specify roles explicitly in design specs. Annotate custom components (e.g., tab lists, accordions, sliders) with their intended ARIA roles. This helps ensure consistent implementation across dev teams and avoids accessibility regressions.
- Mind role restrictions. Some roles can only be used with specific parent or child roles (e.g., tab must be inside a tab list). Improper use can break accessibility.
- Use native elements first. Whenever possible, leverage semantic HTML elements (`<button>`, `<table>`, `<nav>`, etc.) that come with implicit roles. This simplifies development and ensures robust support across platforms and assistive technologies.

Please note: To indicate the corresponding native HTML element for developers, please also use our WAI-ARIA annotations since there is role parity with direct 1:1 mapping between ARIA roles and HTML elements which can be studied in the following W3C specification: [ARIA in HTML](#).

- Do not override native semantics unless necessary. Avoid removing or altering native roles (e.g., turning a `<button>` into a `div` with `role="button"`) unless you have a specific accessibility enhancement in mind—and even then, ensure all necessary keyboard and ARIA support is added.

- Think cross-platform. Some ARIA roles are interpreted differently across web and native mobile screen readers. Be cautious when designing components that will be implemented across multiple platforms.

By proactively addressing roles during the design phase, you support predictable, meaningful, and accessible user interactions—reducing ambiguity for developers and ensuring inclusivity for all users.

Properties

In accessibility design, properties are vital in defining key characteristics of an object that assistive technologies rely on to interpret and communicate meaning to users. Properties provide static or dynamic information about an element—such as how it behaves, what it controls, or how it should be presented—without altering its fundamental role.

For component designers, accurately assigning properties is essential. WAI-ARIA properties like `aria-labelledby`, `aria-describedby`, `aria-controls`, `aria-colspan`, and `aria-autocomplete` help describe relationships, structure, or behaviors that are not visually evident but crucial for screen reader users and other assistive tools.

While many properties tend to remain static (e.g., `aria-labelledby` typically points to a persistent label), others may change depending on user interaction. For instance:

- `aria-activedescendant`, `aria-valuenow`, and `aria-valuetext` are dynamic properties, often updated as the user interacts with a widget (e.g., sliders, combo boxes).
- Some, like `aria-multiline`, are configuration-like properties—set at design time and unlikely to change during use.

It's important to understand that while states and properties both start with the `aria-` prefix, they serve different purposes:

- States (e.g., `aria-checked`, `aria-expanded`, `aria-hidden`) reflect a condition that changes during interaction or automation.
- Properties describe attributes that often remain consistent, though exceptions exist (like `aria-expanded`, which can also act as a state in interactive contexts).

Understanding and applying properties correctly ensures that user agents and assistive technologies can communicate clear, consistent meaning and relationships, which is critical to creating accessible, intuitive experiences—on both web and mobile platforms.

Examples

WAI-ARIA Examples for annotating roles and properties of user interface elements in a web component inventory:

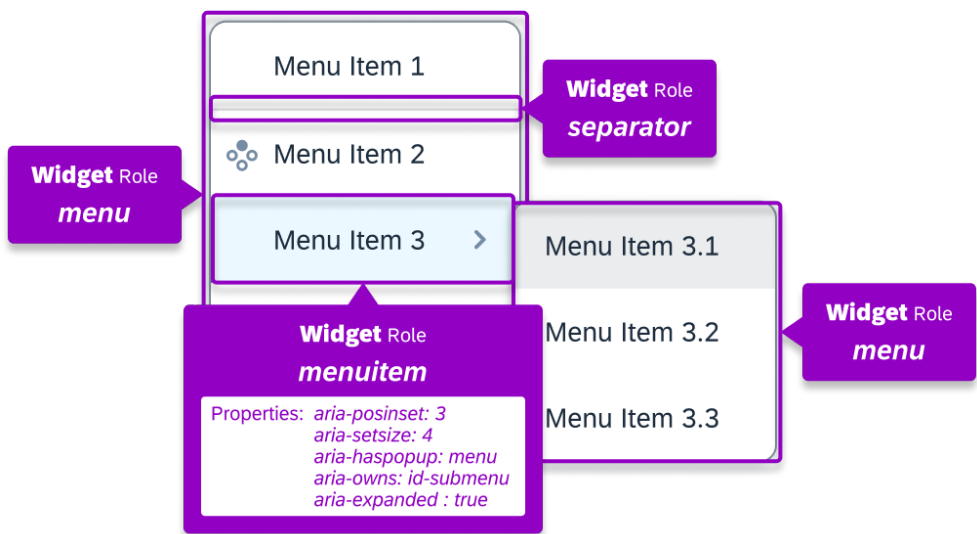


Figure 139: WAI-ARIA roles and properties for a menu with a submenu.

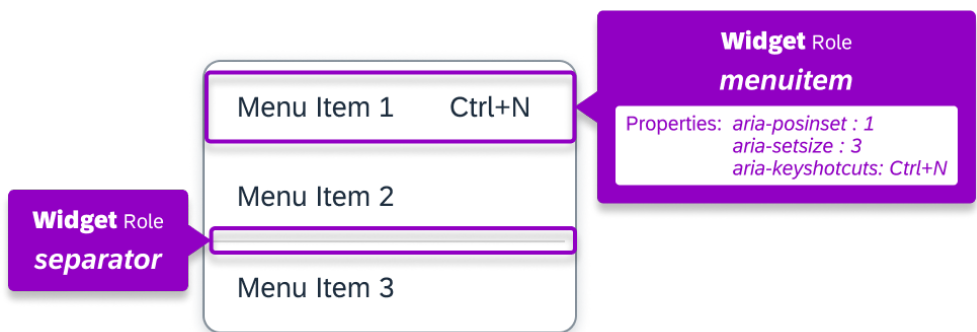


Figure 140: WAI-ARIA roles and properties for menu items



Figure 141: Non-Decorative interactive icon (acts on click)



Figure 142: Icon declared as decorative

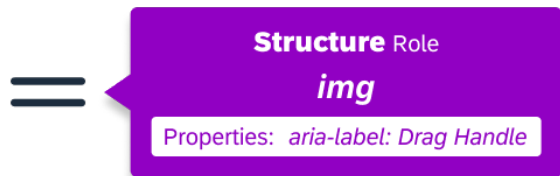


Figure 143: Non-Interactive Icon (but needed for dragging operations as focal point)

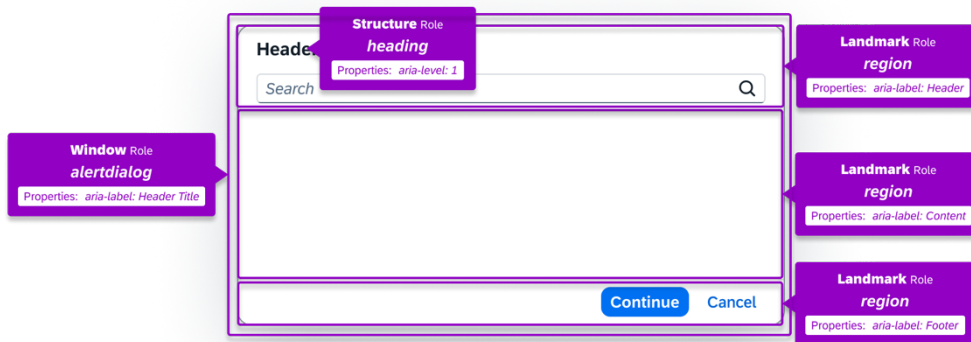


Figure 144: Section subdivisions in a floorplan for a large dialog

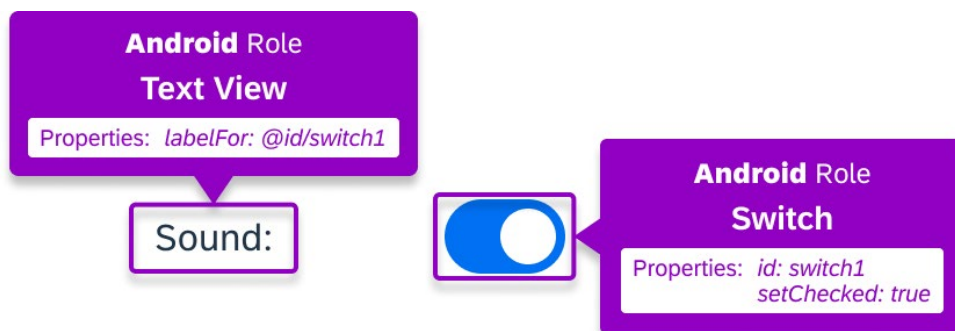


Figure 145: Role declaration and Label-control association for a custom Android switch component

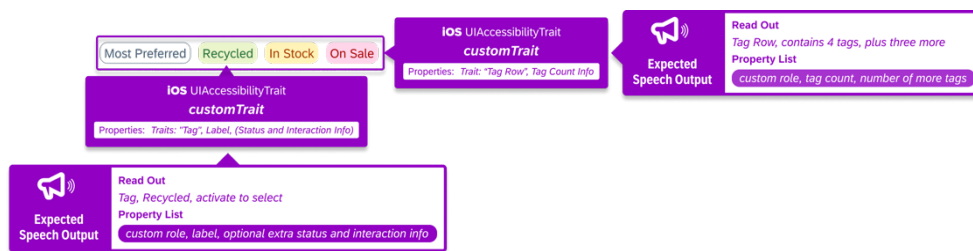
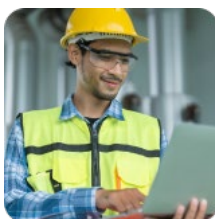


Figure 146: Accessibility Traits declarations for a custom iOS “Tag Row” component, using additionally the Expected Speech Output annotation for clarification

Shared Benefits



Temporary Disabilities

Everyone Benefits

Audio feedback is generally helpful in all situations when users are not looking at the user interface since they are busy with other tasks.

Developer Notes

Use the roles assigned by the designer. Discuss the choices made based on your experience in role assignments for controls.

If you have none, first get familiar studying the respective platform documentation!

If you are using an UI framework that encapsulates one or more of the roles below as part of its inventories, look for respective framework control documentation instead and check if the control maps to the intended usage in the design.

Web Development

WAI-ARIA roles for Web Development

[Accessible Rich Internet Applications \(WAI-ARIA\) 1.3](#)

Table 8: WAI-ARIA roles

Widget Roles	Structure Roles	Landmark Roles	Live Region Roles	Window Roles
button	application	banner	alert	alertdialog
checkbox	article	complementary	log	dialog
gridcell	blockquote	contentinfo	marquee	
link	caption	form	status	
menuitem	cell	main	timer	
menuitemcheckbox	code	navigation		
menuitemradio	columnheader	region		
option	comment	search		
progressbar	definition			
radio	deletion			
scrollbar	directory			
searchbox	document			
separator (when focusable)	emphasis			
slider	feed			
spinbutton	figure			
switch	generic			
tab	group			
tabpanel	heading			
textbox	img			
treeitem	insertion			
combobox	list			
grid	listitem			
listbox	mark			
menu	math			
menubar	meter			
radiogroup	none			
tablist	note			
tree	paragraph			
treegrid	presentation			
	row			
	rowgroup			

	rowheader			
	separator (when not focusable)			
	strong			
	subscript			
	suggestion			
	superscript			
	table			
	term			
	time			
	toolbar			
	tooltip			

Mobile Development

In iOS, control roles are called traits, while Android uses the role term instead.

For using these roles correctly, please consult the respective platform documentation:

iOS Traits

- [UIAccessibilityTraits at Apple Developer Documentation](#)

Android Roles

- [Android accessibility: roles and TalkBack - TetraLogical](#)
- [Talkback accessibility roles](#)

Table 9: iOS Traits and Android roles

iOS Traits	Android Roles
adjustable	Action Bar Tab
allowsDirectInteraction	Alert Dialog
button	Button
causesPageTurn	Check Box
header	Checked Text Preview
image	Date Picker
keyboardKey	Date Picker Dialog
link	Drawer Layout
none	Drop Down List
notEnabled	Edit Text
playsSound	Horizontal Scroll View
searchField	Icon Menu
selected	Image
startsMediaSession	Image Button

staticText	Keyboard Key
summaryElement	List
supportsZoom	Number Picker
tabBar	None
toggleButton	Pager
updatesFrequently	ProgressBar
	Radio Button
	Role Grid
	Scroll View
	Seek Control
	Sliding Drawer
	Staggered Grid
	Switch
	Tab Bar
	Talkback Edit Text Overlay
	Text Entry Key
	Time Picker
	Time Picker Dialog
	Toast
	Toggle Button
	View Group
	Web View

Custom Components

Sometimes it is necessary to create custom components (instead of using the available controls) and assign roles to them.

iOS

Make your component to be an AccessibilityContainer then give it a label, a group (if needed), child elements, etc.

Custom components require much effort, see videos and documentation

- <https://developer.apple.com/videos/play/wwdc2021/10119/>
- <https://developer.apple.com/videos/play/wwdc2021/10122/>

Android

How to make accessible components

1. Extend existing view or subclass with own class
2. Override the methods from the superclass
3. Handle directional controller clicks
4. Implement accessibility API methods

5. 5. Send AccessibilityEvent objects specific to your custom view
6. Populate AccessibilityEvent and AccessibilityNodeInfo for your view
7. Add custom actions for components

```
ViewCompat.setAccessibilityDelegate(set_actions_button
, object : AccessibilityDelegateCompat() {
    override fun onInitializeAccessibilityNodeInfo(v:
    View, info: AccessibilityNodeInfoCompat) {
        super.onInitializeAccessibilityNodeInfo(v, info)
        info.addAction(AccessibilityActionCompat(
        AccessibilityNodeInfoCompat.ACTION_CLICK, "Edit
        note"))
        info.addAction(AccessibilityActionCompat(
        AccessibilityNodeInfoCompat.ACTION_LONG_CLICK, "Copy
        note"))
    }
})
```

8. Add custom role names for components

```
ViewCompat.setAccessibilityDelegate(login_text_view_as
_button_with_role,
object : AccessibilityDelegateCompat() {
    override fun onInitializeAccessibilityNodeInfo(v:
    View, info: AccessibilityNodeInfoCompat) {
        super.onInitializeAccessibilityNodeInfo(v, info)
        info.roleDescription = "Button"
    }
})
```

See also:

<https://developer.android.com/develop/ui/views/layout/custom-views/custom-components>

<https://developer.android.com/guide/topics/ui/accessibility/custom-views>

<https://developer.android.com/codelabs/advanced-android-kotlin-training-custom-views#0>

<https://tetralogical.com/blog/2022/07/07/android-accessibility-roles-and-talkback/>

<https://developer.android.com/reference/kotlin/androidx/compose/material/package-summary#Slider>

References

WCAG 2.2 Reference

[1.3.1 Info and Relationships](#) (A)

[4.1.2 Name, Role, Value](#) (A)

SAP Product Standard Accessibility 4.3

[ACC 263 - Information, Structures and Relationships](#)

[ACC 264 - Interoperability with Assistive Technologies](#)

Speech Output

Plan the Speech output by describing what a screen reader would read out during keyboard focus.

Variants

Speech Output



Figure 147: Describe what a screen reader should speak on element focus

About Speech Output

From the large context of a page to the details of a component. Just like an entire page must fulfill accessibility requirements to help users explore content in a macro level, a component must fulfill accessibility requirements to enable users to conduct micro interactions.

With that in mind, we recommend addressing all annotation groups, from foundation to complementary. The combination of annotations from these different groups will guarantee a pleasant experience for all user types, including sighted or blind users.

But there are three basic aspects for users of screen readers you must inform in the component level: Speech Output, Role and Properties. These annotations will be made available on the component library for consultation and do not require redundant information when handing off designed pages.

Speech Output is an example of a screen reader reading out according to the component attributes. However, a simulated speech output is just an expectation expressed by component and application designers. There is no guarantee that implementation will correctly map the text to how a screen reader will read it out.

A sequence of properties recommended consists of the following:

- LABEL(S): visible or invisible associated label(s)

- **ROLE:** component type/role (such as “button”)
- **VALUE:** contained value(s) (such as numbers or text)
- **STATE(S):** state(s) (such as “disabled”)
- **OTHER PROPERTIES:** other properties (such as “required” or the placeholder)
- **DESCRIPTION(S):** visible or invisible description(s) suggesting some helpful information to the user such as units (“EUR”)
- **HOT KEY:** assigned hot keys (such as “Ctrl+S” for saving operation)
- **ACCESS KEY:** assigned access keys (such as “Alt+S”)
- **TUTOR INFO:** additional info for blind users (other keys to use)

For clarity, you can use dashes as delimiters between the different properties. These will not be included in speech output. Keep the text as short as possible and remember that blind people frequently need to listen; the more text-based information you provide, the more tedious this will become.

Use this annotation also to avoid the applications to become too chatty. Less is more.

Examples

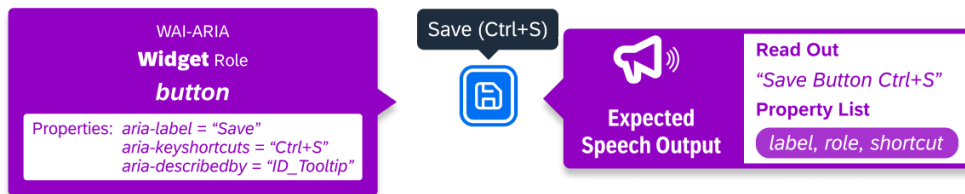


Figure 148: Expected speech output on keyboard focus for a custom tooltip of an icon button as combination of hidden button label and keyboard shortcut

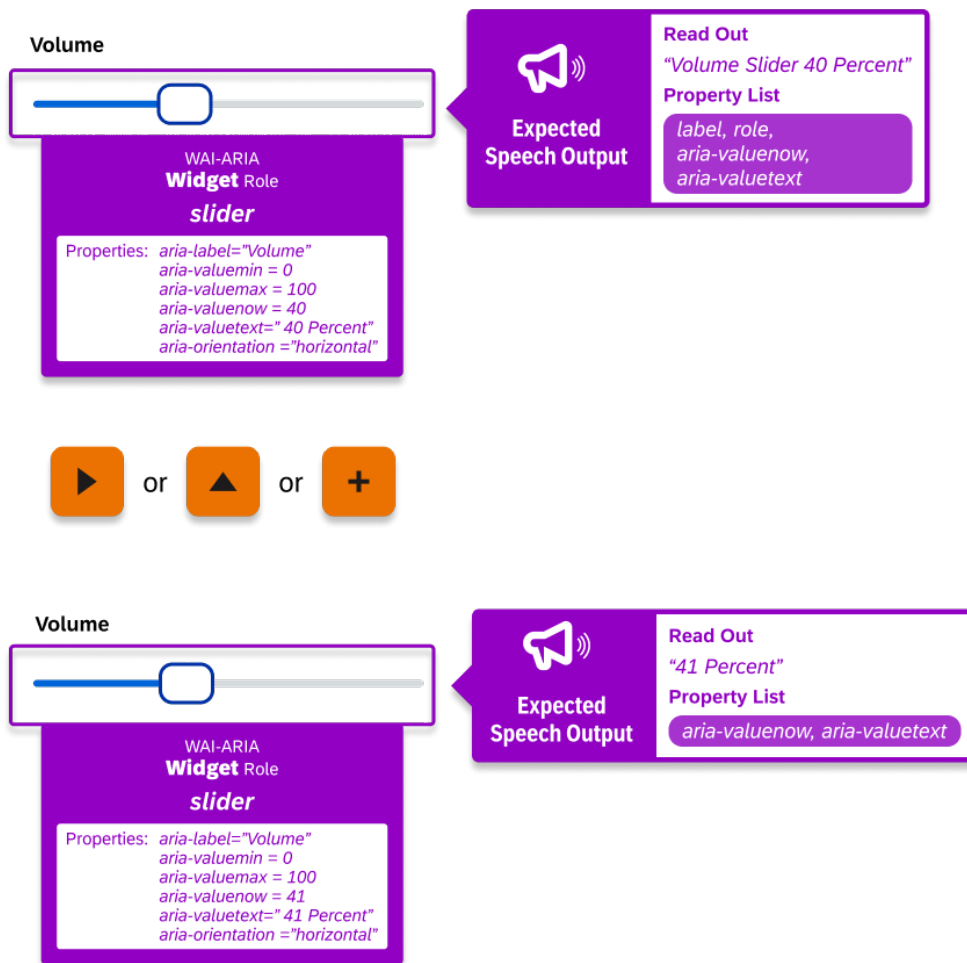
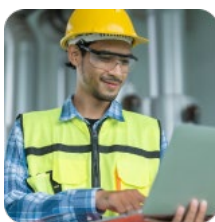


Figure 149: Expected speech output for a change in a value of a focused volume slider control (combined with Role and Property annotations, arrow right/up plus keys increment value by one)

Shared Benefits



Temporary Disabilities

Everyone Benefits

Audio feedback is generally helpful in all situations when users are not looking at the user interface since they are busy with other tasks.

Developer Notes

What you get from the annotation is a rough expectation of the individual parts of the speech string to be spoken, it is NOT a binding command with respect to the sequence of the different parts.

Screen readers on the different platforms have built-in heuristics in which they speak the roles and attributes. Don't try to change that by artistically juggling with properties and over-denote property meaning and content.

For instance, it is NOT OK to put descriptions or even object values in object labels “just that it will be spoken”. Use description properties for that!

Follow the rules for correct role assignment and attributing given in the API documentation for web and mobile development, test with a screen reader / User agent combination of choice on the platform and evaluate if the result fits to the output the designers have given you as speech expectation.

References

WCAG 2.2 Reference

[1.3.5 Identify Input Purpose](#) (AA)

[2.5.3 Label in Name](#) (A)

[3.1.5 Reading Level](#) (AAA)

[4.1.2 Name, Role, Value](#) (A)

[4.1.3 Status Messages](#) (AA)

SAP Product Standard Accessibility 4.3

[ACC 263 - Information, Structures and Relationships](#)

[ACC 264 - Interoperability with Assistive Technologies](#)

Audio Control

Keep sound auto play under 3s and provide mechanisms to pause or stop, or to control volume.

Variants

Audio Control Pause – Stop



Figure 150: Stop and Pause helps users prevent screen reader announcement overlap with background sound.

Audio Control - Volume



Figure 151: Volume controls mainly benefit blind and cognitive users, not those with hearing loss.

About Audio Control

If any audio on a Web page plays automatically for more than 3 seconds, either a mechanism is available to pause or stop the audio, or a mechanism is available to control audio volume independently from the overall system volume. any user, including those who are hard of hearing, neurodivergent, or easily distracted must be able to identify, control, stop, or muting these sounds.

The WCAG Success Criteria 1.4.2 Audio Control (A) is meant to provide mechanisms to stop or pause sound or to make available an audio volume control independently from the overall system volume level. Blind users need control over audio playback and descriptions of visuals because they find unexpected audio disorienting because the overlap can block access to the announced content. Screen reader speech needs to be clear and uninterrupted.

People who use screen readers rely heavily on audio for navigation. Overlapping sounds from auto-playing media and screen readers can create confusion, cognitive overload, or even make it impossible to understand either stream. Sound control helps prevent interference, allowing users to manage the timing and presence of audio.

If any audio plays automatically for more than 3 seconds, the sound can overlap with screen reader output, making it hard or impossible for blind users to hear what the screen reader is saying. In this case either a mechanism is available to pause or stop the audio, or the user is warned and given the opportunity to control audio volume independently from the overall system volume.

Why This Matters? Screen reader users rely on audio output — overlapping sounds (from autoplay videos, music, or alerts) can interrupt or obscure the screen reader. But also, cognitive users may be startled or distracted by unexpected sound, disrupting focus. And users with auditory processing difficulties can become overwhelmed when unable to control competing audio streams.

When documenting audio in the design make controls accessible to provide an accessible experience to all users. Avoid auto playing audio. If autoplay is necessary, ensure it lasts less than 3 seconds, or provide a visible and programmatically accessible control to pause or stop the audio. This ensures blind users can use their screen readers without interference—and access content confidently.

Compliance with accessibility guidelines requires sounds to be available to a broader range of accessibility needs:

- controls are accessible and navigable via keyboard.
- labels are clear for screen reading users.
- UI element from control is focusable.
- sound is paired with captions or transcripts.

Play/Stop Controls:

- Provide visible, easy-to-use controls to Play and Stop media content.
- Controls should be clearly labeled (e.g., "Play" and "Stop") and
- accessible via keyboard and screen reader.
- Ensure the controls are prominent in the UI, so users can quickly pause or stop content as needed.
- Allow for easy muting of audio or adjusting volume when content includes background sound, music, or noise that might interfere with captions or reading.

Examples

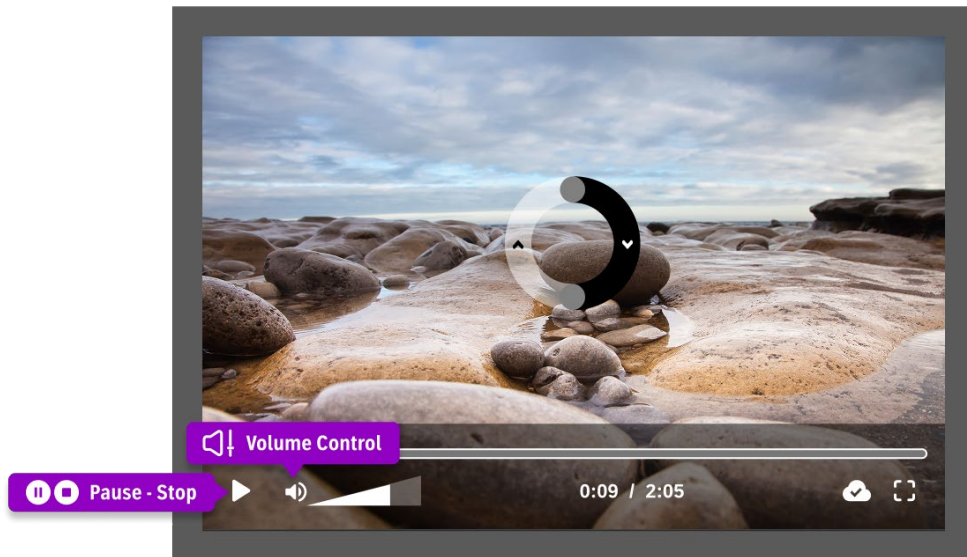


Figure 152: Volume of automatically played audio content can be changed independently of the system volume. Videos can be paused or stopped. Animated controls must be created to be flickering safe, like busy indicators.

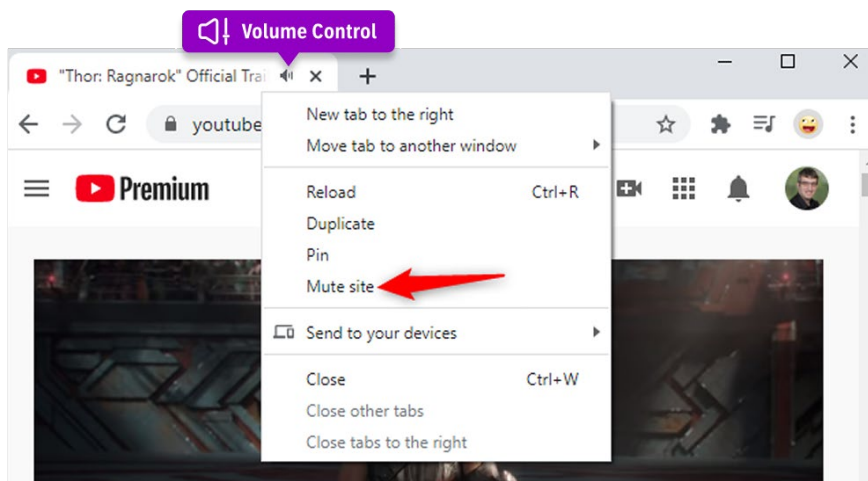


Figure 153: Automatically played audio content can be muted

Shared Benefits



Vision and Color Limitation

Visuals

Some low vision users rely on screen magnifiers and audio cues. Being able to pause or control audio allows them to consume information at their own pace, without being rushed by background media they can't quickly identify or stop.



Limited Cognition

Cognitive

For users who are sensitive to unexpected sounds, such as people with autism, ADHD, or anxiety, unanticipated audio can be distressing or disruptive. Sound controls give them the option to avoid sensory overload, helping them stay focused on the task. Reducing unwanted audio by muting or stopping the audio entirely helps reading captions or reviewing a transcript. Providing clear Play/Stop or Mute controls ensures they are not forced to listen to audio content they may not need, enhancing their experience.



Without or Limited Hearing

Sound

Deaf and hard-of-hearing users need text or sign-based alternatives to sounds - audio alone is not accessible. These users rely on visual alternatives like captions and transcripts. But accessibility isn't just about providing those alternatives—it's also about giving control. Include clear Play, Pause, Stop, and Mute controls for audio and media content so that users read at their own pace while reviewing captions or transcripts, users avoid missing key information by controlling playback timing, and users mute or skip audio they don't need, reducing distraction or overload. Sound control supports autonomy, improves comprehension, and respects user preferences—especially for those navigating media content visually rather than aurally.

Developer Notes

Use the audio controls for the platform or framework inventory components triggering respective sound output API properties on the platform.

References

WCAG 2.2 Reference

[1.4.2 Audio Control](#) (A)

SAP Product Standard Accessibility 4.3

[ACC 279 - Animation, Audio and Video Control](#)

Audio Description

Use the “Audio Description” annotation to indicate that key visual content is described in audio or a separate track, ensuring users who are blind or have low vision can fully understand the media.

Variants

Audio Description



Figure 154: Available audio describes key visual content inside or in a separate track.

About Audio Description

Audio Description is a required requirement to describe important visual details not spoken in the main audio, but it's meant for technical writers to provide description to non-spoken dialogs.

To make videos accessible to people who are blind or cannot understand visual content, provide synchronized or extended audio descriptions—spoken explanations of key visual elements. These descriptions ensure users can follow what's happening on screen even without seeing it.

An audio description must be provided for any visual content in the video that is not conveyed through the existing audio (e.g., silent actions, facial expressions, on-screen text). This description is typically added as a secondary audio track that fits into pauses in the main narration.

When standard pauses in the video aren't enough to convey essential visuals, extended audio descriptions must be added to meet accessibility requirements. This supports compliance with WCAG Success Criteria 1.2.5 and 1.2.7, which require audio descriptions for all pre-recorded synchronized media. Extended Audio Description highlights temporal accommodation for more detailed descriptions and requires pausing video to insert longer descriptions of visual content. WCAG 1.3.3 Sensory Characteristics ensures that information that isn't conveyed by visuals alone is also accessible in non-visual formats.

If both 1.2.3 and 1.2.5 requirements are met by providing an audio description and a text description, there is no need to fulfill the 1.2.8 Media Alternative (Prerecorded) (AAA) requirement.

Examples

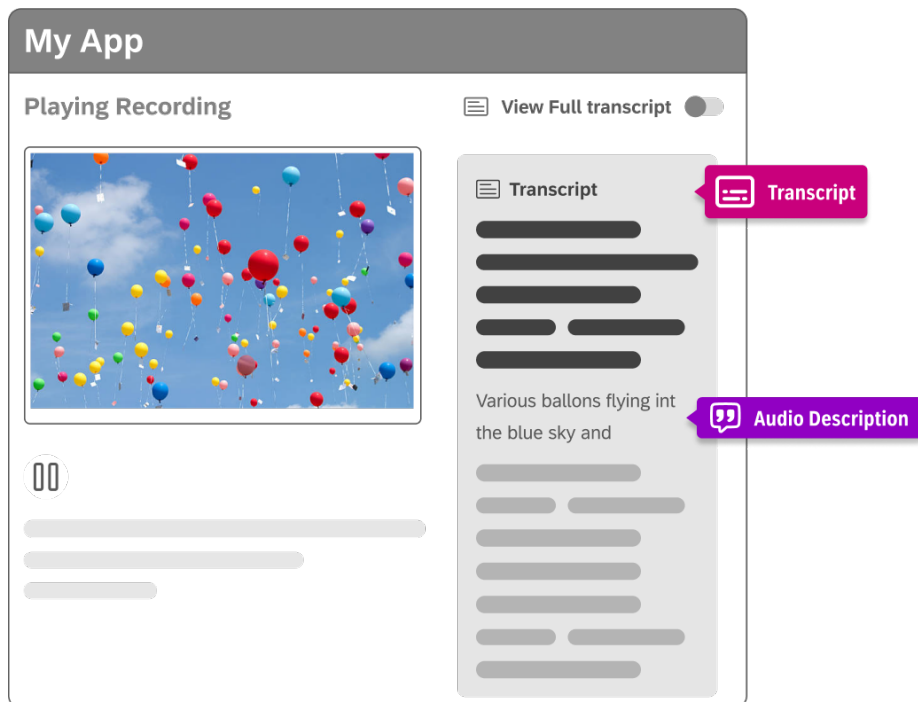


Figure 155: Synchronized Audio Descriptions as text alternative for non-text content (video) as part of a transcript

Developer Notes

Use framework inventory components containing or linking to respective transcript and audio description resources on the platform.

References

WCAG 2.2 Reference

[1.2.5 Audio Description \(Prerecorded\)](#) (AA)

[1.2.7 Extended Audio Description \(Prerecorded\)](#) (AAA)

[1.3.3 Sensory Characteristics](#) (A)

SAP Product Standard Accessibility 4.3

[ACC 255 - Text Alternative for Non-Text Content](#)

Language

Specify the default language for the user interface and identify where language changes on the user interface occur.

Variants

Page Language

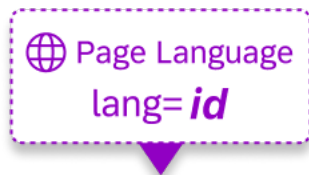


Figure 156: Specify the default language of a page

Part Language

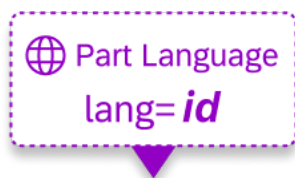


Figure 157: Specify where language changes are expected

About Language

Language of Page, levels A and AA are success criteria that ensures correct assignment of primary language of a webpage or any change in language within a page. The goal is to consider multilingual content by ensuring the primary language is clearly defined for screen readers and other assistive technologies. Planning language of page and language of parts prevents miscommunication across globalized interfaces.

Examples where language may be considered in the page layout are:

- quote or sentence in French within an English page (e.g., Bonjour)
- greetings in a banner that updates the language upon page refresh (e.g. Bienvenido, Bienvenu, Willkommen, Benvenuto)

Why It Matters: Screen readers rely on this to read content with correct pronunciation and intonation. It also important for screen readers to pronounce foreign phrases correctly, while supporting language learners and global users, For pubic websites it improves SEO and machine translation quality.

As a designer you should observe and specify the following:

- **Default Page Language:** Define the main language for the entire page or screen. Inform the default language in the design documentation to guide developers to set the lang attribute in the HTML tag. Use consistent language throughout the page to avoid confusion for users relying on assistive technologies. If designing for localization, specify which version of the design corresponds to which language (e.g., English, French, Spanish).
- **Part Language:** Informs the human language of each passage or phrase. Designer must identify any text in a different language from the page's main language and inform the content in the page that is exposed to language changes. Flag it as a different-language element that needs to be marked with the correct language attribute. Avoid mixing languages without clear separation in UI.

Examples

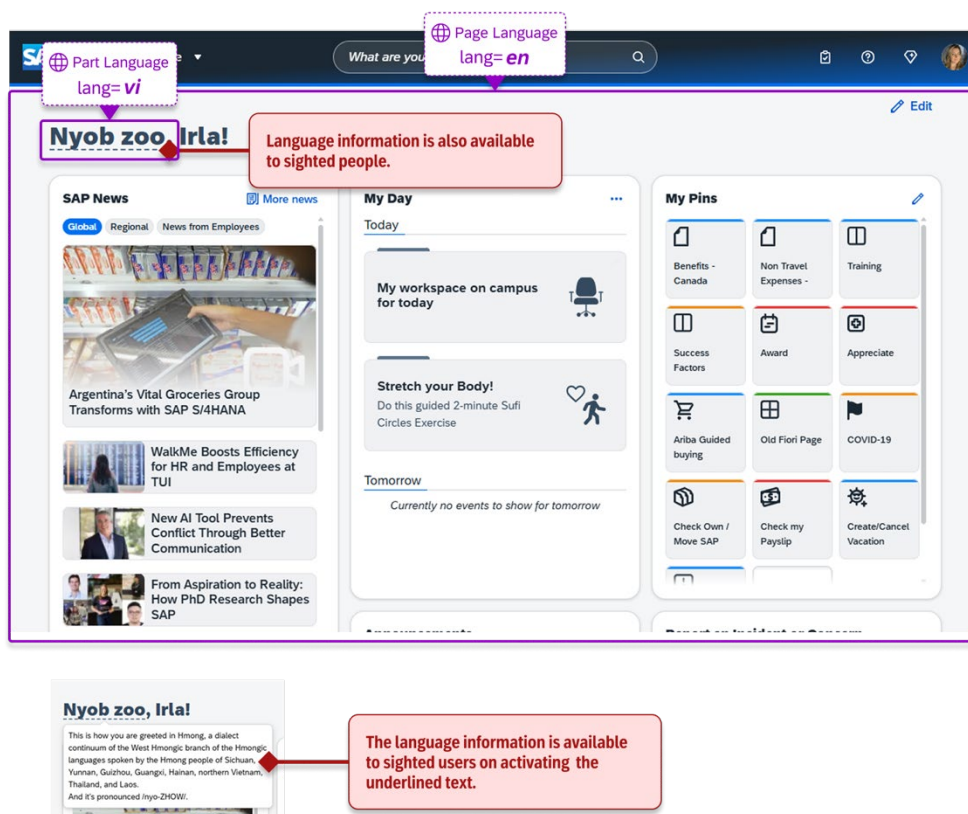


Figure 158: Page is in English but contains quotes in other languages

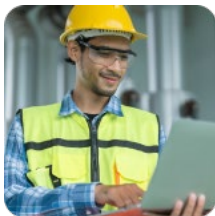
Shared Benefits



Limited Cognition

Cognitive

Those with cognitive and learning disabilities may use text-to-speech (TTS) tools that rely on correct language metadata to pronounce words correctly. For example, someone with dyslexia might use TTS in English, and without lang="es", a phrase like “¿Cómo estás?” might be read in a garbled way. Proper language cues help comprehension and reduce confusion.



Temporary Disabilities

Everyone Benefits

Multilingual or Non-Native Speakers may rely on machine translation tools (like Google Translate), which perform better when the language is explicitly defined. Clear lang attributes improve translation accuracy and grammar context, making information more accessible. Helps language learners navigate hybrid-language pages (e.g., English sites with French quotes).

Users Relying on Assistive Technologies or Smart Agents benefit from voice assistants (like Siri or Google Assistant) and screen readers use lang tags to determine pronunciation, pitch, and grammar parsing. This also affects speech input systems (for example, dictation in a bilingual context), which may misinterpret words without a clear language definition.

Sighted Users Using Read-Aloud Tools like Read Aloud, Natural Reader, or native browser TTS (e.g., Safari's Reader Mode) rely on language tags to deliver natural-sounding speech. Incorrect language attribution can create frustration due to robotic or incorrect pronunciation — even for sighted users listening instead of reading.

Organizations and Legal/Compliance Teams: while not a “user” group, this requirement serves legal departments trying to mitigate risk and demonstrate inclusive practice. Mispronounced phrases due to missing language declarations can be used in legal cases to show lack of due diligence, especially in public sector or multilingual countries.

Developer Notes

Web Development

Developers should ensure proper markup to use the `lang` attribute for specific sections.

On body tag (for page language, all UI text) and in page (for parts of UI text):

```
<body lang="en">  
Search results:  
<a href="oldage_german.doc" lang="de">  
Die Geschichte des Mittelalters</a>  
<a href="us_history.doc" >  
American History</a>  
</body>
```

Mobile Development

As of today, there is no concept for part language identification in mobile platforms.

References

WCAG 2.2 Reference

[3.1.1 Language of Page](#) (A)

[3.1.2 Language of Parts](#) (AA)

SAP Product Standard Accessibility 4.3

[ACC 265 - Language Attributes](#)

Cognitive Experience

Imagine you're looking at a screen with lots of information. How much can you read and understand before it feels like too much? How many ideas can you hold in your mind at once to decide what to do next? What helps you stay focused, feel confident, and not overwhelmed?

This chapter helps designers to observe cognitive needs like learning and memorization.

The Inclusive Cognitive Experience

Meeting the needs of people who are neurodiverse

Cognitive annotations focus on reducing cognitive load by providing clear, contextual information that aids comprehension, wayfinding, and decision-making for users with diverse cognitive abilities.

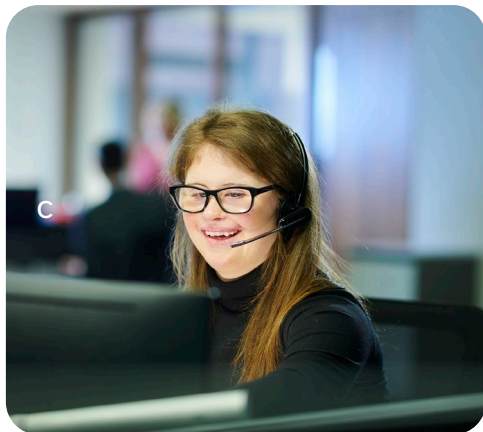
While visual, screen reading, and interactive experiences address a significant portion of accessibility standards, additional considerations are needed to support invisible disabilities, such as neurodiversity. Cognitive annotations help bridge these gaps by addressing aspects like time limits and refresh mechanisms, motion content, wayfinding, and error handling—enhancing both accessibility and overall usability.

By integrating cognitive annotations, interfaces become more intuitive and inclusive, ensuring that users with diverse cognitive abilities can navigate, process information, and complete tasks with greater ease. Design for individuals who may have difficulties with memory, attention, problem-solving, or processing information, results in experiences that are simple, clear, and easy to understand.



Principles

Neurodiversity



Persona: Chloe

"I have memory loss, trouble focusing, and problems solving complex tasks, and sometimes confusion and disorientation. My dyslexia, dyscalculia, and ADHD affect my focus, attention, and learning. My preferred tool in digital products are helpers and documentation."

This user group represents individuals with dyslexia, dyscalculia, and ADHD, who rely on helpers and documentation in digital products to support their focus, attention, and learning. We use the term Neurodivergent to talk about cognitive differences.

Neurodivergent is an umbrella term including many conditions like Tourette's Syndrome, ADHD, Autism, Gifted, Dyslexia, Dyscalculia, and Dyspraxia. It is the contrary of Neurotypical, which refers to individuals who doesn't meet what's deemed "normal" by societal standards and struggle with intellectual and functional skills. These individuals may have unique cognitive strengths and perspectives that contribute to diverse ways of thinking, learning, problem-solving and socializing. Neurodiversity includes all human brains - after all, everyone's experience of the world is slightly different.

Statistics:

- 1 in 6 people worldwide live with a neurological condition that can affect cognitive functioning (WHO).
- Around 13.9% of adults in US reports having a cognitive disability, defined as serious difficulty concentrating, remembering, or making decisions (CDC).
- Dyslexia affects 5–10% of the population - some studies estimate 17%.



- ADHD affects approximately 2.5% of adults and 5–7% of children globally.

Situational & Temporary Disabilities

Permanent: Conditions like ADHD, dyslexia, brain injuries, learning disabilities, or developmental disabilities.

By integrating situational and temporary disabilities into this disability group, we help design teams address challenges that might not be permanent but still impact user experience under certain conditions.

Situational and temporary

- Stress-induced cognitive overload: Situations of high emotional stress (e.g., work deadline or personal crisis) could impair cognitive processing, making it hard to focus on tasks.
- Fatigue: A lack of sleep, burnout, or exhaustion could impair concentration or memory, similar to cognitive disabilities.
- Multitasking demands: In busy environments (like open offices or crowded spaces), cognitive overload can temporarily impact the ability to process information effectively.



General Design Tips

1. **Clear and Predictable Interfaces:**

Sudden layout shifts, ambiguous flows, or inconsistent patterns increase anxiety and disrupt focus.

- Design Tip: Keep navigation, layout, and terminology consistent. Use progressive disclosure and prepare users for change through tooltips or banners.

2. **Step-by-Step Guidance:**

Difficulty solving problems, remembering previous steps, or interpreting system feedback.

- Design Tip: Break tasks into smaller steps. Use clear instructions, plain language, familiar icons, and specific feedback messages.

3. **Low-Stimulation, Calm Environments:**

Sensory sensitivity and attention regulation difficulties. Visual clutter, motion, or loud sounds can trigger anxiety or distraction.

- Design Tip: Reduce motion, avoid flashing or autoplay elements, and use calm color palettes. Offer distraction-free modes when possible.

4. **Reduced Memory Reliance:**

Users may forget context, instructions, or prior steps. Working memory limitations can lead to errors or abandonment.

- Design Tip: Keep critical info visible (e.g., task summaries, form labels). Use persistent headers or indicators to anchor users in the flow.

5. **Visual Structure and Organization:**

Disorganized interfaces make it harder to track progress or complete tasks.

- Design Tip: Use progress bars, checklists, or visual groupings to clarify structure. Minimize decisions per screen.

6. **Plain and Direct Communication:**

Complex language or vague content increases cognitive load and misunderstanding.

- Design Tip: Use short sentences, familiar terms, and bullet points. Highlight key messages and avoid jargon.



7. Routine and Familiarity:

Unexpected design changes disrupt mental models and create stress

- Design Tip: Maintain predictable behavior. If change is necessary, announce it clearly and give support.

8. Customizable Experience:

Cognitive needs vary widely; one-size-fits-all design causes friction.

- Design Tip: Let users adjust layout density, motion, font size, and colors. Save preferences across sessions when possible.

9. Visual Support for Comprehension:

Trouble understanding abstract UI elements or instructions.

- Design Tip: Use clear labels, icons with text, and simple metaphors. Reinforce meaning visually and verbally.

10. Focus-Friendly Layouts:

Difficulty sustaining attention due to cognitive overload or distractions.

- Design Tip: Design clean interfaces with a clear hierarchy. Group related items, reduce noise, and avoid multitasking traps.



Annotations

Cognitive Annotations

The checklist help designers to remember the important aspects of accessibility that should be addressed in the design phase.

Cognitive Experience

- ☐ Wayfinding and Orientation
- ☐ Semantic
- ☐ Error Handling
- ☐ Motion Content
- ☐ Time Limit
- ☐ Multiple Ways
- ☐ Redundant Entry



Wayfinding and Orientation

Provide descriptive labels to allow users to understand transfers between pages. Allow users to locate and orient themselves within a set of pages. Offer more than one method to find individual screens and content, like navigation menus and search.

Variants

Navigation Identifier



Figure 159: Navigation identifiers promote safe transfers and predictability. Links transfer users to new pages or sections within a page. Buttons open dialogs, pop-ups, panels, or expand areas in the screen to show more information. Menus open pages or sections and are part of a common navigation structure organized vertically or horizontally like tab lines or breadcrumbs.

Change in Context (Button)

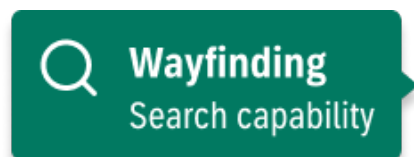


Figure 160: Eliminate potential confusion caused by change in context, which can be disruptive to many user groups: motor impairments, low vision, blind, and people with cognitive impairments. Provide support like and icon and invisible label when link opens a new window or sends user to a new product to inform the user what will happen next.

Label and Title Match



Figure 161: Labels on buttons, links, and menus build expectations for planning and executing routes.

Search



Figure 162: Search capability offers more ways for users to find what they are looking for. A list of possible destinations complements the navigation of complex sites or products with several layers of information and dense content.

About Wayfinding and Orientation

Wayfinding strategies, including visual cues and descriptive texts, ensure a clear and consistent exploration. It allows users to identify and plan routes or navigation flows, maintaining their path and helping them stay on track to reach desired pages or content. Wayfinding is commonly associated with transfers and is supported by key UI elements like links, buttons, menus, search and breadcrumb.

Users orient themselves looking for elements strategically located on the screen. They will have a successful exploration if these elements are placed consistently across different pages, accounting for primary, secondary, and tertiary navigation patterns. Wayfinding allows them to ascertain their position and plan new routes.

Navigation identifiers help users understand where they are within a digital product, making it easier to navigate, avoid getting lost, and stay oriented—especially in multi-step processes or large content structures.

The WCAG 2.4.8 Location (AAA) ensures that users can determine where they are within a website or application. This helps with orientation and navigation, especially for users with cognitive disabilities, screen reader users, or anyone exploring unfamiliar content. This means that users should have access to information about their current location in the interface by identifying navigation identifiers, such as:

- Breadcrumb navigation and trails (e.g., Home > Products > Electronics > Laptops).
- Page titles that reflect the content or section. Ensure clear and unique page titles.
- Visible section headers in complex flows.
- Menus like tabs or navigation menu lists. Highlight the current navigation state in menus and tabs showing which menu item or tab is currently active.
- ARIA landmarks or headings for screen reader users. Use semantic structure and ARIA landmarks to support assistive technology.



Navigation identifiers are common UI elements presented consistently in the layout informing a clear information architecture that allow users to identify the current location. For instance, the user checks a title and recognizing the highlighted items on a navigation control like a breadcrumb, tab line, or menu. Be mindful that long series of navigation steps may confuse users with a short attention span. The goal is to allow users with memory, attention, or learning challenges to benefit from clear indicators of where they are and how to move back or forward. Screen reader users also rely on page titles, headings, and landmarks to understand their position within a larger structure.

Labels for links, buttons, and menus also contribute to wayfinding and predictability.

Links and Buttons: Change in Context

The text that triggers the transfer must have a clear purpose for the user, and the target must have a clear reference to the trigger.

The WCAG 2.4.4 Link Purpose (In context) (A) and 2.4.9 Link Purpose (Link Only) (AAA) ensure that the purpose of every link is clear to users without needing additional context, especially when the link is removed from surrounding content. At level A (2.4.4), the link's function must be understandable either from the link text itself or from its surrounding context, helping users decide whether to follow it. At level AAA (2.4.9), the link text alone must clearly convey its purpose, without relying on adjacent content. The goal is to provide users links that stand on its own and is understandable even without surrounding content—supporting independent, efficient navigation for all users. These criteria also support users navigating with screen readers, those with cognitive impairments, and anyone who benefits from clear, predictable navigation.

Sometimes this link can transport the user to an external application or site, a pattern commonly informed with an icon next to the link to inform the user beforehand where he will be redirected.

The target can be a new page, page section, pop-up, dialog, or panel. Meaningful text is critical to blind users who manage expectations based on what they hear while performing various navigation strategies like headings, links, buttons, or lists. When possible, match the title or header of the destination with the label description. The text on both ends should closely match each other to reassure the user about a navigation choice.

Search capabilities offer another way for users to find what they are looking for. The success criteria 2.4.5 Multiple ways from WCAG, recommends



offering more than one way to locate a page within a set of pages, unless the page is the result of, or a step in, the process.

Still in this context, just like navigational UI elements have to be provided at the same place across different pages of the same application, a help mechanism should also follow this requirement. According to WCAG 3.2.6 Consistent Help, this feature should be offered in the same order among the screens or pages of the application.

Link and buttons share similarities but serve distinct purposes and behaviors in user interfaces. Links are typically used to navigate between pages or sections, whereas buttons are intended to trigger actions such as submitting a form, opening a dialog, or performing a task within the same page. Visually and semantically differentiating them helps users, including those using assistive technologies, understand what to expect when interacting with each element, ensuring both clarity and accessibility.

Examples

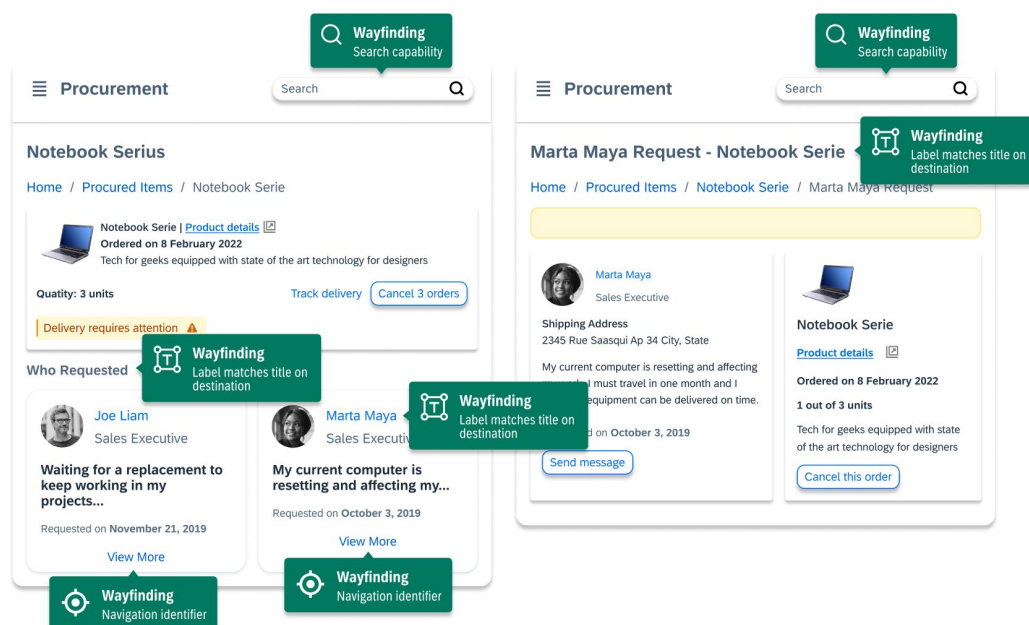


Figure 163: Wayfinding and Orientation annotations used on procurement pages. The page on the right is opened by activating “View More” buttons. The Title/Label/Breadcrumb Step of the right page is composed of respective label/name elements of the previous page, meeting expectations for orientation. In both pages, search capability is kept.



Shared Benefits



Temporary Disabilities

Everyone Benefits

Developer Notes

Discuss with the designer the workflows and the controls to be used for that. Double-check in review cycles if navigation targets are named appropriately accordingly to the labels of the navigation triggers.

Make sure that backward navigation really leads you to the last position in the workflow or navigation step.

References

WCAG 2.2 Reference

[2.4.4 Link Purpose \(In Context\)](#) (A)

[2.4.9 Link Purpose \(Link Only\)](#) (AAA)

[2.4.8 Location](#) (AAA)

[3.2.3 Consistent Navigation](#) (AA)

[3.2.6 Consistent Help](#) (A)

SAP Product Standard Accessibility 4.3

[ACC 257 - Purpose and Target of a Reference](#)

[ACC 267 - Consistent Navigation and Help](#)



Semantic

Semantic is both visual and programmatic—support comprehension and orientation, which are core cognitive tasks. Semantic is meaning - the UI element means what it is supposed to - and meaning refers to the significance, purpose, or definition of something, often conveyed through language, symbols, or actions. This overlap between perception, cognition, and semantics in accessibility covers Visual semantic cues (like color, icons, spacing), Text comprehension (clarity, reading complexity), and Screen reader semantics (roles, properties, landmarks).

Variants

Semantic Strategies

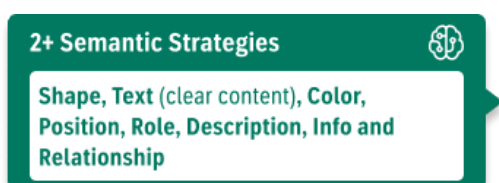


Figure 164: Use this annotation to provide a reassurance that semantics are respected using different methods: Text, Icon, Color, Role, Description, Clear Content, Pattern, Look and behave

About Semantic

This annotation was called ‘Sensory’ and located in the visual experience in the first edition of these Design Tools Guidelines. The change to Semantic aims to create a direct relationship of information and perception not only visually, but also consistent to various assistive tools. Although Semantic can be based on relevant visual cues like color-coding status icons or conveying information via shape, the rationale to move the annotation to the Cognitive Experience checklist is grounded in semantic meaning, structure, and interpretability converge. Screen reader roles or language comprehension go beyond visual perception and get into cognitive territory.

Semantics is important for sighted and blind people. Those who are color blind and those who are neurodivergent rely on on visual cues to perceive semantics (color, icon, shape, text, position). Screen reading users rely roles and properties cues. Consistency is what all of them expect to find - consistency of meaning, content, and interactions.

We simplified the annotation on the design and include declarations of how semantics can be achieved in the design. This broad the reach of

WCAG standards by including the Success Criterion 1.3.1 Info and Relationships from the WCAG 2.2 to ensure roles and properties match the visual outcome. The proposal is to avoid changing roles to ensure the communication of a blind user and sighted user remains consistent. A link should look like a link and behave like a link - the same should apply to buttons.

The semantic concept relies on the ability to explore two or more strategies to trigger different senses. This is especially helpful to differentiate similar elements like statuses, items on a wizard, progress trackers, graphs, and data visualization. Add a second signifier (or more if needed) to convey messages unequivocally. Semantic are applicable to Text, Icon, Color, Role, Description, Headings structure, Clear Content, Pattern, Look, interactions and behaviors.

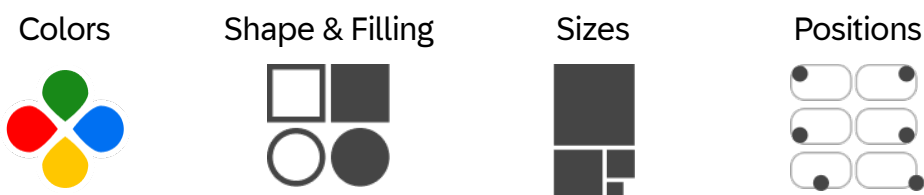
Visual Semantics

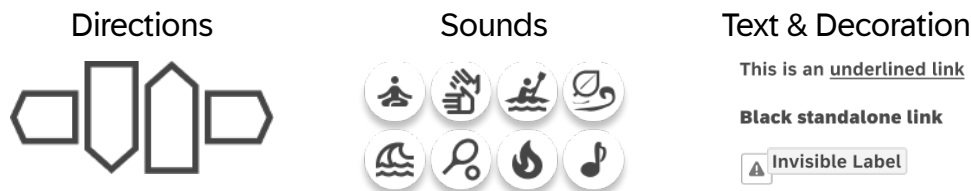
Visually safe semantics are still perceived in Greyscale - observe text, colors, shape, position, decoration, orientation, and sound. Use the semantic annotation to validate the usage of two or more channels to present information. The channels that can be explored are: text, shape, position, color.

Do not use color alone to convey a message. A color blind person, a blind users or users with low vision may not be able to understand instructions, shape, size, or location if this is not informed with text.

Stimulate visual senses by adding text, colors, shape, position, decoration, orientation, and even sound. If all the information maintains meaning and purpose on greyscale, you achieved sensory concept for elements represented by color!

If you are using colors, convert the Design to greyscale and check if the text still conveys the same information. Enhance perception and meaning by exploring various visual senses like text, colors, shape, position, decoration, orientation, and sound. Complement the visual semantic with hidden semantics and text.





Explore different strategies to trigger different senses: Colors, Shape & Filling, Size, Position.

Hidden Semantics

Besides visual presented information, semantic is also expected to be announced by Screen Readers. Every semantic content communicated visually must be accessible to screen readers. If visible text is provided (i.e. status name) it should be able to communicate the semantic by itself. If this isn't sufficient provide invisible labels, text alternative, invisible headings, descriptions.

But sometimes the visual semantic is associated to names like roles and properties for used on UI controls. The expectation in this case is to maintain a consistent semantic between visual and screen reading terminology avoiding alterations. For example, a link should look like a link and behave like a link. A link will always transport the user to a new URL page - only exception is when the user is transported to another section of the same page (href or anchor links). On the other hand, a button should look like a button and behave like a button. But sometimes a button can behave like a link, and this is ok. Submit, Save, Add, Create are common labels for buttons that transports the user to a new page.

Address how different users will give meaning and consistently understand information, instructions and interactions, visually or through screen readers. Observe how properties and roles can provide the same experience to both sighted and blind users.

Semantics for Screen Reading

Screen Reading roles and properties are meaningful with consistent pattern announced by the assistive tool.

Rule of thumb for Links vs Buttons: Buttons and links are visually distinctive. Buttons should look like buttons, and links should look like links.

Button keeps the user within the page. Link transports the user to another page (same or new browser tab).

The types of actions performed by buttons are distinctly different from the function of a link:

- Use a **button** when the action causes a change on the same page, like opening a popover or a dialog.
- Use a **link** when navigating to another page (URL).

Links are used to transport users away from the current page, but it can also be used to jump to a different section within the same page (anchor bar, Tabs).

This is semantic that can be interpreted by visual semantics (button looks like a button and a link looks like a link) and Screen reader semantics, which relies on roles to match the functionality suggested by visual design of the element and ensure that users dependent of assistive technology can accurately understand and interact with page elements.

Sighted users build expectations based on assertive visual style and proper content choices. A link should look clickable without mouse hover, which means it should be styled to inform that it is clickable. The color and the underline are two common patterns to identify links.

The role announced by the screen reader is equal to the visual cue or affordance perceived by the sighted user. A blind user creates expectation when the role is announced as they get prepared to what will happen next. Link will send them to another page. Button will keep them on the page.

Semantic on Text

Semantic text principles ensure that content is meaningful, well-structured, and accessible to all users. Providing clear, meaningful, structured and accessible content also contributes to semantics. Some examples are proper use of headings, lists, emphasis, statuses, and labels (for links and buttons). The goal is to enhance readability and comprehension while providing clear context and information.

For example, headings should follow a logical hierarchy (H1 for the main topic, followed by H2s and H3s for subsections) to convey structure, rather than relying on visual styling alone.

Lists should be correctly formatted using `<ul>`, `<ol>`, or `<dl>` elements to group related information meaningfully.

Additionally, links should have descriptive text that conveys their purpose rather than vague phrases like "Click here." By applying semantic text principles, content becomes more inclusive, reduces cognitive effort, and improves the overall user experience.



We propose five key principles to guide UX writers' format and structure text to improve readability, comprehension, and navigation, particularly for assistive technologies.

1. Proper Heading Hierarchy

Use headings (<h1> to <h6>) to create a clear structure, ensuring logical flow. The <h1> should represent the main topic, followed by <h2> for sections, <h3> for subsections, and so on. Avoid skipping heading levels (e.g., jumping from <h2> to <h4>), as it disrupts assistive technology navigation.

2. Meaningful Link Text

Links should describe their purpose (avoid click here). Repeating links should be prepared for Screen Reading using a reference for association (See Label's chapter and `ariaLabelledBy`). Avoid generic phrases that require users to rely on surrounding context to understand the link's function. Use an appropriate aria element to provide clear label or description like title attributes if extra clarification is needed.

3. Emphasis and Meaning, Not Just Style

Use for important content and for emphasis, rather than relying solely on bold or italic styling. Avoid using all caps for emphasis, as it can be harder to read and may be misinterpreted by screen readers. Use meaningful status names (labels). If color can't be perceived or translated to statuses categories, the label alone will be able to convey the semantic.

4. Avoiding Ambiguity and Redundant Text

Keep text concise and to the point, avoiding unnecessary jargon. Ensure that error messages and instructions are direct and actionable (e.g., "Enter a valid email address" instead of "Invalid input").

Identify Purpose

lastly, the WCAG 1.3.6 Identify Purpose (AAA) bridges the gap between what something looks like and what it is meant to do, helping designers and developers build interfaces that can adapt meaningfully to diverse user needs. This requirement suggests that users adapt the interface to their needs, making the purpose of elements clear to assistive technologies so that common user interface components and icons are understood semantically — symbol systems, simplified language, or personalized UI tools. This matter because:

- Helps users who rely on symbol-based assistive technologies, cognitive support tools, or custom UI adaptations.



- Improves predictability, especially for users with cognitive disabilities or low digital literacy.
- Enables assistive technologies to personalize the interface, such as replacing text with symbols or grouping functions by type.

Examples

Visual Semantic

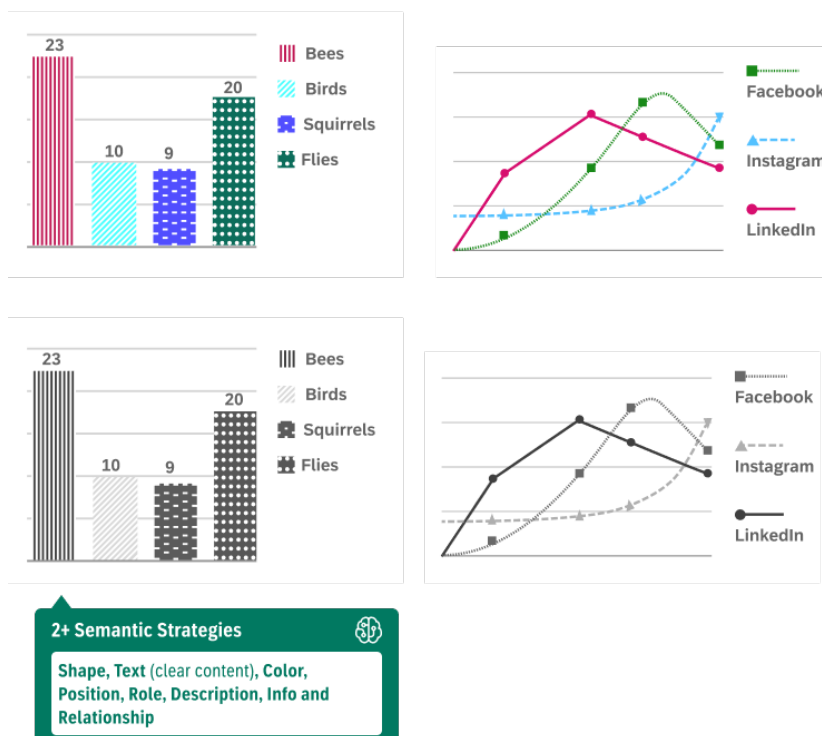


Figure 165: Data visualization with treatment to differentiate data not only with colors, but line types and patterns to meet the needs of blind color users.

Label

Value

✖ The value should contain numbers only

Label

Value

✖ The value should contain numbers only

2+ Semantic Strategies

Shape, Text (clear content), Color, Position, Role, Description, Info and Relationship

Figure 166: Error message treatment to inform an error has occurred by showing an icon, beside the red color.

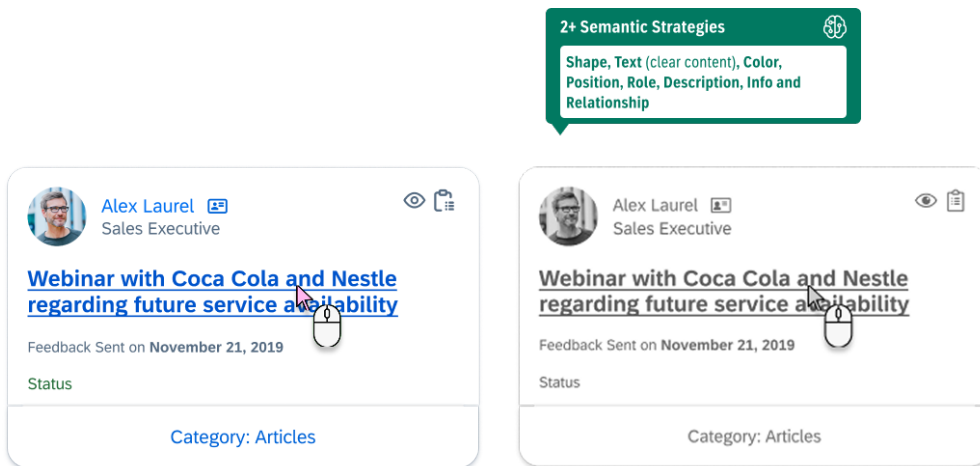


Figure 167: Link is still perceived because the text is not only blue, it is also underlined.

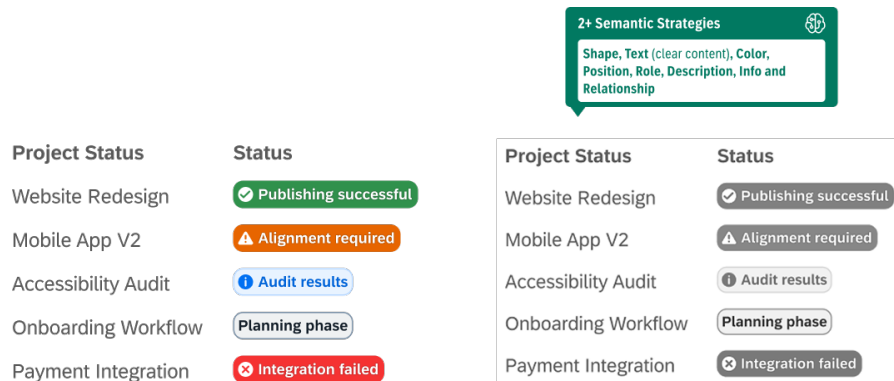


Figure 168: Statuses are recognized if color is not available by their labels or icons

Semantics are used to inform something on:

- Colors + Labels + Icons + message (status of the system or element)
- Components (links vs buttons, tags and object status, strip messages, dialog message)
- Font size on Heading to inform hierarchy

Color combinations to avoid:

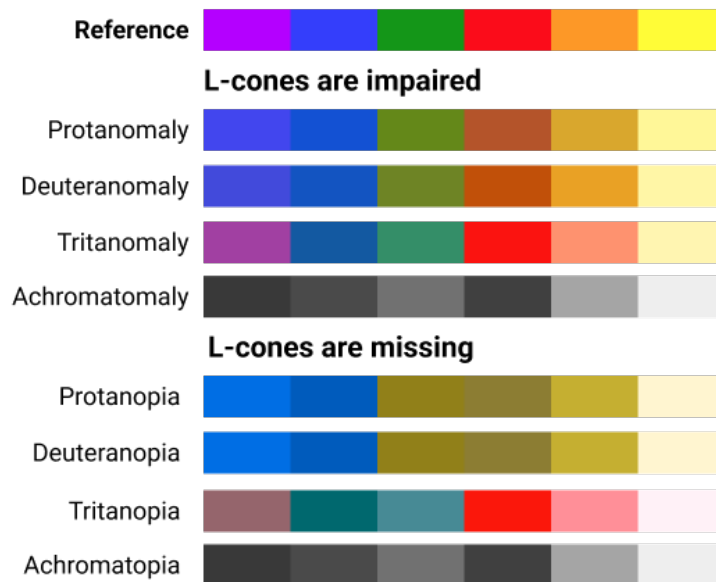


Figure 169: The sensory concept supports design decisions where colors are used to represent something like a state. Make informed decisions about combining colors. Investigate how a person with protanopia or deuteranopia sees green and red.

Avoid problematic color combinations when possible:

- Green & Black
- Green & Red
- Green & Grey
- Green & Brown
- Green & Blue
- Blue & Gray
- Blue & Purple

Shared Benefits



Temporary Disabilities

Everyone Benefits

Developer Notes

Check with the designer in the review cycle that the semantics for a respective UI part are expressed visually and non-visually in multiple ways using different technical attributes for color, text, relationships etc..

References

WCAG 2.2 Reference

[1.3.1 Info and Relationships](#) (A)

[1.3.3 Sensory Characteristics](#) (A)

[1.3.6 Identify Purpose](#) (AAA)

SAP Product Standard Accessibility 4.3

[ACC 256 - Two-Senses Concept](#)



Error Handling

Prevent errors from happening. When errors occur, clearly identify them. Allow users to review, correct, or cancel inputs so that they can avoid or fix errors. Associate an error to its location. To handle errors, use sufficient descriptions so the user can understand what happened and make suggestions to fix it.

Variants

Error Prevention

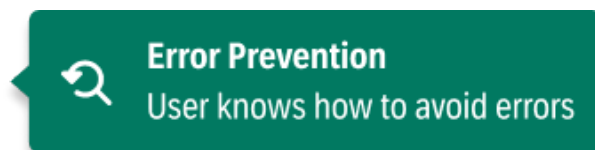


Figure 170: Error prevention helps users with information to get things right from the start.

Error Identification and Recovery

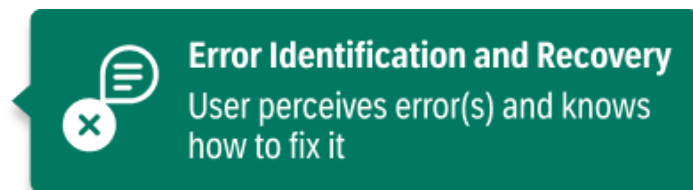


Figure 171: Error identification and recovery helps users to recover from mistakes.

About Error Handling

Suppose a user's input implies legal obligations, involves a financial transaction, or results in data being lost or modified in data repository systems. In such cases, the application must ensure that the user can check, correct, or cancel their actions.

Error Prevention

Preventing errors from happening should be a designer's primary objective when creating forms. The interface must describe how a form field should be filled out, and input fields should refuse incorrect values and redirect users to enter correct values.

Error Identification



Error prevention helps user to avoid errors. Error identification and recovery allows the user to fix errors if they occur.

When error prevention is not possible, the application must allow the user to review, correct, or cancel an input. The designer should anticipate such scenarios and address the following:

- When and how the error will be appeared.
- How the error message will address the problem to enable user to handle the error.
- Along with a detailed description of the error, what resources will be offered to the user to fix it.

In case of input errors, the location of the error has to be indicated, a description of the error has to be provided, and suggestions for correcting the error have to be provided. Efficient error handling saves vast amounts of effort for:

- Finding errors
- Navigating to the element causing the error
- Resolving the error by using pre-selected values or hints.

Context-specific suggestions presented to the user next to the form field enhance error identification awareness and enables the user to recover faster from the error. This strategy helps users of screen readers, as it provides quick access to the specific information related to the input error.

Confirmation messages are commonly used to notify users when an error is resolved, or the form was successfully submitted. This is necessary when error validation cannot be implemented while the form is being filled out.



Examples

The figure consists of two screenshots of a web application interface titled "My Tour Settings".

The top screenshot shows the "Profile" tab. It contains several input fields: "Name" (with the value "Kim"), "Tour Type" (with a search icon and the text "Search"), "Tour Distance" (with the value "abc" and a unit "Km"), "Time Length", "Meeting Point" (with the value "Rue de la Maison"), "City", and "Zip Code". There are "Save" and "Cancel" buttons at the bottom right. An orange border highlights the "Tour Distance" field, and a yellow warning icon with the text "Only numbers accepted" is shown below it. A green callout box with a magnifying glass icon points to the "Tour Distance" field, containing the text "Error Prevention" and "Provides opportunities to correct information informing 'Unit'".

The bottom screenshot shows the same "My Tour Settings" form, but with the "Confirm information before publishing" section expanded. It displays a summary of the user's input: Name (Mary Liam), Tour Type (Cultural), Tour Distance (10 Km), Time Length (7 hours), Meeting Point (234 Rue de la Maison, Paris, Yt67), Bring with you (Comfortable shoes), Get to the meeting point (Bus 345, Metro Line A), and Accepted payments (Credit Cards). There are "Publish" and "Edit" buttons at the bottom right. A green callout box with a magnifying glass icon points to the "Confirm information before publishing" section, containing the text "Error Prevention" and "User can review and correct mistakes before publishing".

Figure 172: Error Prevention annotation used to indicate respective areas or functionality in the application design

Personal

Information

Street / Number: *
South Gettysburg Avenue 369

Postal Code / City: *
AAA Heidelberg

Country: *
Germany

Contact

Email: *

All 2 1 1 1

Personal, Information

- Enter a valid postal code. Postal Code
- Special shipping conditions apply. Country

Personal, Contact

- Enter an email address. Email
- User name JMILLER was assigned. User

2 Save Cancel

Error Identification & Recovery
Error type and description with reference to error location

Figure 173: Error Identification & Recovery annotation used to indicate respective applied patterns in the application design

Shared Benefits



Temporary Disabilities

Everyone Benefits

Developer Notes

All developers should provide mechanisms to indicate and to correct errors if they occur by using respective controls or patterns from their framework inventories.

References

WCAG 2.2 Reference

[1.3.1 Info and Relationships](#) (A)

[3.1.3 Unusual Words](#) (AAA)

[3.1.4 Abbreviations](#) (AAA)

[3.1.5 Reading Level](#) (AAA)

[3.3.1 Error Identification](#) (A)

[3.3.3 Error Suggestion](#) (AA)

[3.3.4 Error Prevention \(Legal, Financial, Data\)](#) (AA)

SAP Product Standard Accessibility 4.3

[ACC 257 - Purpose and Target of a Reference](#)

[ACC 264 - Interoperability with Assistive Technologies](#)

[ACC 267 - Consistent Navigation and Help](#)

Motion Content

Animations can be stopped or disabled. Video can be paused or stopped. Automatically played audio content can be muted. The volume of the automatically played content can be changed independently of the system volume. Generally, avoid flickering and flashing content.

Variants

Autoplay & Motion-Based Content



Figure 174: Use to inform content affected by autoplay or motion based (moving or updating content). Inform also where user can find controls or settings to control motion.

About Motion Content

Animation, video, and audio streams attract the user's attention. But content that moves or auto-updates can be a barrier to anyone who has trouble reading static text. It also affects users who have difficulty tracking moving objects and users of screen readers. Users of screen readers find it difficult to listen to speech output if other audio files are played automatically at the same time. Users with hearing impairments are also distracted by content played automatically.

All users must be able to pause, stop, or hide, videos, animations, and audio. Any automatic audio or video content must allow users to control and change the volume independent of the system volume.

Common examples include motion pictures, synchronized media presentations, animations, real-time games, and scrolling stock tickers. Use the annotation 'Multimedia' to point to where users can find the controls to pause, stop, hide moving content.

Certain groups, particularly those with attention deficit disorders, find blinking or flashing content distracting, making it difficult for them to concentrate on other parts of the page. Therefore, avoid flickering and flashing content. Ensure that the application has no flickering content that flashes more than three times per second. If your component or screen uses a content in motion, identify it as "Flicker safe" using the annotation to inform that content is secure for sensitive users with epilepsy or seizures.



A refined terminology for UX & Accessibility Guidelines related to motion includes these three groups: Autoplay & Motion-Based Content, Moving & Updating Content, and High-Risk Motion Content.

Autoplay & Motion-Based Content:

- "Autoplaying Media" – Covers videos, GIFs, and animations that start playing without user interaction.
- "Auto-advancing Content" – For carousels, slideshows, or other elements that change automatically.
- "Motion-Based Content" – Includes parallax effects, animated scrolling, and any UI elements that respond to movement.

Moving & Updating Content:

- "Moving Content" – Covers auto-scrolling text, ticker messages, and elements that shift without user interaction.
- "Auto-Updating Content" – Refers to changing dashboards, live sports scores, or stock tickers that refresh dynamically.

High-Risk Motion Content (Accessibility Considerations):

- "Flickering Content" – Any flashing, blinking, or strobing elements (e.g., fast animations, certain video effects). Should follow WCAG's three flashes or below threshold to avoid seizures or discomfort.
- "Looping Motion Content" – Covers endlessly repeating animations or videos that might be distracting or overwhelming.



Examples

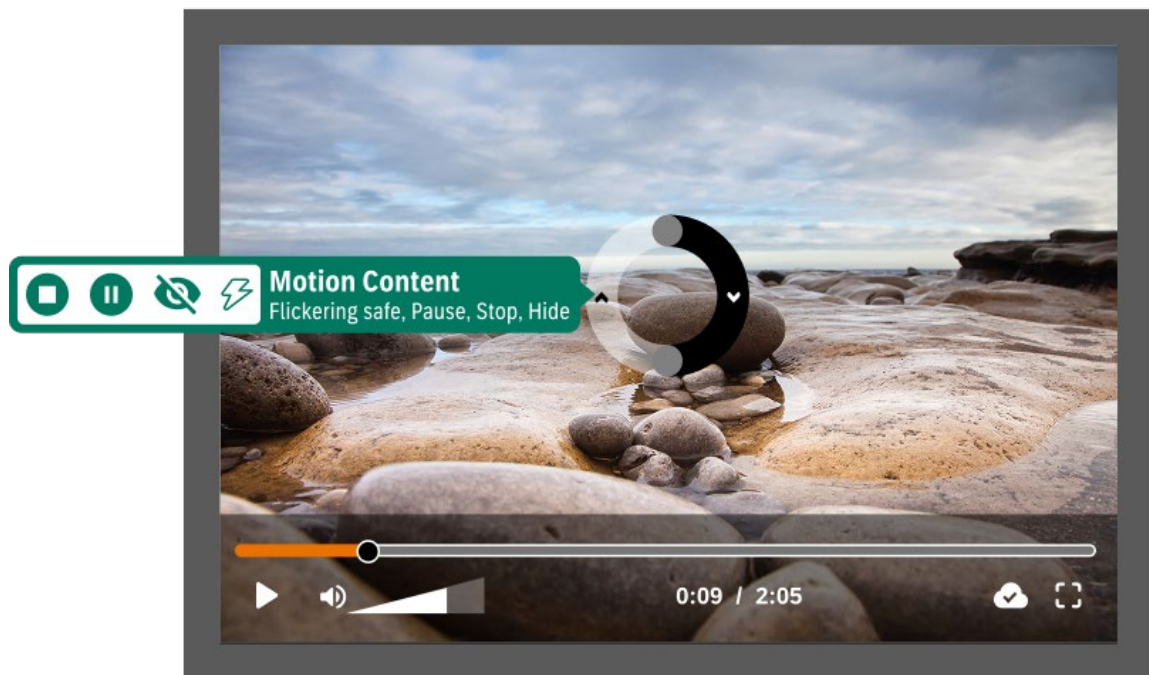


Figure 175: Volume of automatically played audio content can be changed independently of the system volume. Videos can be paused or stopped. Animated controls must be created to be flickering safe, like busy indicators.

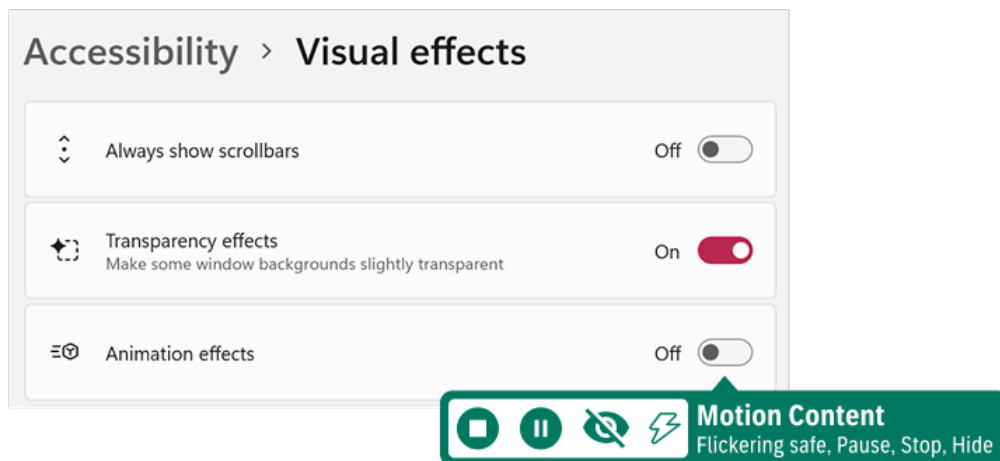


Figure 176: Animations can be hidden or disabled



Shared Benefits



Without Vision

Screen Reading

Observing autoplay, motion, and dynamic content during design helps screen reader users by preventing unexpected changes that can disrupt navigation, shift focus, or overload announcements—ensuring a stable, predictable experience. It also allows designers to provide proper pause controls, ARIA live regions, or alternatives, making content updates accessible without overwhelming or disorienting users relying on assistive tech.

Developer Notes

Ensure by providing respective options that:

- Animations can be stopped or disabled
- Video can be paused or stopped
- Automatically played audio content can be muted
- The volume of the automatically played content can be changed independently of the system volume

Generally, avoid flickering and flashing content

References

WCAG 2.2 Reference

[2.2.2 Pause, Stop, Hide](#) (A)

[2.3.1 Three Flashes or Below Threshold](#) (A)

[2.3.2 Three Flashes](#) (AAA)

SAP Product Standard Accessibility 4.3

[ACC 279 - Animation, Audio and Video Control](#)



Time Limit

Provide options to the user to disable time limits, customize the length of time limits, or request to extend the time limit before it expires. Indicate areas on the screen that are refreshed automatically or because of user interaction.

Variants

Time Limit - Timeout Progress



Figure 177: Informs the UI prepares users for system events tied to timing (e.g., session expiration or auto-logout). A message confirms data and progress will be saved.

Time Limit - Timeout Info

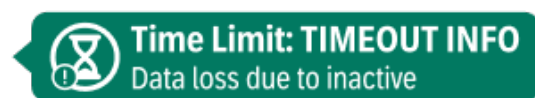


Figure 178: Informs the UI prepares users for system events tied to timing. A message reinforces the risk of data loss or progress reset due to inactivity or predefined time limits.

Time Limit - Timeout Interruption Expected



Figure 179: Informs the UI prepares users for system events tied to timing (e.g., session expiration or auto-logout). A message reinforces the risk of data loss or progress reset due to inactivity or predefined time limits.

Time Limit – Configuration



Figure 180: Use this annotation to indicate where users can adjust and manage time to be turned off, adjusted, or extended.



Time Limit – Content Refresh Area



Figure 181: Informs the area in the UI that will be affected by a refresh upon time limit

Time Limit – Visual Status Update



Figure 182: System Status Awareness for Background Processes ensure users are aware of ongoing background processes and can access or perceive system status at any time.

About Time Limit

Time-based interactions—automatic updates, periodic processes, and session expirations—can interrupt tasks, erase work, and confuse users. The Time Limits annotation helps designers surface these risks early, giving users predictability, control, and protection.

According to WCAG 2.2.3 No Timing (AAA), content must not rely on time limits to function, unless it's essential (like live events). Users should be able to complete tasks at their own pace. But sometimes this is not possible, and this is what this chapter covers.

Automatic updates, session expirations, auto-logout, form timeout, dashboard refresh, expiring access to a task, and timeouts can introduce unexpected barriers, disrupt user tasks, or even result in data loss. Any process that fires after a set period counts as a time limit. Such triggers can be major barriers for people who need extra time to read, think, navigate, or act.

A poorly designed time constraint may cause confusion, frustration, lost work or loss of data. A thoughtful and well-considered time limit design shields users from disruption and protects users from disruption supporting a smoother, more accessible experience.

Designers should aim to avoid time-dependent features whenever possible or minimize the use of time-dependent features if possible. When time constraints are necessary, interfaces must offer users clear control and predictability. If unavoidable, give users control in this order of preference:

- a. Disable the time limit entirely (most inclusive).



- b. Customize the duration of the time limit. Configure the length to suit individual needs.
- c. Extend it on demand via a clear prompt before it expires. Request more time before the limit expires.

The Time Limits annotation supports inclusive design by helping teams plan for four critical aspects:

- **Timeouts**
- **Content Refresh**
- **System Status Updates**
- **Configuration Options**

These dimensions ensure users are informed, protected from unexpected behaviors, and empowered to complete tasks at their own pace. Disabling time limits is always preferred. If that's not possible, customizing them or offering time extensions is essential to support diverse user needs and reduce cognitive load.

When users understand and control the timing of system behaviors, they avoid unwelcome surprises and interact with greater confidence—leading to more inclusive, frustration-free experiences.

These annotations support WCAG 2.2 compliance and empower users to complete tasks without pressure or surprises enabling a more inclusive, frustration-free experiences.

Timeouts

Composed of three scenarios they may or may not warn users about timeout

1. Data entered or progress tracked - when progress and input data is saved and tracked when time is out.
2. Data loss due to inactivity - when progress or input might be lost due to inactivity.
3. Interruption Expected - anticipate and inform about automatic refreshes or system updates.

The WACG 2.2.6: Timeouts determines that users are warned of any user inactivity that could cause data loss, unless the data is preserved for more than 20 hours. The Timeout annotation reminder designers to provide users with alerts about possible data loss due to inactivity, upcoming or occurred interruptions, and also reassurance message that data and progress was saved.

Content Refresh

Informing users about content that will refresh after a time limit is a requirement covered by WCAG 2.1. The relevant WCAG 2.2.1: Timing Adjustable defines that content updates after a time limit (e.g., auto-refresh), users must be able to turn off, adjust, or extend that time unless it's essential. This definition is relevant to inform users which area will refresh and prepare them to act within the time limit or allows support for adjusting timing.

The WCAG 2.2.2: Pause, Stop, Hide also supports this requirement. For moving, blinking, scrolling, or auto-updating information, users must be able to pause, stop, or hide the content unless it's essential. If the refreshed content is auto-updating, like a dashboard or notification panel, controls must be offered to manage the update behavior - see Time Limits Configuration.

System Status Updates

Communicate clearly when system behaviors are time-triggered, helping users anticipate outcomes and maintain focus. When users are informed and in control of timing, they avoid surprises and complete tasks more confidently. Use messages with precise information about timeout outcomes.

Communicating system status clearly becomes essential when time-based behaviors are in play—like session timeouts, auto-logout, or delayed content refresh. Users need to know what is happening, how much time they have left, and what action (if any) is required.

But Status Update also includes visual (and also verbal) cues that show system status in real time, using consistent indicators such as countdowns, progress indicators, alert messages, spinner, progress bar, badge. They represent a start with the process and stops when complete. The design should focus on clarity and provide control to the user with plain, concise messages (“Uploading 45 %...”) and, where possible, buttons to pause, resume, or cancel. This helps reduce anxiety, supports informed decisions, and gives users a sense of control.

Assistive Tech require ARIA live region or role="status" / aria-busy="true" so screen readers announce updates without shifting focus. Keyboard Access should be provided to reach the status info and dismiss controls reachable with the keyboard. This requirement is in alignment with WCAG 4.1.3 Status Messages to announce updates programmatically,



3.2.2 On Input to warn the user before auto-refreshing on user action, and 2.2.2 Pause, Stop, Hide to let users halt distracting auto-updates.

Clear status feedback, combined with timeout and refresh annotations, ensures a smoother, more accessible experience across the board.

Configuration Options

Gives users flexibility over time limits. While information is good to keep users aware, they prefer when the time limit can be extended and controlled. Designers address time outs by providing options to the user to disable time limits (switch off), customize the length of time limits by requesting more time before a time limit occurs, or extend the time before the time limit expires. A time limit configuration helps:

- Refresh content after a certain amount of time
- Signal the time to log off from an application
- Keep a network connection to a server online (keep it alive)
- Define time limit to work on a task

The WACG 2.2.1: Timing Adjustable determines that for each time limit that is set by the content, at least one of the following is true: the user can turn off, adjust, or extend it. The Time Limit Configuration annotation reminds designers to provide user with the ability to control session timeouts, task limits, or inactivity-based events



Examples

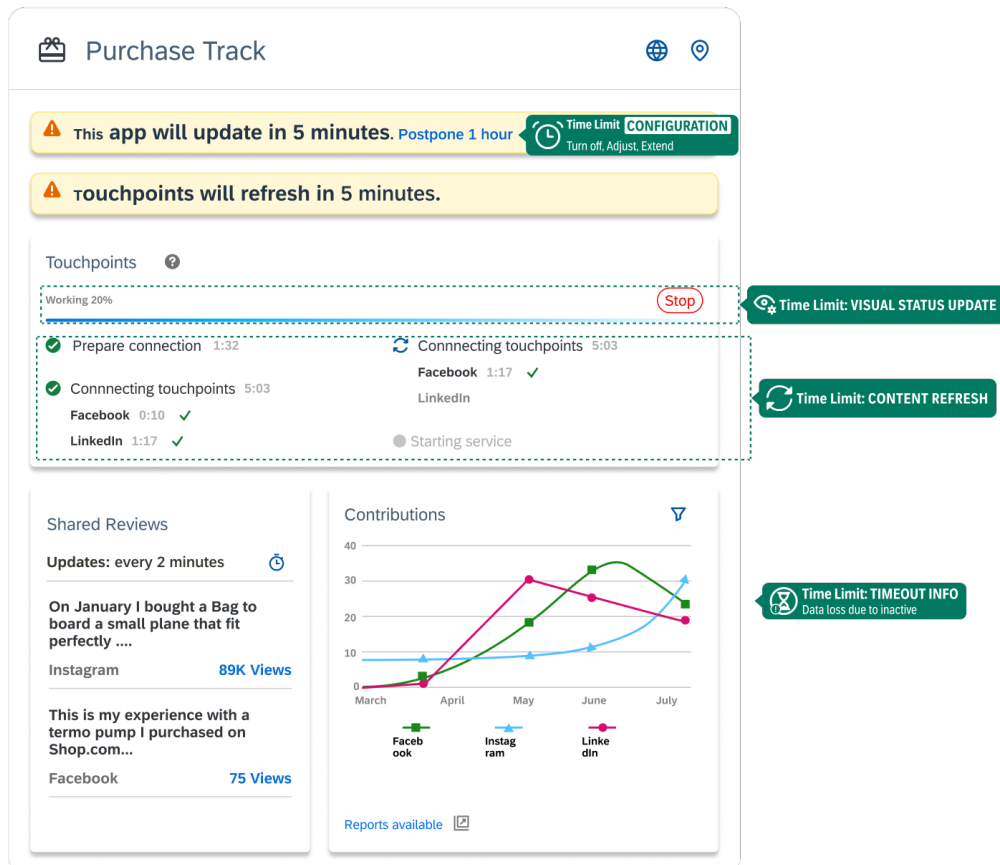


Figure 183: Multiple UI elements that update, refresh or change based on time are annotated with time limit annotations.



Figure 184: A progress bar annotated with the Timeout - Data and Progress tracked annotation

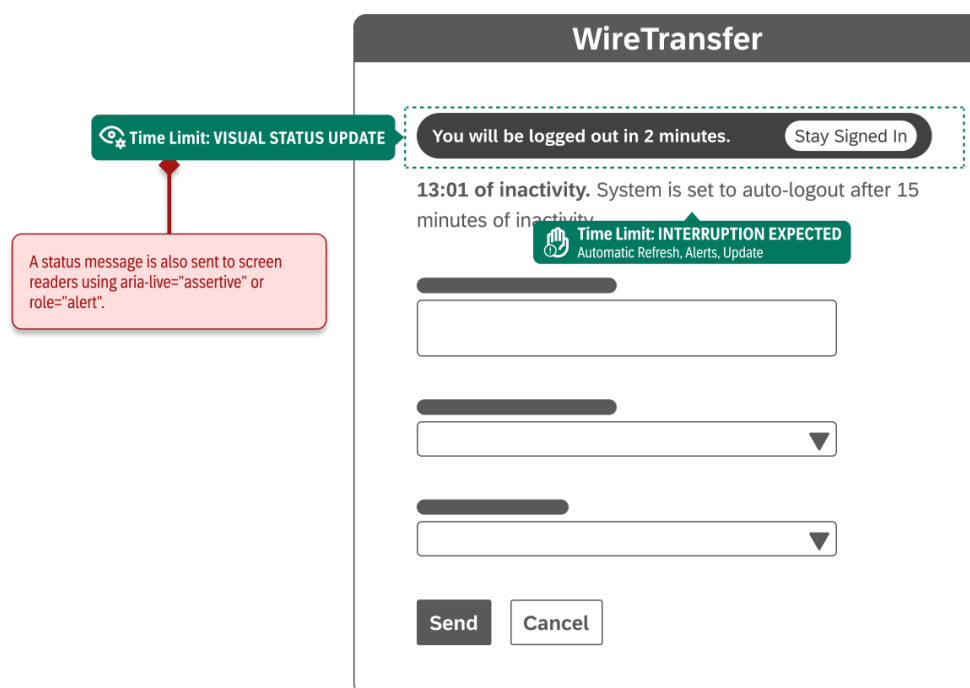


Figure 185: A dialog for a wire transfer that will automatically log the user out in 2 minutes annotated with the Timeout - Interruption expected annotation and the Time Limit: Visual Status Update

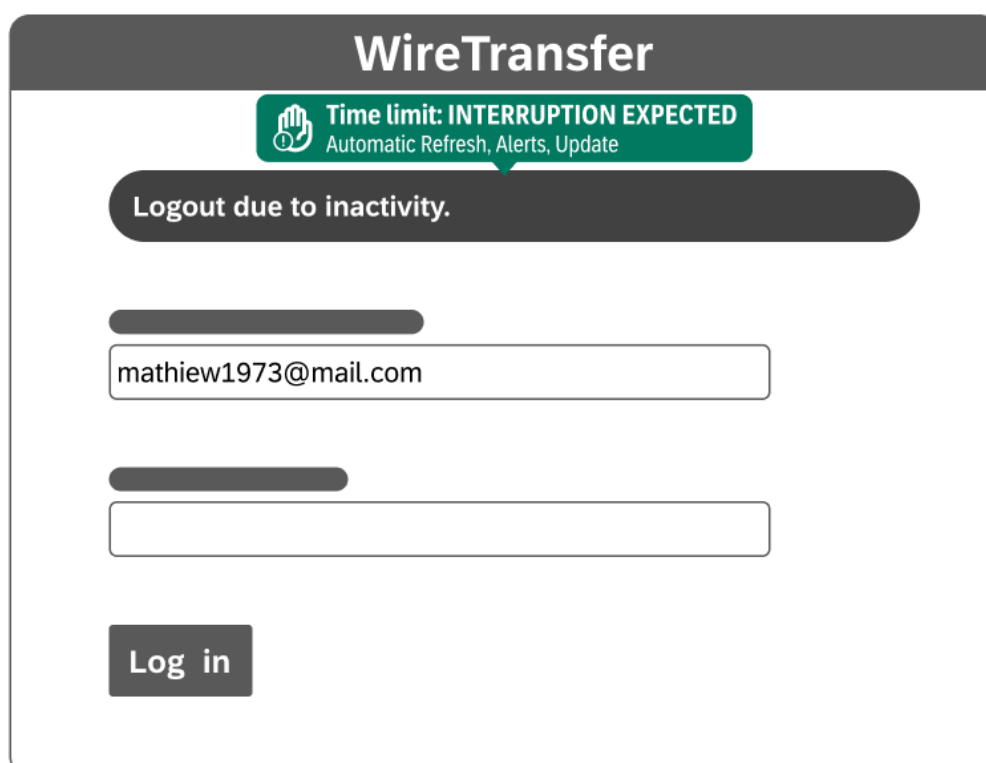


Figure 186: A dialog for a wire transfer that logs the user out due to inactivity annotated with the Timeout - Interruption expected annotation

Shared Benefits



Without Vision

Screen Reading

For screen reader users, updating regions should use ARIA live regions (e.g., `aria-live="polite"` or `assertive`) to notify without disrupting interaction. This ties into WCAG 4.1.3: Status Messages.



Limited Mobility

Interactions

Time limits can exclude users with limited mobility who need more time to navigate or complete tasks due to slower or assistive input methods. Giving options to extend or disable time limits reduces stress, prevents errors, and supports equal access.

Developer Notes

Ensure by providing respective framework controls that there are options to:

- disable time limits
- customize the length of time limits
- request to extend the time limit before it expires

Indicate clearly areas on the screen that are refreshed automatically or because of user interaction.

References

WCAG 2.2 Reference

[2.2.1 Timing Adjustable](#) (A)

[2.2.2 Pause, Stop, Hide](#) (A)

[2.2.3 No Timing](#) (AAA)

[2.2.6 Timeouts](#) (AAA)



[3.2.2 On Input \(A\)](#)

[4.1.3 Status Messages \(AA\)](#)

SAP Product Standard Accessibility 4.3

[ACC 257 - Purpose and Target of a Reference](#)

[ACC 267 - Consistent Navigation and Help](#)

[ACC 264 - Interoperability with Assistive Technologies](#)

[ACC 278 - Time Limits](#)



Multiple Ways

Provide users with multiple clear ways to find and return to content—such as search, navigation menus, breadcrumbs, or contextual links—so they can move forward with confidence and easily backtrack without losing their place.

Variants

Multiple Ways

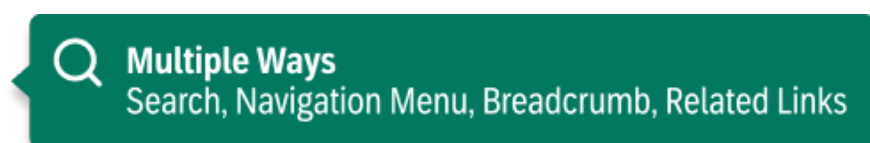


Figure 187: Use this to indicate multiple ways for accessing content by navigation

About Multiple Ways

Not everyone navigates content in the same way. Offer more than one way - at least two different options - to enable users to locate and reach a page within a set of pages - or the same content - to fulfill the multiple ways requirement. This is not applicable where the page or content is the result of, or a step in, a process.

The WCAG 2.4.5 Multiple Ways requirement ensures users aren't stuck in one path to find content. Offering alternative navigation methods helps people discover and recover information more distributed across multiple pages.

Providing users Multiple Ways to access the pages in a product comes in alignment with WCAG 3.2.3 Consistent Navigation requirement. Repeating UI elements and features like search, navigation menus, site maps, and breadcrumbs provides a predictable and consistent strategy for users to get to the content they are looking for using the ways that most meet their needs. Users may find one technique easier or more comprehensible to use than another

The design specification is likely to live in the product level so that such options are found consistently applied across pages of the same product. Here's what designers should observe and implement:

1. Include a Search Function (Where Relevant): If the product has a large set of content (like help docs, product listings, FAQs), is search available? Designers are responsible to design a prominent and

accessible search field as an alternate method to locate specific content. Category filters are also helpful as an alternate way to find content.

2. Provide Navigation Menus or Site Maps: Is there a consistent global or section-level navigation system? Designer should ensure users can browse through menus, and that these menus provide access to all major areas of the interface.
3. Use Lists of Related Content (e.g., "See Also", "More Like This"): Are related or recently viewed items presented? Designers should find patterns that suggest additional paths to content users may find helpful.
4. Provide Clear Hierarchical Navigation (Breadcrumbs): Are users able to understand where they are and jump to previous levels? Designers should include breadcrumbs in your design for multi-level page structures.
5. Facilitate Filtering and Sorting: Can users control how they access content (e.g., filters for job listings)? Designers should provide flexible browsing tools that support diverse information-seeking behaviors.

Observe flags that could cause accessibility violations:

- Design lacks Search or secondary navigation.
- Content is deeply nested and inaccessible.
- Content that leads to more content on a non-linear exploration.



Examples

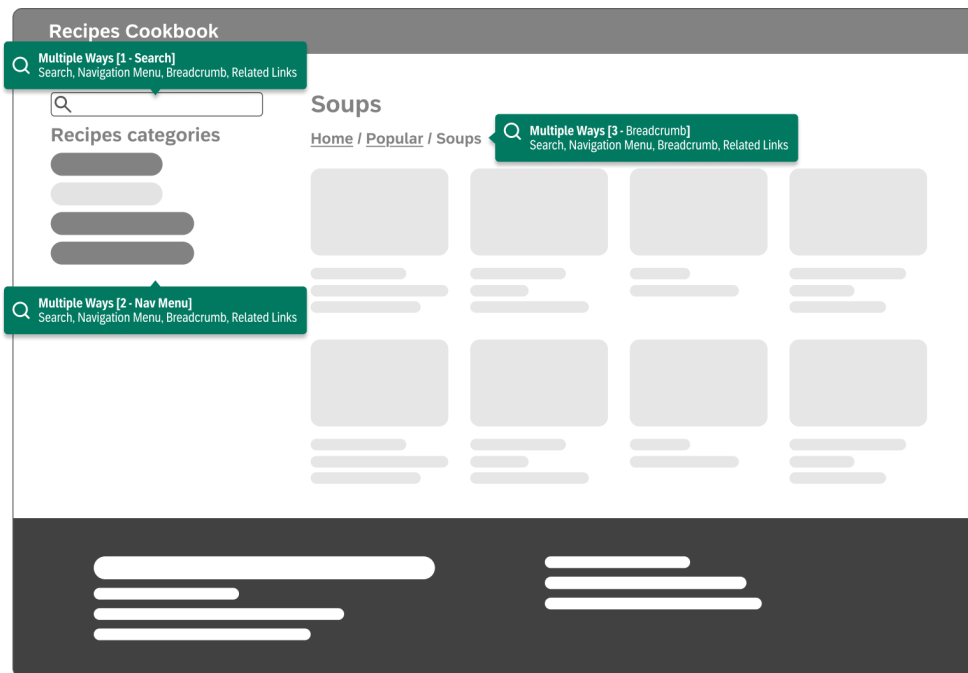


Figure 188: A cooking webpage with three different ways to find recipes annotated with the multiple ways annotation

Shared Benefits



Limited Mobility

Interactions

Users with limited dexterity, switch devices, or speech navigation may struggle to navigate through nested menus or perform precise movements. Multiple Ways helps them because it enables shortcuts to content (e.g., through search or quick links), reducing the need for repeated or fine motor interactions. It also reduces fatigue by avoiding long sequences of navigation or scrolling. And finally, it allows users to choose the most accessible navigation method for their setup (e.g., skipping menus in favor of filtered tables or voice-based navigation).



Vision and Color Limitation

Visuals

Users who rely on screen readers or magnification tools may not perceive visual menus or may get lost in long pages. This requirement helps users because search or landmark navigation allows users to jump directly to content without navigating visual menus.

- Consistent heading structure and landmark roles help screen reader users skim and locate content more efficiently.
- For magnification users, avoiding excess scrolling through alternative paths (e.g., filters, in-page links) reduces cognitive and physical strain.

Developer Notes

Provide users using established UI patterns to find and return to content—such as search, navigation menus, breadcrumbs, or contextual links—so they can move forward with confidence and easily backtrack without losing their place.

References

WCAG 2.2 Reference

[2.4.5 Multiple Ways](#) (AA)

[3.2.3 Consistent Navigation](#) (AA)

SAP Product Standard Accessibility 4.3

[ACC 268 - Multiple Ways to Find Content](#)



Redundant Entry

Reuse previously entered information by prefill or auto-populate input

Variants

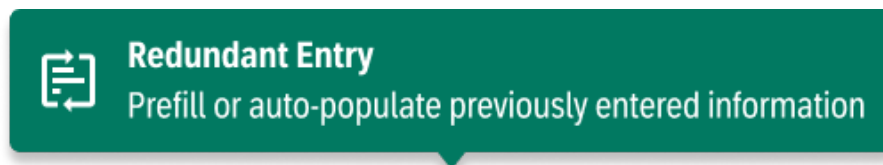


Figure 189: Reuse previously entered information by prefill or auto-populate input.

About Redundant Entries

If a user has already entered information earlier in a process, they shouldn't have to re-enter it unless it's essential. This requirement fulfills the WCAG 3.3.7 Redundant Entry and reminds designers to observe how to reduce the user's cognitive workload and physical burden in multi-step forms, processes, or flows (e.g., checkout, onboarding, applications).

Is the process multi-step?

- If yes, track what data is being collected at each step.
- Avoid asking for the same piece of info more than once (e.g., name, address).

Can earlier info be prefilled or referenced?

- Design forms to carry user input forward to later steps.
- Provide clear confirmation (e.g., "Is this still your shipping address?").

Can earlier info be prefilled or referenced?

- Design forms to carry user input forward to later steps.
- Provide clear confirmation (e.g., "Is this still your shipping address?").

Allow editing previously entered data

If users need to correct something, link back to the relevant step or make fields editable in a summary.

Design for memory relief

- Don't expect users to recall what they entered.
- Show previously entered values clearly and accessibly.



Examples

The image shows a login screen titled "WireTransfer". At the top, a dark grey bar contains the text "Logout due to inactivity.". Below this, there are two input fields. The first field is populated with the email address "mathiew1973@mail.com". A green callout box with a document icon and the text "Redundant Entry" points to this field, with a subtext "Prefill or auto-populate previously entered information". Below the email field is another empty input field. At the bottom left, there is a dark grey button labeled "Log in".

Figure 190: Login Screen with the E-Mail input field auto populated. That field is annotated with the redundant entry annotation

The image shows a checkout dialog box. It has two sections: "Billing Address" and "Shipping Address". The "Billing Address" section contains three rows of input fields: "Name" (John Johnson), "ZIP Code/City" (12345, Maintown), and "Street/No" (Mainstreet, 1618). The "Shipping Address" section has a checkbox labeled "same as billing address" which is checked. Below this checkbox, there are two rows of input fields: "Name" (John Johnson) and "ZIP Code/City" (12345, Maintown). A green callout box with a document icon and the text "Redundant Entry" points to the "same as billing address" checkbox, with a subtext "Prefill or auto-populate previously entered information".

Figure 191: Dialog showing how the shipping address in a checkout process is auto populated, when the option "same as billing address" is selected. This is annotated with the redundant entry annotation

Shared Benefits



Limited Mobility

Interactions

Minimize physical effort for people with limited mobility since these users have limited dexterity. Avoid re-typing the same long entries and use techniques like autofill, simple selection (checkboxes) like in “Use billing address as shipping address”, and single sign-on (SSO) or account prefill when appropriate.

Developer Notes

Prefill information as often as possible.

Use caching mechanisms of the platform to reuse previously entered information to auto-populate respective input on new pages.

References

WCAG 2.2 Reference

[3.3.7 Redundant Entry](#) (A)

SAP Product Standard Accessibility 4.3

[ACC 286 - Avoidance of Redundant Entries](#)



*Í A ηBfinōfiūōfiƏĩ xō mūū ĩ fflĩ uqĩit Lpĩ ôτ B K9b
ĩ ЯФЭЅ ĩ ĩ ũ ħxōÊ*

Zhanara Baisalova – UX Designer



Auditory Experience

Imagine all the information on the screen must be read—there's no sound to guide or support you. How much text feels like too much to follow? Are the visuals, captions and transcriptions clear enough to help you understand what's happening and what to do next without guessing or re-reading?

This chapter helps designers to address audio content to meet the needs of users who are deaf and hard of hearing.

The Inclusive Auditory Experience

Meeting the needs of people who are hard of hearing

Auditory annotations focus on improving accessibility for users who are hard of hearing by providing clear, alternative ways to access auditory information, ensuring effective communication and comprehension.

While visual and interactive experiences cover some accessibility concerns, additional considerations complement the requirements to meet the needs of users with partial or complete hearing loss. Auditory annotations help bridge these gaps by incorporating features such as captions, transcripts, visual alerts and sign language - if you are feeling fancy, you can also include haptic feedback. These elements enhance accessibility by ensuring that important audio cues, spoken content, and notifications are conveyed through alternative, perceivable means.

By integrating auditory annotations, interfaces become more inclusive, enabling users who are hard of hearing to engage with content, receive alerts, and participate in interactive experiences without relying solely on sound.



Principles

Hard of Hearing



Persona: Harold

"I have a severe hearing impairment. I struggle to acquire knowledge or follow meetings when audio alternatives are not available: captions, transcripts or sign language."

‘Disabling’ hearing loss refers to hearing loss greater than 35 decibels (dB) in the better hearing ear. To be considered disabled the person must have a permanent condition like profound hearing loss or deafness. Deaf individuals have little to no auditory perception, while those who are hard of hearing may have varying levels of hearing.

But the prevalence of hearing loss increases with age, among those older than 60 years, over 25% are affected by disabling hearing loss. These individuals may encounter various challenges when using digital products attributed to communication needs and preferences.

Common causes of hearing loss are:

- Age, Medication, Infection
- Foreign body, Fluid in the middle ear
- Perforated tympanic membrane
- Otosclerosis
- Neurologic conditions

Statistics:

- Over 430 million people worldwide have disabling hearing loss (WHO).
- About 15% of US adults report some hearing difficulty (NIDCD).
- Hearing loss affects 1 in 3 people over 65 and nearly 1 in 2 over 75.
- 12.3% of interviewed US adults had some difficulties hearing even when using a hearing aid in 2020. (CDC)



Permanent Disabilities

Users with chronic hearing loss or impairment may be associated to genetic factors, age-related hearing loss (presbycusis), or due to noise exposure, infections and illnesses, injury or trauma and medication.

Situational & Temporary Disabilities

By integrating situational and temporary disabilities into this disability group, we help design teams address challenges that might not be permanent but still impact user experience under certain conditions.

While hearing loss affects 1 in 3 people over 65 and nearly 1 in 2 over 75, temporary hearing challenges in noisy environments or infections affect many more people. These are some examples of situational and temporary disability for hearing.

- **Background noise:** A noisy environment (like a crowded restaurant or public transportation) could temporarily impair hearing, making it hard to hear conversations or instructions.
- **Temporary hearing loss:** Conditions like ear infections, colds, or loud noise exposure could lead to short-term hearing loss.
- **Fatigue or stress:** Mental fatigue or emotional stress could cause temporary difficulty processing sounds, making communication harder.



General Design Tips

1. **Captions, Transcripts, and Sign Language Interpretation:**

Audio cues like videos, podcasts, or voice instructions are often inaccessible, causing missed information. Video conferencing tools may lack real-time captions or transcription, and inaccessible social media limits participation.

- Design Tips: Provide closed captions, subtitles, and transcripts for all audio and video content. Use clear visual alerts and error indicators. Ensure video platforms support live transcription and interpretation. Promote accessible social media and community features.

2. **Real-Time Transcription for Conversations and Meetings:**

Users may feel excluded without live transcription during one-on-one or small group conversations.

- Design Tips: Support live transcription and captioning features during meetings to ensure inclusive communication.

3. **Adjustable Volume and Vibration Notifications:**

Audio-only alerts can be missed by users with hearing impairments.

- Design Tips: Incorporate vibration alerts and visible notifications alongside audio cues to ensure important updates are perceived by all users.

4. **Visual Alerts and Clear Error Signifiers:**

Relying solely on sound for warnings or errors excludes users with hearing loss.

- Design Tips: Use visual signifiers like icons, color changes, or message banners to communicate errors and warnings clearly.

5. **Text-Based Communication (Chat and Messaging):**

Many users are unaware of assistive technologies such as speech-to-text apps, limiting their communication options.

- Design Tips: Integrate robust chat and messaging features. Support speech-to-text tools that convert spoken words into text and vice versa. Provide user education about assistive technologies.



Annotations

Hearing Annotations

The checklist helps designers to remember the important aspects of accessibility that should be addressed in the design phase.



Auditory

- ☐ Caption
- ☐ Transcript



Caption

Provide synchronized captions for all prerecorded and live media containing speech or meaningful sound.

Variants

Caption as an audio media alternative



Figure 192: Informs written text alternatives for audio content (speech, sounds, etc.)

Live Caption as an audio media alternative



Figure 193: Informs caption for live audio events

About Caption

People who are Deaf or Hard of Hearing rely on visual alternatives to understand pre-recorded or live audio-based content like videos and live events. Captions are visual text alternatives and must be synchronized with the media timeline for spoken dialogue, narration, and important sounds within videos, ensuring that users can fully engage with media and audio content. Pre-recorded tutorials, promotional videos, training materials, and other recorded content are expected to include captions to ensure accessibility.

As per WCAG 2.2 1.2.2 Captions (Prerecorded) (A) all pre-recorded audio in video (synchronized media) should include captions, unless the video is already a text alternative. While captions make speech, sound effects, and audio cues visible, it can also include non-spoken words - meaningful sounds like phone rings and cheering complement the context of the audio. WCAG 2.2 1.2.4 Captions (Live) (AA) recommends providing live captions for live audio content, also for live events ensuring real-time delivery.

Live captioning



Live captions are intended to help deaf and hard of hearing people access to real-time presentations. It is used to support broadcast of synchronized media, but it's not required on two-way multimedia calls. It requires the identification of who is speaking, inform sound effects and other significant audio. Responsibility for providing captions falls to the content providers (the callers) or the "host" caller, and not the application.

Automated live captions will have errors if the audio quality is poor or the content is complex resulting in captions provided by a captioner to be manually edited and corrected later on - given that audio transcript was created. The synchronized and searchable transcript will then appear next to the recording video more accurate than live captions.

WCAG 2.2 1.2.9 Audio-only (Live) (AAA)) recommends live caption as real-time audio alternative synchronized with speech and sounds. This service is typically provided via trained human operators. Captions are preferred over transcripts for live events because they adapt to unscripted content and follow the audio's pace. While transcripts may be acceptable for scripted events, they don't meet accessibility needs when audio deviates from the script.

Design Tips

- The annotations allow designers to inform the location of the caption button and the location where the caption text must appear within the video.
- The design should ensure that captions are clearly visible (caption placement, font size, and contrast), synchronized with the video, do not block key visuals.
- Where possible, provide access and control to the trigger to turn the caption on/off. The placement of the captions must be within the video.

UA Assistant

Captions may include speaker identification, non-speech sounds like laughter and door slams. For those working on the captions - possible user assistant roles - should consider including high-quality captions to support not only speech but environmental context, improving comprehension. These must be included in the captions:

All spoken dialogue.

Speaker identification (especially in multi-speaker videos).



Relevant non-speech audio cues (e.g., [Applause], [Dramatic music], [Phone ringing]).

Caption vs Transcript

Use captions when designing media players, video components, or live streaming tools — include a toggle in your UI and space for captions near the video.

Use transcripts in supporting documentation, below or next to media content, especially for podcasts, training materials, and legal content.

For more details consult the comparison table below.

Table 10: Comparison table with key features for caption and transcript.

Feature	Caption	Transcript
Definition	Text displayed on screen that represents audio content in real-time, synchronized with video.	Text Document that contains the full dialogue and audio content – don't have to be time-synchronized with a video.
Purpose	To provide real-time access to spoken dialogue, sound effects, and music cues during a video.	To allow users to read or search through the content at their own pace, often for reference or review.
Media Type	Video with audio (transcript is optional)	Audio-only
Includes speaker identification	Yes, often	Yes
Supports real-time access	Yes	Yes (but less immediate)
Time-synced to media	Yes	Sometimes, often not precisely
Includes non-speech sounds	Yes (e.g., [laughter], [door slams])	Rarely
Displayed with media	On screen, during playback	Separate from the media (can be downloaded and viewable as text)
Who benefits	Deaf and hard of hearing users, non-native speakers, anyone in noisy or silent environments	Users who are Deafblind (screen-reader users), people who prefer reading, researchers, or those seeking quick reference



Examples

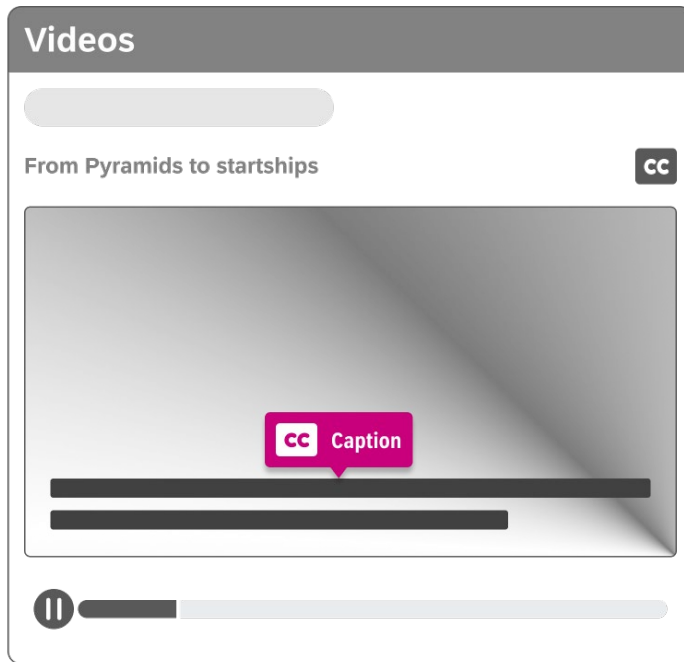


Figure 194: Annotate where and how the captions withing the video will appear. Observe placement, text size and contrast.

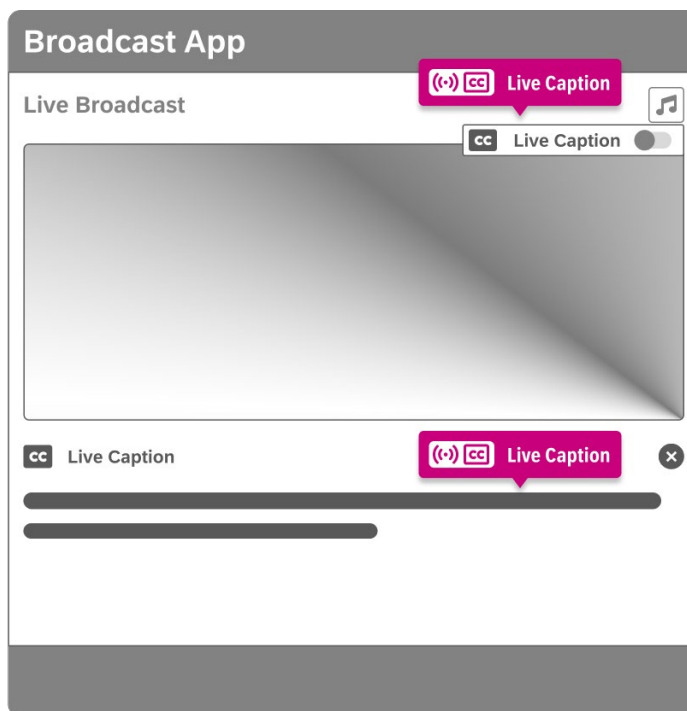


Figure 195: Extra services or functionalities that could be made available are move caption window on the screen, pin or dock the caption window, and translation that could be delivered with live caption.

Shared Benefits



Temporary Disabilities

Everyone Benefits

Captions aren't just for Deaf or Hard of Hearing users—they benefit everyone. In noisy settings like conferences or public transportation, captions make it possible to follow videos without missing important information. In quiet environments like libraries or shared workspaces, they let users engage with content discreetly.

During meetings, events, or webinars, live captions help non-native speakers keep up with fast speech, unfamiliar accents, or technical jargon—making communication more inclusive for all. Live captions is also helpful in conferences, events, and broadcasts, for non-native speakers to follow real-time conversation, if the speaker talks fast or uses unfamiliar terms.



Limited Cognition

Cognitive

Captions can improve comprehension and focus for users with cognitive disabilities, including ADHD and neurodivergence and anyone who process text better than audio. Reading along with spoken content can help reinforce memory, clarify meaning, and reduce cognitive load.

Captions also provide a visual buffer for users who are sensitive to sound or easily distracted by background noise, enabling them to engage with media without being overwhelmed by sudden or repetitive audio. When paired with system preferences like “Reduce Motion” or “Do Not Disturb”, captions contribute to a more comfortable and focused experience.

Developer Notes

Use framework inventory components linking to respective caption resources on the platform.



References

WCAG 2.2 Reference

[1.2.2 Captions \(Prerecorded\)](#) (A)

[1.2.4 Captions \(Live\)](#) (AA)

[1.2.9 Audio-only \(Live\)](#) (AAA)

SAP Product Standard Accessibility 4.3

[ACC 279 - Animation, Audio and Video Control](#)



Transcript

Offer text transcripts for all audio-only content to support users who prefer reading or use screen readers. Transcripts should include spoken dialogue and relevant sound cues when necessary.

Variants

Transcript as an audio media alternative



Figure 196: Informs written text alternatives for audio content (speech, sounds, etc.)

Live Transcript as an audio media alternative



Figure 197: Informs written text alternatives for audio content (speech, sounds, etc.)

About Transcript

People who are Deaf or Hard of Hearing rely on visual alternatives to understand audio-based content. Transcripts are text-based documents provided as an alternative version for audio content including dialog speech and meaningful sounds. They are used for pre-recorded or live media like interviews, podcasts, and webinars, allowing users to read, search, and reference the information at their own pace—making the experience more inclusive for everyone.

Pre-recorded transcripts support users who need complete control over timing, like those who read transcripts at their own pace. They're particularly useful when users need to skim, search, or copy text from a media source. There are two types of transcripts:

- **Plain Transcript:** Covers all spoken dialogue or narration ensuring that users who cannot hear can still access the full meaning of the content. It does not include visual descriptions or audio cues, and it's mostly used on podcasts, interviews, and audio messages. Reference for audio only: WCAG 2.2 SC 1.2.1 Audio-only and Video-only (Prerecorded) (A).



- **Full Description Transcript:** Includes spoken dialogue plus detailed visual descriptions. It acts as a comprehensive alternative to video media including those who are deaf and blind. The benefits for those who cannot see the video (blind and low vision users) is the description of the scenes available in the transcript. Example: “The presenter picks up a pen and pretends to draw a circle over the diagram.” Reference for video with visuals: WCAG 2.2 SC 1.2.3 Audio Description or Media Alternative (Prerecorded) (A) and SC 1.2.8 Audio Description or Media Alternative (Prerecorded) (AAA).

Live transcript (for Audio-only or Video content)

Live transcripts provide a real-time, running text version of live audio content. They support Deaf, Hard of Hearing, non-native speakers, and users in noisy environments to follow the content as it unfolds. Unlike captions, they are not strictly time-synced, may include speaker names, and can omit sound effects. They are especially useful when live captioning is not available or practical.

Live transcripts may:

- Be displayed alongside the live media
- Include dialogue, and sometimes speaker names
- Not be strictly synchronized like captions
- Usually do not include sound effects or visual cues unless added manually

Live transcripts are helpful when full captioning is not feasible. It remains a valuable feature for real-time understanding, reviewing, or skimming. Because it is more flexible in terms of format and delivery, it may lag slightly more than captions.

Design tips for annotating transcripts

- Add transcript access points using clear buttons or links labeled “Transcript” that are easy to find.
- Placement of transcripts are recommended below video or player, near the link or the button media, in a collapsible section, as a download link or beside the media player.
- Label transcript links or buttons clearly (e.g., “Transcript”)
- Let users control playback pace and transcript scrolling (when possible).
- For live content, offer real-time text via a visible transcript panel or chat-style display

When audio-only live content happens, provide a real-time transcript, live Developer Notes, or chat-based summaries. If possible, offer a text



alternative channel for participation (e.g., Q&A in chat). Give users control over pace allowing the user to pause or replay the audio if using streaming platforms that support it. But most important, offer post-event captioned recordings or full transcripts.

User Assistant roles

Provide Clear Transcripts to enable users to read content at their own pace without relying on sound. Indicate if the transcript includes speaker names or non-verbal cues. Observe a comprehensive coverage of spoken content and audio descriptions, file format, or download availability. Cover visual elements described in the media.



Examples

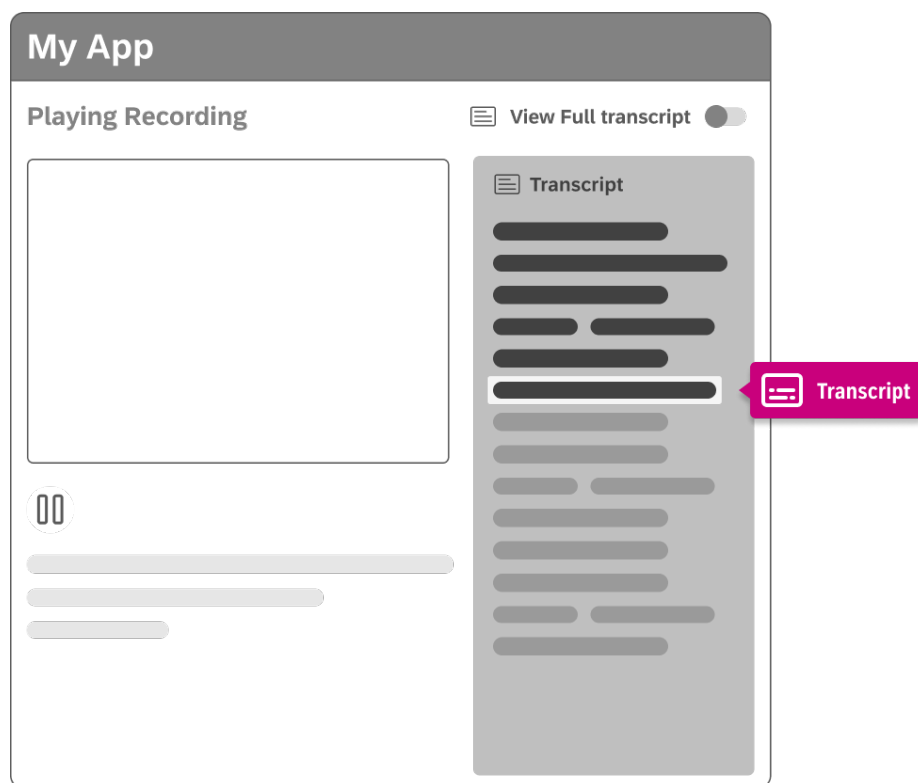


Figure 198: Pre-recorded audio with an option to show transcript

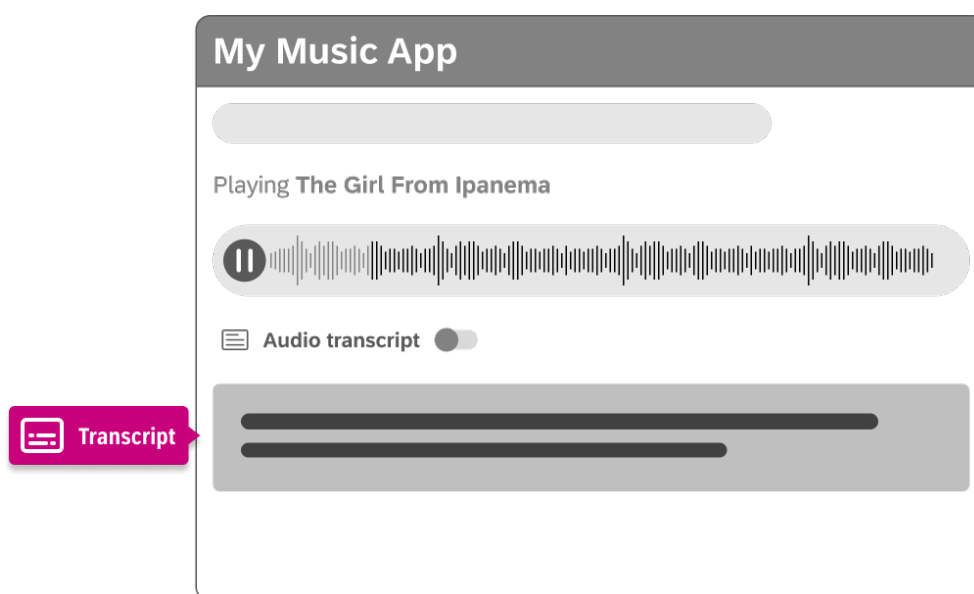


Figure 199: Pre-recorded audio with an option to show transcript

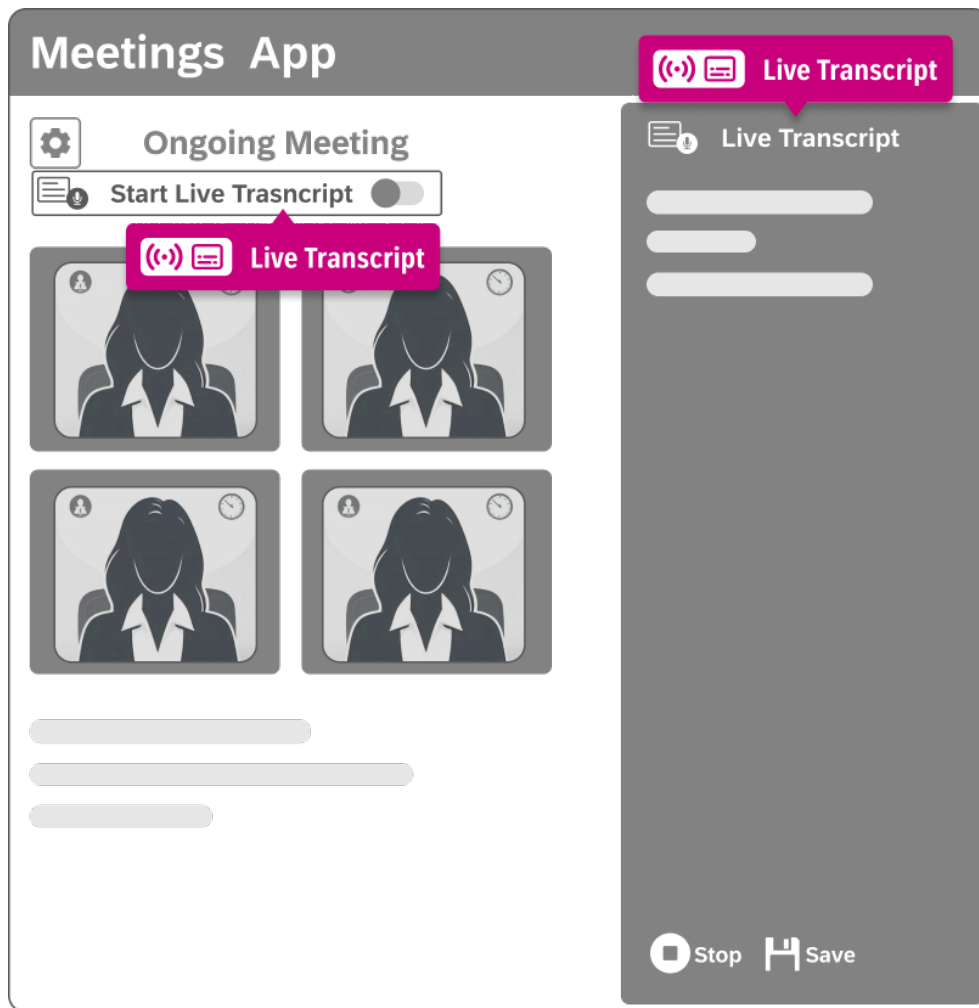


Figure 200: Manage the live transcript window
Functionalities such as stop and save are available to the user.

Shared Benefits



Temporary Disabilities

Everyone Benefits

Transcripts are a valuable resource for everyone—not just for accessibility. They make it easy to skim or search through audio or video content to find specific information quickly, which is especially useful for busy professionals, students, or researchers.

In meetings or webinars, transcripts offer a convenient way to review what was said, catch up if something was missed, or reference quotes accurately.

They're also helpful for people in quiet environments or those who prefer reading over listening. Live transcripts help non-native speaker's people in conferences, events, and broadcasts to follow real-time conversation, if the speaker talks fast or uses unfamiliar terms



Without Vision

Screen Reading

While Blind users may not benefit from captions, transcripts offer a scannable, screen-reader-friendly alternative to audio content. A well-structured transcript—especially one enhanced with headings, timecodes, or ARIA landmarks—allows users to navigate, search, or review the material without needing to listen from start to finish.

Transcripts also minimize reliance on complex sound-based navigation, which can conflict with screen reader output. When audio cues (like chimes or voice prompts) are used, they should be clearly documented, optional, and timed to avoid overlap with assistive technology feedback. In summary, transcripts empower Blind users with greater control, efficiency, and independence when accessing media content.



Limited Cognition

Cognitive

Transcripts support users with cognitive or learning disabilities by allowing them to process content at their own pace. The ability to scan, reread, highlight, or translate written information helps users retain key ideas and manage attention.

For individuals who experience anxiety, language processing challenges, or information overload, transcripts offer a calm, predictable alternative to listening. They also promote task continuity, allowing users to return to key information when needed—without the pressure of real-time comprehension.

Developer Notes

Use framework inventory components linking to respective transcript resources on the platform.

References

WCAG 2.2 Reference

[1.2.1 Audio-only and video-only \(Prerecorded\)](#) (A)

[1.2.3 Audio Description or Media Alternative \(Prerecorded\)](#) (A)

[1.2.8 Media Alternative \(Prerecorded\)](#) (AAA)

SAP Product Standard Accessibility 4.3

[ACC 279 - Animation, Audio and Video Control](#)



Closing Remarks

Knowledge is of no value unless you put it into practice.

We hope you found this updated guideline helpful. If you've made it this far, congratulations—your journey toward accessible design is well underway. Now it's your turn to lead the way.

Craw. Walk. Run. Iterate,

We know that building internal alignment isn't always easy. Like any complex practice, accessibility in design demands intention, planning, collaboration, and persistence. Don't let obstacles slow you down, start small, stay focused, and keep learning.

Always remember accessibility is amplified usability. If you're feeling like a team of one, start with what's in your control.

Choose a few meaningful annotations and embed them into your current design process. Every element you document, label, or clarify brings real value to the user experience.

This could be the spark that changes how your team thinks about inclusion. This could be the shift that inspires a broader conversation across your organization. Be patient, be persistent—and above all, stay curious.

Let this edition be your toolkit. We're excited to see what you do next.

Irla, Nina and Stefan



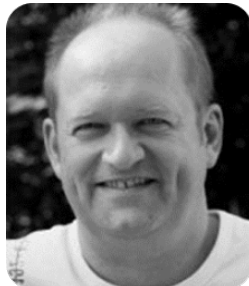
Authors



Irla Rebelo

User Experience Design Expert and Accessibility Lead
SAP Success Factors

Montreal Canada



Stefan Schnabel

User Experience Design Expert
SAP Accessibility and Inclusive Design

Walldorf Germany



Nina Krauss

Accessibility Specialist
SAP Accessibility and Inclusive Design

Walldorf Germany

Contributors

Kimberly McGee – Reviewer

Senior Equity Strategist, SAP Inclusive Design

Dominika Zamojska

UX Designer and Accessibility Lead CX UX

Ryan Lum

UX Designer Concur

Manuel Rodrigues

UX Designer CX UX

Georg Christoph

UX Designer SBN UX Project

Nakul Saxena

UX Designer SuccessFactors

Paul Hopkins

UX Designer SuccessFactors

Liu Joy

UX Designer SuccessFactors



This entire Figma file is internal only

© 2025 SAP SE or an SAP affiliate company. All rights reserved. See Legal Notice on www.sap.com/legal-notice for use terms, disclaimers, disclosures, or restrictions related to this material.



Appendix

WCAG 2.2 Coverage

SAP created this set of annotations in alignment with the Web Content Accessibility Guidelines (WCAG) version 2.2 from W3C.

The WCAG 2.2 guideline is divided into 4 groups of Success Criterion and 78 success criteria used by many organizations to create accessibility reports.

This documentation covers all A and AA success criteria and some AAA.

Annotation Symbols



Foundation



Visual Experience



Interaction Experience



Screen Reader Experience



Cognitive Experience



Auditory Experience



Perceivable

Information and user interface components must be presentable to users in ways they can perceive.

1.1 Text Alternative

[1.1.1 Non-Text Content](#) (A)



1.2 Time-Based Media

[1.2.1 Audio-only and video-only \(Prerecorded\)](#) (A)



[1.2.2 Captions \(Prerecorded\)](#) (A)



[1.2.3 Audio Description or Media Alternative \(Prerecorded\)](#) (A)



[1.2.4 Captions \(Live\)](#) (AA)



[1.2.5 Audio Description \(Prerecorded\)](#) (AA)



[1.2.6 Sign Language \(Prerecorded\)](#) (AAA)

no annotation

[1.2.7 Extended Audio Description \(Prerecorded\)](#) (AAA)



[1.2.8 Media Alternative \(Prerecorded\)](#) (AAA)



[1.2.9 Audio-only \(Live\)](#) (AAA)



1.3 Adaptable

[1.3.1 Info and Relationships](#) (A)



[1.3.2 Meaningful Sequence](#) (A)



[1.3.3 Sensory Characteristics](#) (A)



[1.3.4 Orientation](#) (AA)



[1.3.5 Identify Input Purpose](#) (AA)



[1.3.6 Identify Purpose](#) (AAA)



1.4 Distinguishable

[1.4.1 Use of Color](#) (A)



[1.4.2 Audio Control](#) (A)



[1.4.3 Contrast \(Minimum\)](#) (AA)



[1.4.4 Resize text](#) (AA)



[1.4.5 Images of Text](#) (AA)



[1.4.6 Contrast \(Enhanced\)](#) (AAA)

no annotation

[1.4.7 Low or No Background Audio](#) (AAA)

no annotation

[1.4.8 Visual Presentation](#) (AAA)



[1.4.9 Images of Text \(No Exception\)](#) (AAA)

no annotation

[1.4.10 Reflow](#) (AA)



[1.4.11 Non-text Contrast](#) (AA)



[1.4.12 Text Spacing](#) (AA)



[1.4.13 Content on Hover or Focus](#) (AA)



Operable

User interface components and navigation must be operable.

2.1 Keyboard Accessible

[2.1.1 Keyboard](#) (A)



[2.1.2 No Keyboard Trap](#) (A)



[2.1.3 Keyboard \(No Exception\)](#) (AAA)



[2.1.4 Character Key Shortcuts](#) (A)



2.2 Enough Time

[2.2.1 Timing Adjustable](#) (A)



[2.2.2 Pause, Stop, Hide](#) (A)



[2.2.3 No Timing](#) (AAA)



[2.2.4 Interruptions](#) (AAA)



[2.2.5 Re-Authenticating](#) (AAA)

no annotation

[2.2.6 Timeouts](#) (AAA)



2.3 Seizures and Physical Reactions

[2.3.1 Three Flashes or Below Threshold](#) (A)



[2.3.2 Three Flashes](#) (AAA)



[2.3.3 Animation from Interactions](#) (AAA)

no annotation

2.4 Navigable

[2.4.1 Bypass Blocks](#) (A)



[2.4.2 Page Title](#) (A)



[2.4.3 Focus Order](#) (A)



[2.4.4 Link Purpose \(In Context\)](#) (A)



[2.4.5 Multiple Ways](#) (AA)



[2.4.6 Headings and Labels](#) (AA)



[2.4.7 Focus Visible](#) (AA)



[2.4.8 Location](#) (AAA)



[2.4.9 Link Purpose \(Link Only\)](#) (AAA)



[2.4.10 Section Headings](#) (AAA)



[2.4.11 Focus Not Obscured \(Minimum\)](#) (AA)



[2.4.12 Focus Not Obscured \(Enhanced\)](#) (AAA)



[2.4.13 Focus Appearance](#) (AAA)



2.5 Input Modalities

[2.5.1 Pointer Gestures](#) (A)



[2.5.2 Pointer Cancellation](#) (A)



[2.5.3 Label in Name](#) (A)



[2.5.4 Motion Actuation](#) (A)



[2.5.5 Target Size](#) (AAA)



[2.5.6 Concurrent Input Mechanisms](#) (AAA)



[2.5.7 Dragging Movements](#) (AA)



[2.5.8 Target Size \(Minimum\)](#) (AA)



Understandable

Information and the operation of user interface must be understandable.

3.1 Readable

[3.1.1 Language of Page](#) (A)



[3.1.2 Language of Parts](#) (AA)



[3.1.3 Unusual Words](#) (AAA)



[3.1.4 Abbreviations](#) (AAA)



[3.1.5 Reading Level](#) (AAA)



[3.1.6 Pronunciation](#) (AAA)

no annotation

3.2 Predictable

[3.2.1 On Focus](#) (A)



[3.2.2 On Input](#) (A)



[3.2.3 Consistent Navigation](#) (AA)



[3.2.4 Consistent Identification](#) (AA)



[3.2.5 Change on Request](#) (AAA)

no annotation

[3.2.6 Consistent Help](#) (A)



3.3 Input Assistance

[3.3.1 Error Identification](#) (A)



[3.3.2 Labels or Instructions](#) (A)



[3.3.3 Error Suggestion](#) (AA)



[3.3.4 Error Prevention \(Legal, Financial, Data\)](#) (AA)

[3.3.5 Help](#) (AAA)

no annotation

[3.3.6 Error Prevention \(all\)](#) (AAA)

no annotation



[3.3.7 Redundant Entry \(A\)](#)[3.3.8 Accessible Authentication \(Minimum\) \(AA\)](#)

no annotation

[3.3.9 Accessible Authentications \(Enhanced\) \(AAA\)](#)

no annotation

Robust

Content must be robust enough that it can be interpreted by a wide variety of user agents, including assistive technologies.

4.1 Compatible

[4.1.1 Parsing \(A\)](#)

(obsolete in WCAG 2.2 and removed)

[4.1.2 Name, Role, Value \(A\)](#)[4.1.3 Status Messages \(AA\)](#)

Conformance

For a web page to conform to WCAG 2.2, all of the following conformance requirements must be satisfied:

5.1 Interpreting Normative Requirements

[5.1 Interpreting Normative Requirements](#)

no annotation

5.2 Conformance Requirements Conformance Claims (Optional)

[5.2.1 Conformance Level](#)

no annotation

[5.2.2 Full Pages](#)

no annotation

[5.2.3 Complete Process](#)

no annotation

[5.2.4 Only Accessibility Supported Ways of Using Technologies](#)

no annotation

[5.2.5 Non-Interference](#)

no annotation

5.3 Conformance Claims (Optional)

[5.3.1 Required Components of a Conformance Claim](#)

no annotation

[5.3.2 Optional Components of a Conformance Claim](#)

no annotation

5.4 Statement of Partial Conformance - Third Party Content

[5.4 Statement of Partial Conformance - Third Party Content](#)

5.5 Statement of Partial Conformance – Language

[5.5 Statement of Partial Conformance - Language](#)

no annotation



5.6 Privacy Considerations

[5.6 Privacy Considerations](#)

no annotation

5.7 Security Considerations

[5.7 Security Considerations](#)

no annotation

